

LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE



Oleh:

Muhammad Adh-Dhiya'Us Salim

NIM. 2310817210022

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Adh-Dhiya'Us Salim
NIM : 2310817210022

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom.
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	5
DAFTAR TABEL.....	7
MODUL 1 : ANDROID BASIC WITH KOTLIN	9
SOAL 1	9
A. Source Code	11
B. Output Program.....	16
C. Pembahasan.....	18
D. Tautan Git	23
MODUL 2 : Android Layout With Compose.....	24
SOAL 1	24
A. Source Code	25
B. Output Program.....	32
C. Pembahasan.....	36
D. Tautan Git	40
SOAL 2	41
A. Jawaban.....	41
MODUL 3 : Build a Scrollable List.....	43
SOAL 1	43
A. Source Code	45
B. Output Program.....	65
C. Pembahasan.....	67
D. Tautan Git	72
SOAL 2	73
A. Jawaban.....	73
MODUL 4 : ViewModel and Debugging	75
SOAL 1	75
A. Source Code	76
B. Output Program.....	99
C. Pembahasan.....	101
Tautan Git	107
SOAL 2	108

A. Jawaban.....	108
MODUL 5 : Connect to the Internet.....	110
SOAL 1	110
A. Source Code	110
B. Output Program.....	126
C. Pembahasan.....	128
Tautan Git	136

DAFTAR GAMBAR

MODUL 1 : ANDROID BASIC WITH KOTLIN

Gambar 1 Tampilan Awal Aplikasi.....	9
Gambar 2 Tampilan Dadu Setelah Di Roll	10
Gambar 3 Tampilan Roll Dadu Double	11
Gambar 4 Screenshot Hasil Jawaban Soal 1 - Compose	16
Gambar 5 Screenshot Hasil Jawaban Soal 1 - Compose	16
Gambar 6 Screenshot Hasil Jawaban Soal 1 - XML.....	17
Gambar 7 Screenshot Hasil Jawaban Soal 1 - XML.....	17

MODUL 2 : Android Layout With Compose

Gambar 8 Tampilan Awal Aplikasi	24
Gambar 9 Tampilan Pilihan Persentasi Tip	24
Gambar 10 Tampilan Aplikasi Setelah Dijalanmkan	25
Gambar 11 Screenshot Hasil Jawaban Soal 1 - Compose	32
Gambar 12 Screenshot Hasil Jawaban Soal 1 - Compose	32
Gambar 13 Screenshot Hasil Jawaban Soal 1 - Compose	33
Gambar 14 Screenshot Hasil Jawaban Soal 1 – Compose.....	33
Gambar 15 Screenshot Hasil Jawaban Soal 1 - XML.....	34
Gambar 16 Screenshot Hasil Jawaban Soal 1 - XML.....	34
Gambar 17 Screenshot Hasil Jawaban Soal 1 - XML.....	35
Gambar 18 Screenshot Hasil Jawaban Soal 1 - XML.....	35

MODUL 3 : Build a Scrollable List

Gambar 19 Contoh UI List	44
Gambar 20 Contoh UI Detail.....	44
Gambar 21 Screenshot Hasil Jawaban Soal 1 - Compose	65
Gambar 22 Screenshot Hasil Jawaban Soal 1 - Compose	65
Gambar 23 Screenshot Hasil Jawaban Soal 1 - XML.....	66
Gambar 24 Screenshot Hasil Jawaban Soal 1 – XML.....	66

MODUL 4 : ViewModel and Debugging

Gambar 25 Contoh Penggunaan Debugger.....	76
Gambar 26 Screenshot Hasil Jawaban Soal 1 - Compose	99
Gambar 27 Screenshot Hasil Jawaban Soal 1 - Compose	99
Gambar 28 Screenshot Penggunaan Timber Soal 1 - Compose	100
Gambar 29 Screenshot Hasil Jawaban Soal 1 - XML.....	100
Gambar 30 Screenshot Hasil Jawaban Soal 1 - XML.....	101
Gambar 31 Screenshot Penggunaan Timber Soal 1 – XML.....	101

MODUL 5 : Connect to the Internet

Gambar 32 Screenshot Hasil Jawaban Soal 1	126
Gambar 33 Screenshot Hasil Jawaban Soal 1	126
Gambar 34 Screenshot Hasil Jawaban Soal 1	127

DAFTAR TABEL

MODUL 1 : ANDROID BASIC WITH KOTLIN

Table 1 Source Code Soal 1 - Compose	14
Table 2 Source Code Soal 1 - XML.....	14
Table 3 Source Code Soal 1 - XML.....	15

MODUL 2 : Android Layout With Compose

Table 4 Source Code Soal 1 - Compose	28
Table 5 Source Code Soal 1 - XML.....	30
Table 6 Source Code Soal 1 - XML.....	31

MODUL 3 : Build a Scrollable List

Table 7 Source Code Soal 1 - Compose	47
Table 8 Source Code Soal 1 - Compose	47
Table 9 Source Code Soal 1 - Compose	48
Table 10 Source Code Soal 1 - Compose	48
Table 11 Source Code Soal 1 - Compose	49
Table 12 Source Code Soal 1 - Compose	52
Table 13 Source Code Soal 1 - Compose	52
Table 14 Source Code Soal 1 - XML.....	54
Table 15 Source Code Soal 1 - XML.....	54
Table 16 Source Code Soal 1 - XML.....	55
Table 17 Source Code Soal 1 - XML.....	56
Table 18 Source Code Soal 1 - XML.....	57
Table 19 Source Code Soal 1 - XML.....	58
Table 20 Source Code Soal 1 - XML.....	58
Table 21 Source Code Soal 1 - XML.....	59
Table 22 Source Code Soal 1 - XML.....	60
Table 23 Source Code Soal 1 - XML.....	61
Table 24 Source Code Soal 1 - XML.....	63
Table 25 Source Code Soal 1 - XML.....	64
Table 26 Source Code Soal 1 – XML.....	64

MODUL 4 : ViewModel and Debugging

Table 27 Source Code Soal 1 - Compose	78
Table 28 Source Code Soal 1 - Compose	78
Table 29 Source Code Soal 1 - Compose	79
Table 30 Source Code Soal 1 - Compose	80
Table 31 Source Code Soal 1 - Compose	81
Table 32 Source Code Soal 1 - Compose	83
Table 33 Source Code Soal 1 - Compose	84
Table 34 Source Code Soal 1 - Compose	84

Table 35 Source Code Soal 1 - Compose	85
Table 36 Source Code Soal 1 - Compose	85
Table 37 Source Code Soal 1 - Compose	85
Table 38 Source Code Soal 1 - XML.....	87
Table 39 Source Code Soal 1 - XML.....	87
Table 40 Source Code Soal 1 - XML.....	88
Table 41 Source Code Soal 1 - XML.....	89
Table 42 Source Code Soal 1 - XML.....	91
Table 43 Source Code Soal 1 - XML.....	92
Table 44 Source Code Soal 1 - XML.....	92
Table 45 Source Code Soal 1 - XML.....	93
Table 46 Source Code Soal 1 - XML.....	93
Table 47 Source Code Soal 1 - XML.....	95
Table 48 Source Code Soal 1 - XML.....	96
Table 49 Source Code Soal 1 - XML.....	97
Table 50 Source Code Soal 1 - XML.....	98
Table 51 Source Code Soal 1 - XML.....	98

MODUL 5 : Connect to the Internet

Table 52 Source Code Soal 1	111
Table 53 Source Code Soal 1	111
Table 54 Source Code Soal 1	112
Table 55 Source Code Soal 1	112
Table 56 Source Code Soal 1	113
Table 57 Source Code Soal 1	114
Table 58 Source Code Soal 1	115
Table 59 Source Code Soal 1	115
Table 60 Source Code Soal 1	116
Table 61 Source Code Soal 1	117
Table 62 Source Code Soal 1	119
Table 63 Source Code Soal 1	122
Table 64 Source Code Soal 1	123
Table 65 Source Code Soal 1	124
Table 66 Source Code Soal 1	124
Table 67 Source Code Soal 1	125
Table 68 Source Code Soal 1	125

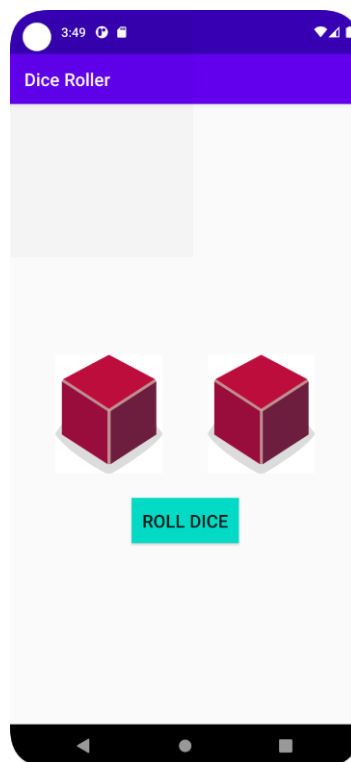
MODUL 1 : ANDROID BASIC WITH KOTLIN

SOAL 1

Soal Praktikum:

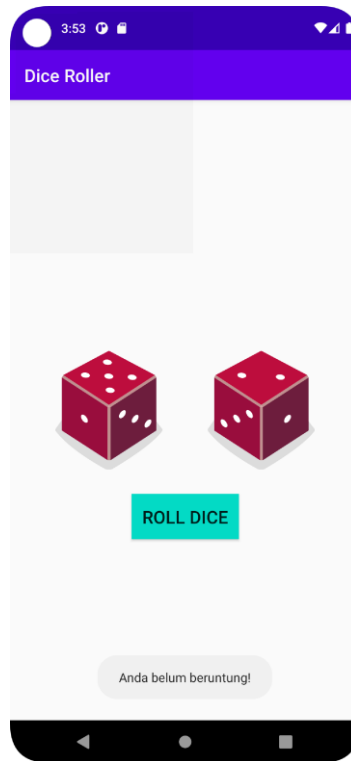
Buatlah sebuah aplikasi yang dapat menampilkan 2 (dua) buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll Dice”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



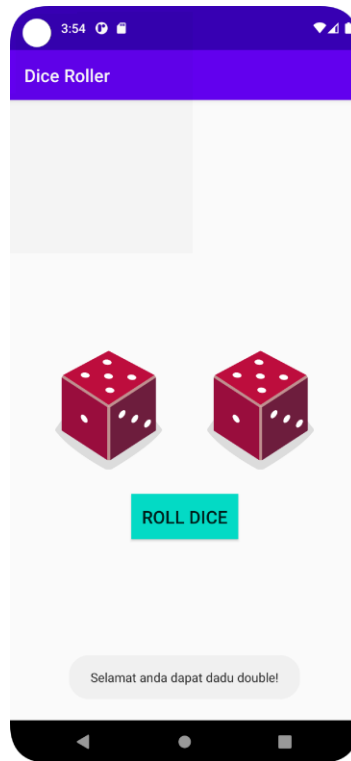
Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll Dice” maka masing-masing dadu akan memunculkan sisi dadu masing-masing dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2 maka akan menampilkan pesan “Anda belum beruntung!” seperti dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat anda dapat dadu double!” seperti dapat dilihat pada Gambar 3.
4. Upload aplikasi yang telah anda buat kedalam repository github ke dalam **folder Module 2 dalam bentuk project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repo.
5. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/u/0/uc?id=147HT2lIH5qin3z5ta7H9y2N_5OMW81Ll&export=download



Gambar 3 Tampilan Roll Dadu Double

A. Source Code

MainActivity.kt / Compose

```

1 package com.example.rolldadu
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.Image
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.res.painterResource
13 import androidx.compose.ui.unit.dp
14 import com.example.rolldadu.ui.theme.RollDaduTheme
15 import kotlin.random.Random
16 import androidx.compose.ui.graphics.Color
17 import kotlinx.coroutines.launch
18 import androidx.compose.material3.SnackbarHost
19 import androidx.compose.material3.SnackbarHostState
20 import androidx.compose.material3.Scaffold
21 import androidx.compose.material3.TopAppBar
22 import androidx.compose.material3.TopAppBarDefaults
23
24
25 class MainActivity : ComponentActivity() {

```

```

26     override fun onCreate(savedInstanceState: Bundle?) {
27         super.onCreate(savedInstanceState)
28         setContent
29             RollDaduTheme
30             Surface(
31                 modifier = Modifier.fillMaxSize(),
32                 color = Color.Black
33             )
34             DiceRollerApp()
35         }
36     }
37 }
38 }
39 }
40
41 @OptIn(ExperimentalMaterial3Api::class)
42 @Composable
43 fun DiceRollerApp() {
44     val snackbarHostState = remember { SnackbarHostState() }
45     val scope = rememberCoroutineScope()
46
47     var dice1 by remember { mutableStateOf(0) }
48     var dice2 by remember { mutableStateOf(0) }
49
50     val message = when {
51         dice1 == 0 && dice2 == 0 -> ""
52         dice1 == dice2 -> "Selamat, anda dapat dadu double!"
53         else -> "Anda belum beruntung!"
54     }
55
56     Scaffold(
57         modifier = Modifier.fillMaxSize(),
58         containerColor = Color.Black,
59         snackbarHost = {
60             SnackbarHost(hostState = snackbarHostState)
61         },
62         topBar = {
63             TopAppBar(
64                 title = {
65                     Text(
66                         text = "Dice Roller",
67                         color = Color.White
68                     )
69                 },
70                 colors = TopAppBarDefaults.topAppBarColors(
71                     containerColor = Color(0xFF6A1B9A)
72                 )
73             )
74         }
75     ) {
76         Column(
77             modifier = Modifier
78                 .fillMaxSize()
79                 .padding(padding)
80                 .padding(16.dp),
81             verticalArrangement = Arrangement.Center,

```

```

82         horizontalAlignment = Alignment.CenterHorizontally
83     )
84     Row(
85         modifier = Modifier.fillMaxWidth(),
86         horizontalArrangement =
87     Arrangement.SpaceEvenly
88     )
89         DiceImage(dice1)
90         DiceImage(dice2)
91     }
92
93     Spacer(modifier = Modifier.height(24.dp))
94
95     Button(
96         onClick = {
97         val newDice1 = Random.nextInt(1, 7)
98         val newDice2 = Random.nextInt(1, 7)
99
100         dice1 = newDice1
101         dice2 = newDice2
102
103         val resultMessage = if (newDice1 ==
104 newDice2) {
105             "Selamat, anda dapat dadu double!"
106         } else {
107             "Anda belum beruntung!"
108         }
109
110         scope.launch {
111
112
113         snackbarHostState.currentSnackbarData?.dismiss()
114
115         snackbarHostState.showSnackbar(resultMessage)
116         }
117     },
118     colors = ButtonDefaults.buttonColors(
119         containerColor = Color(0xFF6A1B9A)
120     )
121 )
122     Text("Roll Dice", color = Color.White)
123 }
124 }
125 }
126 }
127 }
128
129
130 @Composable
131 fun DiceImage(diceValue: Int) {
132     val imageRes = when (diceValue) {
133         1 -> R.drawable.dice_1
134         2 -> R.drawable.dice_2
135         3 -> R.drawable.dice_3
136         4 -> R.drawable.dice_4
137         5 -> R.drawable.dice_5
138         6 -> R.drawable.dice_6
139         else -> R.drawable.dice_0

```

140	}
141	
142	Image (
143	painter = painterResource(id = imageRes),
144	contentDescription = "Dice \$diceValue",
145	modifier = Modifier.size(180.dp)
146	)
147	}

Table 1 Source Code Soal 1 - Compose

activity_main.xml / XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:id="@+id/rootLayout"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:background="@android:color/black"
8	android:orientation="vertical"
9	android:gravity="center"
10	android:padding="24dp">
11	
12	<LinearLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:gravity="center"
16	android:orientation="horizontal"
17	android:layout_marginBottom="24dp">
18	
19	<ImageView
20	android:id="@+id/diceImage1"
21	android:layout_width="140dp"
22	android:layout_height="140dp"
23	android:src="@drawable/dice_0" />
24	
25	<Space
26	android:layout_width="16dp"
27	android:layout_height="match_parent" />
28	
29	<ImageView
30	android:id="@+id/diceImage2"
31	android:layout_width="140dp"
32	android:layout_height="140dp"
33	android:src="@drawable/dice_0" />
34	</LinearLayout>
35	
36	<Button
37	android:id="@+id/rollButton"
38	android:layout_width="wrap_content"
39	android:layout_height="wrap_content"
40	android:text="Roll"
41	android:backgroundTint="#6A1B9A"
42	android:textColor="@android:color/white" />
43	</LinearLayout>

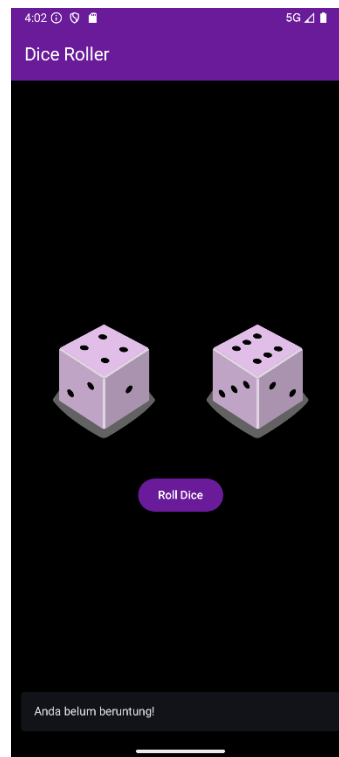
Table 2 Source Code Soal 1 - XML

MainActivity.kt / XML

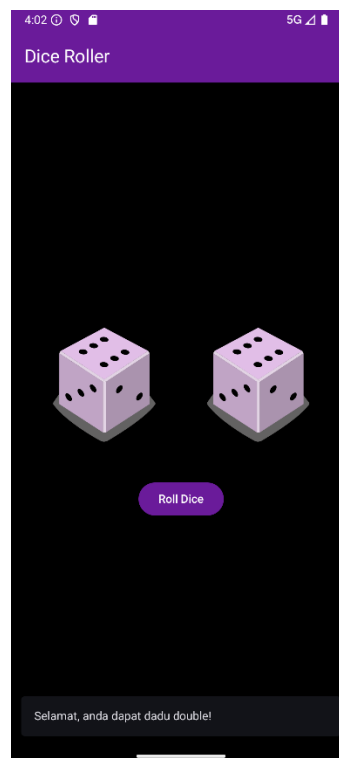
```
1 package com.example.rollxmldadu
2
3 import android.os.Bundle
4 import android.widget.Toast
5 import androidx.appcompat.app.AppCompatActivity
6 import com.example.rollxmldadu.databinding.ActivityMainBinding
7 import kotlin.random.Random
8
9
10 class
11 MainActivity : AppCompatActivity() {
12
13     private lateinit var binding: ActivityMainBinding
14
15     private val diceImages = listOf(
16         R.drawable.dice_0,
17         R.drawable.dice_1,
18         R.drawable.dice_2,
19         R.drawable.dice_3,
20         R.drawable.dice_4,
21         R.drawable.dice_5,
22         R.drawable.dice_6
23     )
24
25     override fun onCreate(savedInstanceState: Bundle?) {
26         super.onCreate(savedInstanceState)
27         binding = ActivityMainBinding.inflate(layoutInflater)
28         setContentView(binding.root)
29
30         binding.rollButton.setOnClickListener {
31             val dice1 = Random.nextInt(6)
32             val dice2 = Random.nextInt(6)
33
34
35             binding.diceImage1.setImageResource(diceImages[dice1])
36
37             binding.diceImage2.setImageResource(diceImages[dice2])
38
39             val message = if (dice1 == dice2) {
40                 "Selamat, anda dapat dadu double!"
41             } else {
42                 "Anda belum beruntung!"
43             }
44
45             Toast.makeText(this, message,
46                 Toast.LENGTH_SHORT).show()
47         }
48     }
49 }
```

Table 3 Source Code Soal 1 - XML

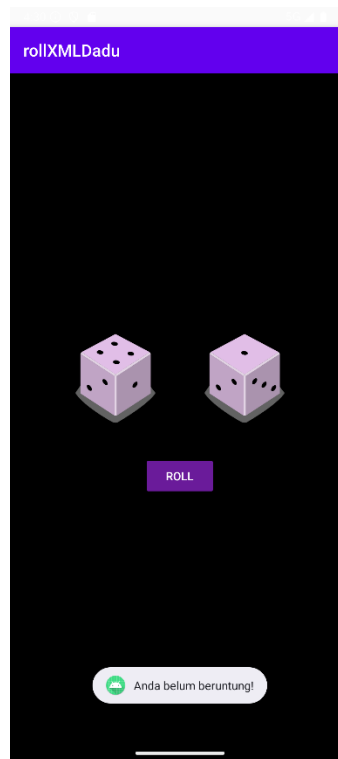
B. Output Program



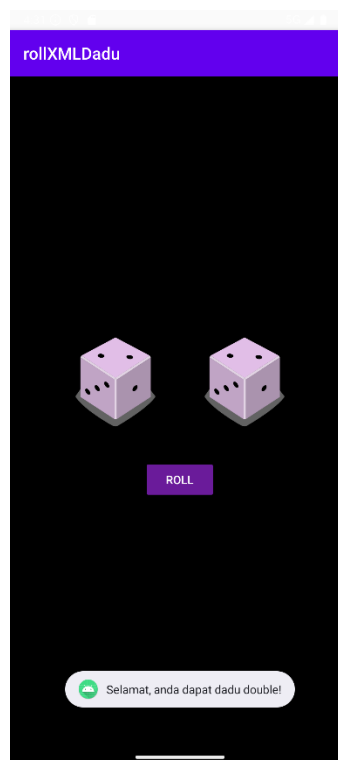
Gambar 4 Screenshot Hasil Jawaban Soal 1 - Compose



Gambar 5 Screenshot Hasil Jawaban Soal 1 - Compose



Gambar 6 Screenshot Hasil Jawaban Soal 1 - XML



Gambar 7 Screenshot Hasil Jawaban Soal 1 - XML

C. Pembahasan

MainActivity.kt / XML :

- **Pada baris 1**, dideklarasikan nama package untuk file Kotlin ini, yang berfungsi sebagai identitas unik lokasinya dalam struktur proyek.
- **Pada baris 3-7**, dilakukan proses import untuk beberapa pustaka yang dibutuhkan:
 - Bundle diimpor untuk menangani penyimpanan data atau state dari activity.
 - Toast diimpor untuk fungsionalitas menampilkan pesan notifikasi singkat di layar.
 - AppCompatActivity diimpor untuk digunakan sebagai kelas dasar (*base class*) sebuah activity agar mendukung fitur-fitur modern di berbagai versi Android.
 - ActivityMainBinding diimpor. Ini adalah kelas yang dibuat otomatis oleh fitur *View Binding* untuk mengakses komponen UI dari file activity_main.xml secara aman dan efisien.
 - Random diimpor dari pustaka standar Kotlin untuk fungsi pembuatan angka acak.
- **Pada baris 10**, dideklarasikan kelas utama bernama MainActivity yang merupakan turunan (*extends*) dari kelas AppCompatActivity.
- **Pada baris 12**, dideklarasikan variabel binding dengan tipe ActivityMainBinding. Variabel ini nantinya akan mereferensikan semua komponen UI pada layout. Penggunaan lateinit menandakan bahwa inisialisasi nilainya akan dilakukan nanti, bukan saat deklarasi.
- **Pada baris 14**, dideklarasikan sebuah List yang tidak bisa diubah (*val*) bernama diceImages. List ini berisi kumpulan referensi ID ke semua gambar dadu (dari dice_0 hingga dice_6) yang tersimpan di dalam folder res/drawable.
- **Pada baris 24**, dilakukan *override* pada fungsi onCreate, yang merupakan fungsi yang pertama kali dipanggil secara otomatis oleh sistem Android saat activity ini dibuat atau dimulai.
- **Pada baris 25**, dipanggil implementasi fungsi onCreate dari kelas induknya (*super.onCreate*) untuk memastikan semua konfigurasi esensial dari sebuah activity dijalankan.
- **Pada baris 26**, variabel binding yang sebelumnya dideklarasikan, kini diinisialisasi dengan cara "mengubah" atau *me-inflate* layout activity_main.xml menjadi sebuah objek binding yang dapat diakses oleh kode Kotlin.
- **Pada baris 27**, ditetapkan tampilan antarmuka (*user interface*) untuk activity ini dengan menggunakan layout akar (*root*) dari objek binding yang telah diinisialisasi.
- **Pada baris 29**, diberikan sebuah *event listener* pada komponen UI rollButton. Ini berarti blok kode yang ada di dalamnya akan dieksekusi setiap kali tombol tersebut diklik oleh pengguna.
- **Pada baris 30**, dideklarasikan variabel lokal dice1 dan langsung diisi dengan sebuah angka integer acak yang dihasilkan oleh Random.nextInt(6), yang akan memberikan nilai dari 0 hingga 5.
- **Pada baris 31**, dideklarasikan variabel lokal dice2 dan diisi dengan sebuah angka integer acak kedua, juga dengan rentang nilai dari 0 hingga 5.

- **Pada baris 33**, diatur sumber gambar (*image resource*) untuk komponen ImageView bernama diceImage1. Gambar yang dipilih diambil dari List diceImages pada indeks yang sesuai dengan nilai acak dari dice1.
- **Pada baris 34**, diatur sumber gambar untuk komponen ImageView diceImage2 berdasarkan nilai acak yang disimpan di variabel dice2.
- **Pada baris 36**, dideklarasikan variabel message yang nilainya akan ditentukan berdasarkan hasil dari sebuah ekspresi kondisional if-else.
- **Pada baris 37**, jika kondisi if pada baris sebelumnya (yaitu dice1 == dice2) terpenuhi, maka variabel message akan diisi dengan teks "Selamat, anda dapat dadu double!".
- **Pada baris 39**, jika kondisi if tidak terpenuhi (masuk ke dalam blok else), maka variabel message akan diisi dengan teks "Anda belum beruntung!".
- **Pada baris 42**, dibuat dan ditampilkan sebuah Toast (pesan pop-up singkat) di layar yang berisi teks yang telah disimpan di dalam variabel message.

activity_main.xml / XML :

1. Layout Utama (LinearLayout)

- **Pada baris 2**, didefinisikan LinearLayout sebagai elemen layout utama atau akar (*root*) dari tampilan ini.
- **Pada baris 3**, atribut android:id="@+id/rootLayout" memberikan identitas unik "rootLayout" pada LinearLayout ini.
- **Pada baris 4**, atribut android:layout_width="match_parent" mengatur agar lebar layout ini mengisi seluruh lebar layar perangkat.
- **Pada baris 5**, atribut android:layout_height="match_parent" mengatur agar tinggi layout ini mengisi seluruh tinggi layar perangkat.
- **Pada baris 6**, atribut android:background="@android:color/black" mengatur warna latar belakang layout menjadi hitam.
- **Pada baris 7**, atribut android:orientation="vertical" mengatur agar semua komponen anak (views) di dalamnya akan disusun secara vertikal dari atas ke bawah.
- **Pada baris 8**, atribut android:gravity="center" mengatur agar semua komponen anak di dalamnya diposisikan di tengah-tengah layout (secara horizontal dan vertikal).
- **Pada baris 9**, atribut android:padding="24dp" memberikan jarak (padding) sebesar 24dp di antara batas layout dengan konten di dalamnya.

2. Layout Penampung Dadu (LinearLayout)

- **Pada baris 11**, didefinisikan LinearLayout kedua yang berfungsi untuk menampung dua gambar dadu.
- **Pada baris 12**, lebarnya diatur match_parent agar mengisi lebar dari layout induknya.
- **Pada baris 13**, tingginya diatur wrap_content yang berarti tingginya akan menyesuaikan dengan tinggi konten di dalamnya (dalam hal ini, tinggi gambar dadu).
- **Pada baris 14**, android:gravity="center" digunakan untuk memposisikan komponen anak (dua gambar dadu) di tengah-tengah layout ini secara horizontal.
- **Pada baris 15**, android:orientation="horizontal" mengatur agar komponen di dalamnya disusun secara horizontal dari kiri ke kanan.
- **Pada baris 16**, android:layout_marginBottom="24dp" memberikan jarak bawah (margin) sebesar 24dp antara layout ini dengan komponen di bawahnya (yaitu tombol "Roll").

3. Komponen Gambar Dadu (ImageView dan Space)

- **Pada baris 18**, didefinisikan sebuah ImageView untuk menampilkan gambar dadu pertama.
- **Pada baris 19**, `android:id="@+id/diceImage1"` memberikan ID unik "diceImage1" agar bisa dimanipulasi dari kode Kotlin.
- **Pada baris 20 & 21**, `android:layout_width` dan `android:layout_height` mengatur ukuran gambar menjadi 140dp x 140dp.
- **Pada baris 22**, `android:src="@drawable/dice_0"` mengatur gambar awal (default) yang akan ditampilkan adalah `dice_0.xml` dari folder `drawable`.
- **Pada baris 24**, didefinisikan komponen `Space` yang berfungsi untuk memberikan jarak atau ruang kosong horizontal di antara dua gambar dadu. Lebarnya diatur 16dp.
- **Pada baris 29**, didefinisikan ImageView kedua dengan ID `diceImage2`, dengan konfigurasi ukuran dan gambar awal yang sama dengan ImageView pertama.

4. Tombol Aksi (Button)

- **Pada baris 37**, didefinisikan sebuah komponen `Button`.
- **Pada baris 38**, `android:id="@+id/rollButton"` memberikan ID unik "rollButton" untuk tombol ini, yang digunakan untuk mendeteksi event klik di kode Kotlin.
- **Pada baris 39 & 40**, `android:layout_width` dan `android:layout_height` diatur `wrap_content`, yang berarti ukuran tombol akan menyesuaikan dengan ukuran teks di dalamnya.
- **Pada baris 41**, `android:text="Roll"` mengatur teks yang akan ditampilkan pada tombol tersebut.
- **Pada baris 42**, `android:backgroundTint="#6A1B9A"` mengatur warna latar belakang tombol menjadi ungu.
- **Pada baris 43**, `android:textColor="@android:color/white"` mengatur warna teks pada tombol menjadi putih.

MainActivity.kt / Compose :

1. Kelas MainActivity

- **Pada baris 26**, kelas `MainActivity` dideklarasikan sebagai turunan dari `ComponentActivity`, yang merupakan kelas dasar untuk activity yang menggunakan Jetpack Compose.
- **Pada baris 28**, di dalam fungsi `onCreate`, dipanggil `setContent`. Ini adalah titik masuk utama untuk Jetpack Compose. Semua kode di dalam blok `setContent` berfungsi untuk mendefinisikan dan membangun antarmuka pengguna (UI).
- **Pada baris 30**, `RollDaduTheme` digunakan sebagai pembungkus UI. Ini adalah sebuah `Composable` yang menerapkan tema (seperti warna, bentuk, dan gaya teks) yang konsisten ke seluruh komponen di dalamnya.
- **Pada baris 31-34**, `Surface` digunakan sebagai container. Atribut modifier diatur untuk membuat `Surface` mengisi seluruh layar (`fillMaxSize`) dan `color` diatur menjadi hitam sebagai warna latar belakang.
- **Pada baris 35**, `DiceRollerApp()` dipanggil. Ini adalah fungsi `Composable` utama yang berisi semua logika dan komponen UI untuk aplikasi lempar dadu.

2. Fungsi Composable DiceRollerApp

Fungsi ini adalah inti dari aplikasi yang menampilkan semua elemen visual dan menangani logika.

- **Pada baris 41**, anotasi `@OptIn(ExperimentalMaterial3Api::class)` digunakan karena beberapa komponen Material 3 (seperti Scaffold dan TopAppBar) masih dianggap eksperimental dan memerlukan persetujuan eksplisit untuk digunakan.
- **Pada baris 42**, anotasi `@Composable` menandakan bahwa `DiceRollerApp` adalah sebuah fungsi Composable, yaitu blok bangunan dasar UI di Compose yang dapat memancarkan (emit) UI.
- **Pada baris 44**, `val snackbarHostState = remember { SnackbarHostState() }` digunakan untuk membuat dan mengingat (*remember*) state dari `SnackbarHost`, yang berfungsi untuk mengontrol kapan Snackbar (notifikasi di bagian bawah layar) harus muncul.
- **Pada baris 45**, `val scope = rememberCoroutineScope()` digunakan untuk mendapatkan sebuah *coroutine scope* yang terikat pada siklus hidup Composable ini. Ini akan digunakan untuk menjalankan operasi asinkron seperti menampilkan Snackbar.
- **Pada baris 47 & 48**, dideklarasikan dua variabel state, `dice1` dan `dice2`. `mutableStateOf(0)` membuat sebuah state yang bisa diobservasi. `remember` memastikan state ini tidak direset setiap kali UI digambar ulang (*recomposition*). Setiap kali nilai `dice1` atau `dice2` berubah, semua Composable yang menggunakan state ini akan otomatis diperbarui.
- **Pada baris 50-54**, dideklarasikan variabel message menggunakan `when` untuk menentukan teks berdasarkan nilai `dice1` dan `dice2`. *(Catatan: Variabel message ini sebenarnya tidak digunakan di dalam UI, karena logika serupa dideklarasikan ulang di dalam tombol 'Roll').*
- **Pada baris 56**, Scaffold digunakan. Ini adalah Composable yang menyediakan struktur layout Material Design standar, dengan slot untuk TopAppBar, SnackbarHost, dan konten utama.
- **Pada baris 59**, `snackbarHost` diatur untuk menampilkan Snackbar yang dikontrol oleh `snackbarHostState`.
- **Pada baris 62-72**, `topBar` diatur untuk menampilkan TopAppBar (bilah judul di bagian atas). Di dalamnya, judul, warna teks, dan warna latar belakang TopAppBar didefinisikan.
- **Pada baris 73**, blok padding dari Scaffold dimulai. padding ini berisi nilai padding yang harus diterapkan pada konten utama agar tidak tertutup oleh TopAppBar.
- **Pada baris 74**, Column digunakan untuk menyusun komponen di dalamnya secara vertikal. modifier digunakan untuk mengatur agar Column mengisi seluruh layar, menerapkan padding dari Scaffold, dan memposisikan semua kontennya di tengah.
- **Pada baris 82**, Row digunakan untuk menyusun gambar dadu secara horizontal. `Arrangement.SpaceEvenly` mendistribusikan ruang kosong secara merata di antara dan di sekitar gambar dadu.
- **Pada baris 86 & 87**, `DiceImage(dice1)` dan `DiceImage(dice2)` dipanggil. Ini adalah pemanggilan fungsi Composable `DiceImage` yang dapat digunakan kembali, dengan meneruskan nilai state `dice1` dan `dice2` sebagai parameter.
- **Pada baris 90**, `Spacer` digunakan untuk memberikan ruang kosong vertikal setinggi 24.dp antara barisan dadu dan tombol.
- **Pada baris 92**, `Button Composable` didefinisikan.

- **Pada baris 93**, di dalam lambda onClick, semua logika yang akan dijalankan saat tombol ditekan didefinisikan.
- **Pada baris 94 & 95**, dua angka acak baru dihasilkan menggunakan Random.nextInt(1, 7), yang akan menghasilkan nilai dari 1 hingga 6.
- **Pada baris 97 & 98**, nilai state dice1 dan dice2 diperbarui dengan angka acak yang baru. Pembaruan ini secara otomatis akan memicu *recomposition* pada Image dan UI lain yang bergantung pada state ini.
- **Pada baris 100-104**, logika pesan untuk Snackbar didefinisikan.
- **Pada baris 106**, scope.launch digunakan untuk memulai sebuah coroutine.
- **Pada baris 107 & 108**, snackbarHostState.showSnackbar(resultMessage) dipanggil untuk menampilkan Snackbar dengan pesan yang sesuai. Pemanggilan dismiss() sebelumnya memastikan Snackbar yang lama hilang sebelum yang baru muncul.
- **Pada baris 111-113**, colors pada Button diatur untuk mengubah warna latar belakang tombol.

3. Fungsi Composable DiceImage

Fungsi ini adalah komponen UI kecil yang dapat digunakan kembali, bertanggung jawab hanya untuk menampilkan satu gambar dadu.

- **Pada baris 119**, fungsi Composable DiceImage dideklarasikan, yang menerima satu parameter diceValue berupa Int.
- **Pada baris 120**, val imageRes dideklarasikan. Nilainya ditentukan oleh ekspresi when yang memetakan nilai diceValue (1-6) ke ID drawable yang sesuai. Jika nilainya di luar rentang (seperti pada kondisi awal yaitu 0), gambar dice_0 akan digunakan sebagai default.
- **Pada baris 129**, Image Composable digunakan untuk menampilkan gambar.
- **Pada baris 130**, painterResource(id = imageRes) digunakan untuk memuat gambar dari drawable.
- **Pada baris 131**, contentDescription diatur untuk tujuan aksesibilitas.
- **Pada baris 132**, modifier digunakan untuk mengatur ukuran Image menjadi 180dp x 180dp.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

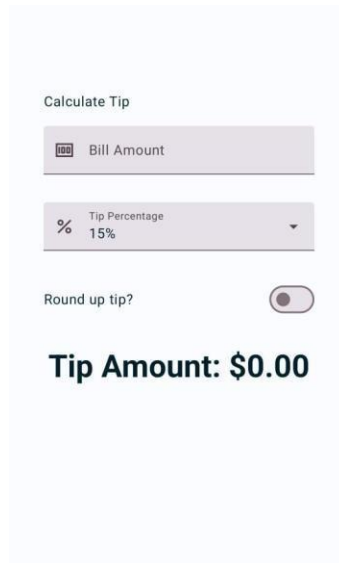
<https://github.com/Adiiaus/Praktikum-PemrogramanMobile.git>

MODUL 2 : Android Layout With Compose

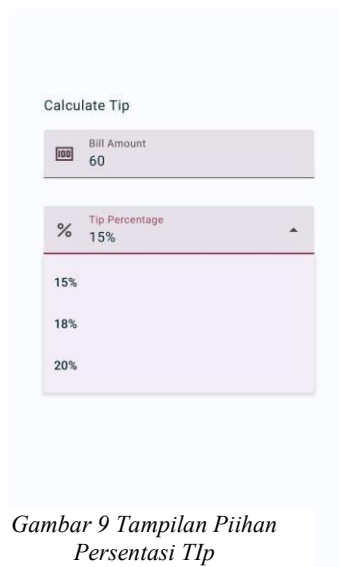
SOAL 1

Soal Praktikum:

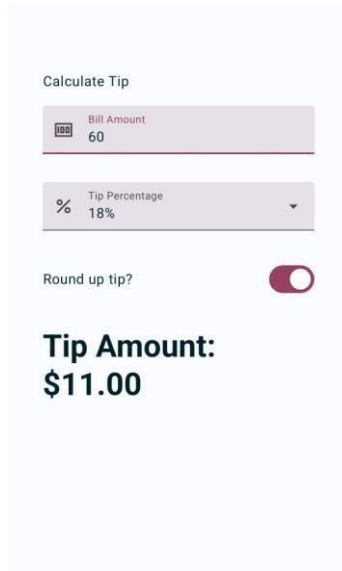
1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:
 - a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
 - b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
 - c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
 - d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 8 Tampilan Awal Aplikasi



Gambar 9 Tampilan Pilihan Persentase Tip



Gambar 10 Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt / Compose

```

1 package com.example.kalkulator
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.text.KeyboardOptions
8 import androidx.compose.material.icons.Icons
9 import androidx.compose.material.icons.filled.Percent
10 import androidx.compose.material.icons.filled.Receipt
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.*
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.text.font.FontWeight
16 import androidx.compose.ui.text.input.KeyboardType
17 import androidx.compose.ui.tooling.preview.Preview
18 import androidx.compose.ui.unit.dp
19 import androidx.compose.ui.unit.sp
20 import com.example.kalkulator.ui.theme.KalkulatorTheme
21 import java.text.NumberFormat
22
23 class MainActivity : ComponentActivity() {
24     override fun onCreate(savedInstanceState: Bundle?) {
25         super.onCreate(savedInstanceState)
26         setContent {
27             KalkulatorTheme {
28                 Surface(
29                     modifier = Modifier.fillMaxSize(),
30                     color =
31 MaterialTheme.colorScheme.background
32 )

```

```

33         TipCalculatorScreen()
34     }
35 }
36 }
37 }
38 }
39
40 @OptIn(ExperimentalMaterial3Api::class)
41 @Composable
42 fun TipCalculatorScreen() {
43     var billAmountInput by remember { mutableStateOf("") }
44     var roundUp by remember { mutableStateOf(false) }
45
46     val tipOptions = listOf("15%", "18%", "20%")
47     var expanded by remember { mutableStateOf(false) }
48     var selectedTipOption by remember {
49 mutableStateOf(tipOptions[0])
50
51     val amount = billAmountInput.toDoubleOrNull() ?: 0.0
52     val tipPercent =
53 selectedTipOption.removeSuffix("%").toDoubleOrNull() ?: 0.0
54     val tip = calculateTip(amount, tipPercent, roundUp)
55
56     Column(
57         modifier = Modifier
58             .padding(32.dp)
59             .fillMaxWidth(),
60         horizontalAlignment = Alignment.CenterHorizontally,
61         verticalArrangement = Arrangement.spacedBy(16.dp)
62     ) {
63         Text(
64             text = "Calculate Tip",
65             fontSize = 24.sp,
66             modifier = Modifier.align(Alignment.Start)
67         )
68
69         EditNumberField(
70             label = "Bill Amount",
71             value = billAmountInput,
72             onValueChange = { billAmountInput = it },
73             leadingIcon = { Icon(Icons.Filled.Receipt,
74 contentDescription = "Bill Amount") }
75         )
76
77         ExposedDropDownMenuBox(
78             expanded = expanded,
79             onExpandedChange = { expanded = !expanded },
80         ) {
81             OutlinedTextField(
82                 value = selectedTipOption,
83                 onValueChange = {},
84                 readOnly = true,
85                 label = { Text("Tip Percentage") },
86                 // MODIFIKASI: Tambahkan leadingIcon di sini
87                 leadingIcon = { Icon(Icons.Filled.Percent,
88 contentDescription = "Tip Percentage") },

```

```

89         trailingIcon = {
90
91 ExposedDropDownMenuDefaults.TrailingIcon(expanded = expanded)
92     },
93     modifier = Modifier
94         .menuAnchor()
95         .fillMaxWidth()
96 )
97 ExposedDropDownMenu(
98     expanded = expanded,
99     onDismissRequest = { expanded = false }
100 ) {
101     tipOptions.forEach { selectionOption ->
102         DropdownMenuItem(
103             text = { Text(selectionOption) },
104             onClick = {
105                 selectedTipOption =
106 selectionOption
107                 expanded = false
108             }
109         )
110     }
111 }
112 }
113 }
114
115 RoundTheTipRow(
116     roundUp = roundUp,
117     onRoundUpChanged = { roundUp = it }
118 )
119
120 Spacer(modifier = Modifier.height(24.dp))
121
122 Text(
123     text = "Tip Amount: $tip",
124     fontSize = 28.sp,
125     fontWeight = FontWeight.Bold
126 )
127 }
128 }
129 }
130
131 @Composable
132 fun EditNumberField(
133     label: String,
134     value: String,
135     onValueChange: (String) -> Unit,
136     leadingIcon: @Composable () -> Unit
137 ) {
138     OutlinedTextField(
139         value = value,
140         onValueChange = onValueChange,
141         label = { Text(label) },
142         leadingIcon = leadingIcon,
143         modifier = Modifier.fillMaxWidth(),
144         singleLine = true,
145         keyboardOptions = KeyboardOptions(keyboardType =
146 KeyboardType.Number)

```

147	<pre>) } @Composable fun RoundTheTipRow(roundUp: Boolean, onRoundUpChanged: (Boolean) -> Unit, modifier: Modifier = Modifier) { Row(modifier = modifier.fillMaxWidth().size(48.dp), verticalAlignment = Alignment.CenterVertically) { Text(text = "Round up tip?") Spacer(modifier = Modifier.weight(1f)) Switch(checked = roundUp, onCheckedChange = onRoundUpChanged) } } private fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String { var tip = tipPercent / 100 * amount if (roundUp) { tip = kotlin.math.ceil(tip) } return NumberFormat.getCurrencyInstance().format(tip) } @Preview(showBackground = true) @Composable fun DefaultPreview() { KalkulatorTheme { TipCalculatorScreen() } } </pre>
-----	---

Table 4 Source Code Soal 1 - Compose

activity_main.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<LinearLayout		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:tools="http://schemas.android.com/tools"		
5	android:layout_width="match_parent"		
6	android:layout_height="match_parent"		
7	android:orientation="vertical"		
8	android:padding="32dp"		
9	tools:context=".MainActivity">		
10	<TextView		
11	android:layout_width="wrap_content"		
12	android:layout_height="wrap_content"		
13	android:text="Calculate		Tip"
14	android:textSize="24sp"		/>
15	<com.google.android.material.textfield.TextInputLayout		
16	android:id="@+id/bill_amount_layout"		

```

1      android:layout_width="match_parent"
4      android:layout_height="wrap_content"
1      android:layout_marginTop="16dp"
5      android:hint="Bill Amount">
1
6      <com.google.android.material.textfield.TextInputEditText
1          android:id="@+id/bill_amount_edit_text"
7          android:layout_width="match_parent"
1          android:layout_height="wrap_content"
8          android:inputType="numberDecimal"      />
1
9      </com.google.android.material.textfield.TextInputLayout>
2
0      <com.google.android.material.textfield.TextInputLayout
2          android:id="@+id/tip_percentage_layout"
1
2      style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox.
2      ExposedDropdownMenu"
2          android:layout_width="match_parent"
3          android:layout_height="wrap_content"
2          android:layout_marginTop="16dp"
4          android:hint="Tip Percentage">
2
5          <AutoCompleteTextView
2              android:id="@+id/tip_percentage_autocomplete"
6              android:layout_width="match_parent"
2              android:layout_height="wrap_content"
7              android:inputType="none"      />
2
8      </com.google.android.material.textfield.TextInputLayout>
2      <com.google.android.material.switchmaterial.SwitchMaterial
9          android:id="@+id/round_up_switch"
3          android:layout_width="match_parent"
0          android:layout_height="wrap_content"
3          android:layout_marginTop="16dp"
1          android:minHeight="48dp"
3          android:text="Round up tip?"      />
2
3      <TextView
3          android:id="@+id/tip_result_text_view"
3          android:layout_width="wrap_content"
4          android:layout_height="wrap_content"
3          android:layout_gravity="center_horizontal"
5          android:layout_marginTop="40dp"
3          android:text="Tip Amount: $0.00"
6          android:textSize="28sp"
3          android:textStyle="bold"      />
7
3      </LinearLayout>
8
3
9
4
0
4
1

```

4	
2	
4	
3	

Table 5 Source Code Soal 1 - XML

MainActivity.kt / XML

```

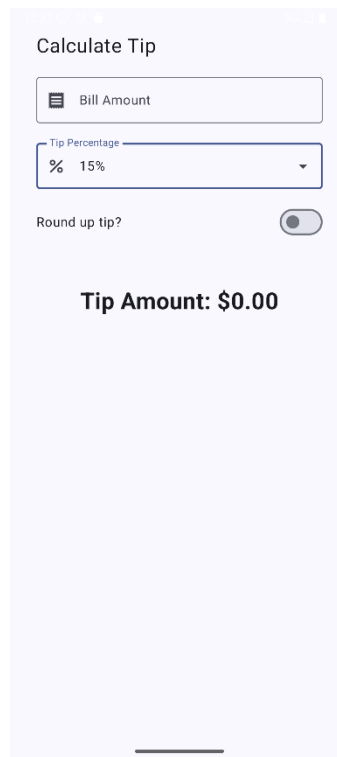
1 package com.example.tipxml
2
3 import androidx.appcompat.app.AppCompatActivity
4 import android.os.Bundle
5 import android.text.Editable
6 import android.text.TextWatcher
7 import android.widget.AdapterView
8 import com.example.tipxml.databinding.ActivityMainBinding
9 import java.text.NumberFormat
10
11 class MainActivity : AppCompatActivity() {
12
13     private lateinit var binding: ActivityMainBinding
14
15     override fun onCreate(savedInstanceState: Bundle?) {
16         super.onCreate(savedInstanceState)
17         binding = ActivityMainBinding.inflate(layoutInflater)
18         setContentView(binding.root)
19
20         val tipOptions =
21 resources.getStringArray(R.array.tip_percentage_options)
22         val adapter = ArrayAdapter(this,
23 android.R.layout.simple_spinner_dropdown_item, tipOptions)
24         binding.tipPercentageAutocomplete.setAdapter(adapter)
25
26 binding.tipPercentageAutocomplete.setText(tipOptions[0], false)
27
28         setupListeners()
29         calculateAndDisplayTip()
30     }
31
32     private fun setupListeners() {
33         binding.billAmountEditText.addTextChangedListener {
34 calculateAndDisplayTip()
35         binding.roundUpSwitch.setOnCheckedChangeListener { _, _
36 -> calculateAndDisplayTip()
37
38
39 binding.tipPercentageAutocomplete.setOnItemClickListener { _, _
40 -> calculateAndDisplayTip()
41
42     }
43 }
44
45     private fun calculateAndDisplayTip() {
46         val billAmountString =
47 binding.billAmountEditText.text.toString()
48         val tipPercentString =
49 binding.tipPercentageAutocomplete.text.toString()

```

	<pre> val amount = billAmountString.toDoubleOrNull() ?: 0.0 val tipPercent = tipPercentString.removeSuffix("%").toDoubleOrNull() ?: 0.0 val roundUp = binding.roundUpSwitch.isChecked var tip = tipPercent / 100 * amount if (roundUp) { tip = kotlin.math.ceil(tip) } val formattedTip = NumberFormat.getCurrencyInstance().format(tip) binding.tipResultTextView.text = "Tip Amount: \$formattedTip" } private fun android.widget.EditText.addTextChangedListener(onTextChanged: (String) -> Unit) { this.addTextChangedListener(object : TextWatcher { override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {} override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) { onTextChanged(s.toString()) } override fun afterTextChanged(s: Editable?) {} }) } } </pre>
--	---

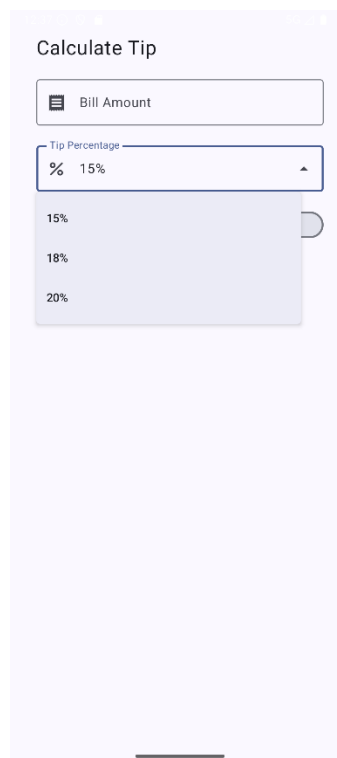
Table 6 Source Code Soal 1 - XML

B. Output Program



The screenshot shows a mobile application titled "Calculate Tip". It features a text input field labeled "Bill Amount" which is currently empty. Below it is a dropdown menu for "Tip Percentage" set to "15%". A toggle switch for "Round up tip?" is turned off. The result, "Tip Amount: \$0.00", is displayed in bold black text.

Gambar 11 Screenshot Hasil Jawaban Soal 1 - Compose



This screenshot shows the same "Calculate Tip" app, but with the "Tip Percentage" dropdown menu open. The menu lists three options: "15%", "18%", and "20%". The "Bill Amount" field remains empty, and the "Round up tip?" toggle is still off. The "Tip Amount: \$0.00" result is still visible at the bottom.

Gambar 12 Screenshot Hasil Jawaban Soal 1 - Compose

Calculate Tip

Bill Amount

2345

Tip Percentage

% 15%

Round up tip? ☐

Tip Amount: \$351.75

Gambar 13 Screenshot Hasil Jawaban Soal 1 - Compose

Calculate Tip

Bill Amount

2345

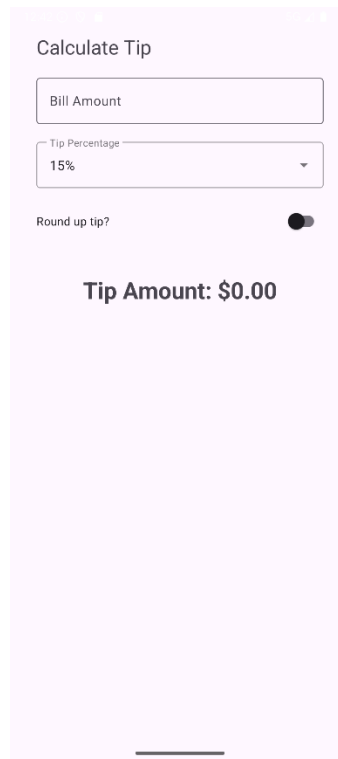
Tip Percentage

% 15%

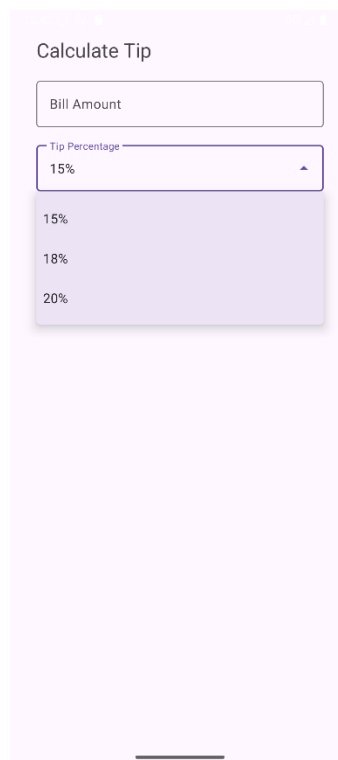
Round up tip? ☒

Tip Amount: \$352.00

Gambar 14 Screenshot Hasil Jawaban Soal 1 – Compose



Gambar 15 Screenshot Hasil Jawaban Soal 1 - XML



Gambar 16 Screenshot Hasil Jawaban Soal 1 - XML

Calculate Tip

Bill Amount
2345

Tip Percentage
15%

Round up tip? ☐

Tip Amount: \$351.75

Gambar 17 Screenshot Hasil Jawaban Soal 1 - XML

Calculate Tip

Bill Amount
2345

Tip Percentage
15%

Round up tip? ☒

Tip Amount: \$352.00

Gambar 18 Screenshot Hasil Jawaban Soal 1 - XML

C. Pembahasan

MainActivity.kt / Compose :

- **Pada baris 21**, kelas MainActivity dideklarasikan sebagai turunan dari ComponentActivity, yang merupakan kelas dasar untuk activity yang menggunakan Jetpack Compose.
- **Pada baris 23**, di dalam fungsi onCreate, dipanggil setContent yang menjadi titik masuk untuk membangun UI dengan Jetpack Compose.
- **Pada baris 25**, KalkulatorTheme membungkus UI untuk menerapkan tema yang konsisten (warna, font, dll.) pada semua komponen di dalamnya.
- **Pada baris 26-29**, Surface digunakan sebagai container utama yang mengisi seluruh layar (fillMaxSize) dan menggunakan warna latar belakang dari tema.
- **Pada baris 30**, TipCalculatorScreen() dipanggil. Ini adalah fungsi Composable utama yang membangun seluruh tampilan dan logika dari kalkulator tip.
- **Pada baris 38**, @OptIn(ExperimentalMaterial3Api::class) digunakan karena beberapa komponen seperti ExposedDropdownMenuBox memerlukan persetujuan eksplisit untuk digunakan.
- **Pada baris 41**, var billAmountInput by remember { mutableStateOf("") } mendeklarasikan sebuah *state* untuk menyimpan input jumlah tagihan dari pengguna dalam bentuk String.
- **Pada baris 42**, var roundUp by remember { mutableStateOf(false) } mendeklarasikan *state* boolean untuk melacak apakah opsi "bulatkan tip" sedang aktif atau tidak, yang dikontrol oleh komponen Switch.
- **Pada baris 44**, val tipOptions dibuat sebagai sebuah List yang berisi opsi-opsi persentase tip yang tersedia.
- **Pada baris 45 & 46**, var expanded dan var selectedTipOption dideklarasikan sebagai *state*. expanded mengontrol apakah menu dropdown ditampilkan atau tidak, sementara selectedTipOption menyimpan opsi tip yang sedang dipilih pengguna.
- **Pada baris 48-50**, dilakukan proses kalkulasi. Nilai billAmountInput (String) diubah menjadi Double secara aman menggunakan toDoubleOrNull(). Persentase tip juga di-parsing dari String menjadi Double. Kemudian, fungsi calculateTip dipanggil dengan parameter-parameter ini untuk mendapatkan hasil akhir.
- **Pada baris 52**, Column digunakan untuk menyusun semua elemen UI secara vertikal. Atribut modifier digunakan untuk memberi padding, mengisi lebar layar, dan Arrangement.spacedBy memberi jarak vertikal yang seragam antar elemen.
- **Pada baris 59**, Text ditampilkan sebagai judul layar.
- **Pada baris 65**, EditNumberField(...) dipanggil. Ini adalah pemanggilan fungsi Composable yang dapat digunakan kembali untuk menampilkan input field jumlah tagihan.
- **Pada baris 71**, ExposedDropdownMenuBox digunakan sebagai container untuk membuat field teks yang bisa diklik untuk menampilkan menu dropdown.
- **Pada baris 76**, OutlinedTextField ditampilkan di dalam ExposedDropdownMenuBox. Field ini bersifat readOnly dan menampilkan selectedTipOption. Field ini juga memiliki leadingIcon

(ikon persen) dan trailingIcon (ikon panah dropdown). Modifier .menuAnchor() penting untuk menandai TextField ini sebagai acuan posisi untuk ExposedDropdownMenu.

- **Pada baris 87**, ExposedDropdownMenu didefinisikan. Ini adalah menu yang akan muncul atau hilang berdasarkan *state expanded*.
- **Pada baris 91**, tipOptions.forEach digunakan untuk melakukan iterasi pada setiap opsi tip dan membuat sebuah DropdownMenuItem untuk masing-masing opsi. Saat sebuah item diklik, selectedTipOption diperbarui dan expanded di-set ke false untuk menutup menu.
- **Pada baris 102**, RoundTheTipRow(...) dipanggil. Ini adalah pemanggilan Composable lain yang dapat digunakan kembali untuk menampilkan baris "Round up tip?" beserta Switch-nya.
- **Pada baris 104**, Spacer digunakan untuk memberi ruang kosong vertikal.
- **Pada baris 107**, Text terakhir digunakan untuk menampilkan hasil kalkulasi tip (Tip Amount: \$tip) dengan teks tebal dan ukuran font yang lebih besar.
- **Pada baris 116**, fungsi Composable ini dideklarasikan dengan parameter untuk label, nilai saat ini (value), fungsi yang akan dipanggil saat nilai berubah (onValueChange), dan ikon awalan (leadingIcon).
- **Pada baris 122**, OutlinedTextField digunakan sebagai elemen input. keyboardOptions diatur ke KeyboardType.Number untuk memastikan keyboard numerik yang muncul saat field ini disentuh.
- **Pada baris 133**, fungsi ini dideklarasikan dengan parameter untuk status Switch (roundUp) dan fungsi yang akan dipanggil saat statusnya berubah.
- **Pada baris 134**, Row digunakan untuk menyusun elemen secara horizontal.
- **Pada baris 138**, Text menampilkan label "Round up tip?".
- **Pada baris 139**, Spacer dengan modifier = Modifier.weight(1f) digunakan. Ini adalah trik untuk membuat Spacer mengisi semua ruang kosong yang tersedia, sehingga mendorong Switch ke ujung kanan baris.
- **Pada baris 140**, Switch ditampilkan. Status checked-nya terikat pada *state roundUp*, dan onCheckedChange akan memperbarui *state* tersebut saat Switch digeser.
- **Pada baris 144**, fungsi ini dideklarasikan sebagai private karena hanya digunakan di dalam file ini.
- **Pada baris 145**, perhitungan tip dasar dilakukan ($\text{persentase} / 100 * \text{jumlah}$).
- **Pada baris 146 & 147**, jika roundUp bernilai true, maka nilai tip dibulatkan ke atas ke bilangan bulat terdekat menggunakan kotlin.math.ceil().
- **Pada baris 149**, NumberFormat.getCurrencyInstance().format(tip) digunakan untuk memformat hasil tip menjadi format mata uang lokal (misalnya, "Rp15.000" atau "\$15.00"), lalu mengembalikannya sebagai String.
- **Pada baris 152**, anotasi @Preview menandakan bahwa fungsi Composable ini akan dirender di dalam panel pratinjau Android Studio, memungkinkan developer melihat tampilan UI tanpa harus menjalankan aplikasi di perangkat atau emulator.

activity_main.xml / XML :

- **Pada baris 2**, LinearLayout digunakan sebagai layout utama yang menyusun semua elemen di dalamnya secara vertikal.
- **Pada baris 11**, TextView digunakan untuk menampilkan judul statis "Calculate Tip". **Pada baris 16**, com.google.android.material.textfield.TextInputLayout digunakan sebagai pembungkus untuk TextInputEditText. Komponen ini menyediakan fitur-fitur modern seperti label yang melayang (floating label) dan ruang untuk pesan error.
- **Pada baris 22**, com.google.android.material.textfield.TextInputEditText adalah field input teks sebenarnya tempat pengguna mengetikkan jumlah tagihan. `inputType="numberDecimal"` memastikan keyboard yang muncul adalah keyboard numerik.
- **Pada baris 28**, TextInputLayout kedua digunakan untuk menu dropdown. style diatur ke `...ExposedDropdownMenu` untuk memberikan tampilan dropdown modern dari Material Design.
- **Pada baris 35**, AutoCompleteTextView ditempatkan di dalam TextInputLayout dropdown. Komponen ini, jika digabungkan dengan style di atas, akan berfungsi sebagai field teks yang menampilkan menu dropdown saat diklik. `inputType="none"` mencegah keyboard muncul saat field ini disentuh.
- **Pada baris 43**, com.google.android.material.switchmaterial.SwitchMaterial adalah komponen Switch dari Material Design yang digunakan untuk opsi "Round up tip?".
- **Pada baris 50**, TextView terakhir digunakan untuk menampilkan hasil akhir dari perhitungan tip. ID `tip_result_text_view` digunakan oleh kode Kotlin untuk memperbarui teks di dalamnya. `textStyle="bold"` dan ukuran font yang besar digunakan untuk memberikan penekanan visual pada hasil.

MainActivity.kt / XML :

- **Pada baris 11**, kelas MainActivity dideklarasikan sebagai turunan dari AppCompatActivity, kelas dasar untuk Activity pada model View-XML.
- **Pada baris 13**, dideklarasikan variabel binding yang akan digunakan untuk mengakses semua komponen (View) dari file activity_main.xml secara aman (View Binding).
- **Pada baris 15**, di dalam fungsi onCreate yang pertama kali dijalankan, binding diinisialisasi dengan "mengubah" layout XML menjadi objek Kotlin.
- **Pada baris 17**, setContentView(binding.root) menetapkan tampilan dari activity ini menggunakan layout yang sudah di-binding.
- **Pada baris 19**, tipOptions diinisialisasi dengan mengambil array string dari file strings.xml. Array ini berisi opsi-opsi persentase tip.
- **Pada baris 20**, sebuah ArrayAdapter dibuat. Adapter ini berfungsi sebagai jembatan yang menghubungkan data (tipOptions) dengan komponen UI yang bisa menampilkannya (dalam kasus ini, AutoCompleteTextView).
- **Pada baris 21**, setAdapter(adapter) digunakan untuk memasang adapter tersebut ke tipPercentageAutocomplete, sehingga komponen tersebut tahu data apa yang harus ditampilkan sebagai opsi dropdown.

- **Pada baris 22**, `setText(tipOptions[0], false)` digunakan untuk mengatur nilai default yang tampil di field dropdown. Argumen `false` mencegah filter dropdown berjalan saat nilai diatur secara programatik.
- **Pada baris 24 & 25**, `setupListeners()` dan `calculateAndDisplayTip()` dipanggil untuk pertama kalinya untuk memasang semua "pendengar event" dan menampilkan nilai tip awal (yaitu \$0.00).
- **Pada baris 29**, sebuah listener ditambahkan ke `billAmountEditText` menggunakan fungsi ekstensi kustom. Setiap kali teks di dalam field ini berubah, fungsi `calculateAndDisplayTip()` akan otomatis dipanggil.
- **Pada baris 30**, `setOnCheckedChangeListener` dipasang pada `roundUpSwitch`. Setiap kali status switch berubah (dicentang atau tidak), `calculateAndDisplayTip()` akan dipanggil.
- **Pada baris 32**, `setOnItemClickListener` dipasang pada `tipPercentageAutocomplete`. Listener ini akan aktif saat pengguna memilih salah satu item dari menu dropdown, yang kemudian akan memanggil `calculateAndDisplayTip()`.
- **Pada baris 38 & 39**, nilai-nilai terbaru dari `billAmountEditText` dan `tipPercentageAutocomplete` dibaca dan diubah menjadi `String`.
- **Pada baris 41-43**, nilai-nilai `String` tersebut di-parsing menjadi tipe data yang sesuai (`Double` dan `Boolean`) secara aman.
- **Pada baris 45-48**, logika perhitungan tip dilakukan. Ini sama persis dengan versi `Compose`, termasuk opsi untuk membulatkan ke atas.
- **Pada baris 50**, hasil tip diformat menjadi format mata uang lokal (misal: "Rp15.000").
- **Pada baris 51**, properti `text` dari `tipResultTextView` diperbarui untuk menampilkan hasil tip yang sudah diformat. Inilah langkah "menulis" kembali ke UI.
- **Pada baris 54**, sebuah fungsi ekstensi dideklarasikan untuk kelas `android.widget.EditText`. Ini artinya, semua objek `EditText` bisa memanggil fungsi ini seolah-olah fungsi ini adalah bawaannya.
- **Pada baris 55-61**, implementasi `TextWatcher` yang biasanya panjang dan butuh tiga fungsi *override* (`before`, `on`, `after`) disederhanakan. Fungsi ini hanya peduli pada `onTextChanged` dan mengeksposnya sebagai sebuah lambda (`onTextChanged: (String) -> Unit`) yang lebih bersih dan mudah digunakan.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Adiiaus/Praktikum-PemrogramanMobile.git>

SOAL 2

2. Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Jawaban

Perbedaan, Kelebihan, dan Kekurangan: XML vs Jetpack Compose

Ini adalah bagian terpenting dari soal praktikummu. Mari kita bandingkan keduanya.

Paradigma Pemrograman

- **XML: Imperatif** (memerintah). Kamu mendefinisikan layout di XML, lalu di kode Kotlin, kamu *memerintah* setiap komponen untuk berubah. Contoh: `binding.tipResultTextView.text = "..."`. Kamu secara manual mencari komponen dan mengubah propertinya.
- **Jetpack Compose: Deklaratif**. Kamu *mendeklarasikan* atau mendeskripsikan bagaimana UI seharusnya terlihat untuk *state* (data) tertentu. Kamu tidak pernah mengubah UI secara langsung. Sebaliknya, kamu mengubah *state*-nya (`var amount by remember { ... }`), dan Compose akan secara otomatis memperbarui UI untukmu.

Kelebihan dan Kekurangan

Jetpack Compose

- **Kelebihan:**
 - **Kode Lebih Sedikit:** Secara signifikan mengurangi jumlah kode yang perlu ditulis.
 - **Intuitif:** Pendekatan deklaratif membuat alur data lebih mudah dipahami. "UI adalah cerminan dari data."
 - **Pengembangan Cepat:** Perubahan kecil pada UI tidak memerlukan build ulang yang kompleks. Fitur *preview* sangat cepat dan interaktif.
 - **Powerful:** Sangat mudah membuat animasi dan UI kustom yang kompleks.
- **Kekurangan:**
 - **Kurva Belajar:** Membutuhkan perubahan pola pikir bagi developer yang sudah sangat terbiasa dengan XML.
 - **Ekosistem Baru:** Meskipun berkembang pesat, beberapa library pihak ketiga mungkin belum sepenuhnya terintegrasi atau tersedia untuk Compose.

XML + Kotlin

- **Kelebihan:**

- **Mapan (Mature):** Telah ada selama bertahun-tahun, sangat stabil, dan didukung oleh banyak sekali library dan contoh di komunitas.
- **Pemisahan yang Jelas:** Beberapa developer menyukai pemisahan ketat antara file desain (XML) dan file logika (Kotlin).
- **Visual Editor:** Editor layout XML di Android Studio sangat powerfull dan mudah digunakan untuk mendesain UI secara *drag-and-drop*.

- **Kekurangan:**

- **Kode Boilerplate:** Membutuhkan banyak kode "perekat" seperti ViewBinding dan listener yang membuat kode lebih panjang.
- **Manajemen State Manual:** Mengelola state di UI yang kompleks bisa menjadi rumit dan rentan terhadap kesalahan (misalnya, lupa memperbarui salah satu TextView).
- **Verbosity:** XML bisa menjadi sangat panjang dan sulit dibaca untuk layout yang rumit.

Singkatnya, **Jetpack Compose adalah masa depan pengembangan UI Android** karena lebih modern, efisien, dan ringkas. Namun, **XML masih sangat relevan** dan akan tetap ada untuk waktu yang lama, terutama dalam proyek-proyek besar yang sudah ada.

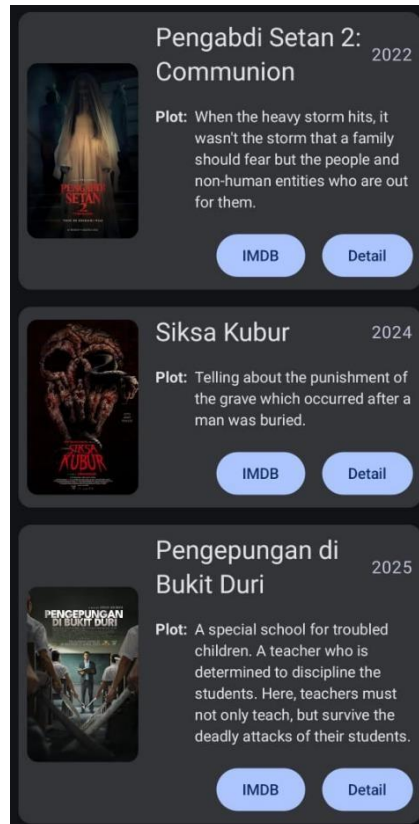
MODUL 3 : Build a Scrollable List

SOAL 1

Soal Praktikum:

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
 1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
 2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
 3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
 4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
 5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
 6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
 7. Aplikasi menggunakan arsitektur *single activity* (satu activity memiliki beberapa fragment)
 8. Aplikasi berbasis XML harus menggunakan ViewBinding
2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 19 Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 20 Contoh UI Detail

A. Source Code

Data/MovieRepository/ Compose

```
1 package com.example.scrollablelist_modul3.data
2
3
4 import com.example.scrollablelist_modul3.R
5 import com.example.scrollablelist_modul3.model.Movie
6
7 object MovieRepository {
8     fun getMovies(): List<Movie> {
9         return listOf(
10             Movie(
11                 title = "Uncanny Counter Season 1",
12                 year = "2020",
13                 plot = "Para Counter yang karyawan toko mi di siang
14 hari dan pemburu iblis di malam hari, memakai berbagai kemampuan
15 khusus untuk memburu roh-roh jahat yang mengincar manusia.",
16                 imdb = "https://www.imdb.com/title/tt13273826/",
17                 posterResId = R.drawable.dukun
18             ),
19             Movie(
20                 title = "Sweet Home Season 1",
21                 year = "2020",
22                 plot = "Saat banyak manusia berubah menjadi monster
23 ganas dan dunia terpuruk ke dalam teror, sekelompok penyintas berjuang
24 untuk hidup-dan mempertahankan kemanusiaan mereka..",
25                 imdb =
26 "https://www.imdb.com/title/tt11612120/?ref_=nv_sr_srsrg_0_tt_8_nm_0_
27 in_0_q_Sweet%2520Home",
28                 posterResId = R.drawable.rm
29             ),
30             Movie(
31                 title = "Money Heist Korea",
32                 year = "2022",
33                 plot = "Pencuri menguasai gedung percetakan uang
34 milik Korea bersatu dan menawan banyak sandera. Polisi harus
35 menghentikan aksi mereka, serta dalang yang bersembunyi di belakangnya.",
36                 imdb =
37 "https://www.imdb.com/title/tt13696452/?ref_=nv_sr_srsrg_0_tt_8_nm_0_
38 in_0_q_Money%2520Heist%2520Ko",
39                 posterResId = R.drawable.duitheist
40             ),
41             Movie(
42                 title = "My Name",
43                 year = "2021",
44                 plot = "Setelah ayahnya dibunuh, seorang wanita yang
45 ingin membalas dendam memutuskan untuk memercayai bos kriminal yang
46 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
47                 imdb =
48 "https://www.imdb.com/title/tt12940504/?ref_=nv_sr_srsrg_3_tt_8_nm_0_
49 in_0_q_My%2520Name",
50                 posterResId = R.drawable.ngaran
51             ),
52             Movie(
53                 title = "The 8 Show",
```

```

54         year = "2024",
55         plot = "Delapan orang yang terjebak di gedung delapan
56 lantai misterius mengikuti acara yang menarik tetapi berbahaya. Dengan
57 berjalannya waktu, mereka akan mendapat hadiah uang.",
58         imdb =
59 "https://www.imdb.com/title/tt30423279/?ref_=nv_sr_srsq_0_tt_8_nm_0_
60 in_0_q_The%25208%2520Show",
61         posterResId = R.drawable.delapan
62     ),
63     Movie(
64         title = "Kingdom Season 1",
65         year = "2020",
66         plot = "Saat rumor aneh tentang raja yang sakit
67 menguasai kerajaan, putra mahkota menjadi satu-satunya harapan mereka
68 untuk melawan wabah misterius yang menjangkiti negeri.",
69         imdb =
70 "https://www.imdb.com/title/tt6611916/?ref_=nv_sr_srsq_6_tt_8_nm_0_i
71 n_0_q_Kingdom",
72         posterResId = R.drawable.raja
73     ),
74     Movie(
75         title = "The Frog",
76         year = "2024",
77         plot = "Di musim panas nan damai, seorang wanita
78 misterius masuk ke sebuah penginapan—memicu beragam peristiwa yang
79 mengganggu kehidupan si pemilik dan orang-orang di sekitarnya.",
80         imdb = "https://www.imdb.com/title/tt26767508/",
81         posterResId = R.drawable.frog
82     ),
83     Movie(
84         title = "All of Us Are Dead",
85         year = "2022",
86         plot = "Sebuah SMA menjadi titik nol merebaknya wabah
87 virus zombi. Para murid yang terperangkap pun harus berjuang untuk
88 kabur jika tak mau terinfeksi dan berubah menjadi buas.",
89         imdb = "https://www.imdb.com/title/tt14169960/",
90         posterResId = R.drawable.allofus
91     ),
92     Movie(
93         title = "Taxi Driver",
94         year = "2021",
95         plot = "Seorang sopir taksi di Seoul mengantar seorang
96 wartawan Jerman untuk menyelidiki isu kerusuhan di Gwangju tanpa tahu
97 apa yang menanti mereka. Berdasarkan kisah nyata.",
98         imdb = "https://www.imdb.com/title/tt13759970/",
99         posterResId = R.drawable.taxi
100    ),
101    Movie(
102        title = "A Killer Paradox",
103        year = "2024",
104        plot = "Saat suatu pembunuhan tak disengaja
105 memunculkan pembunuhan serupa lainnya, pemuda biasa ini terjebak dalam
106 aksi kucing-kucingan tanpa henti dengan detektif yang lihai.",
107        imdb = "https://www.imdb.com/title/tt28642796/",
108        posterResId = R.drawable.paradox
109    )
110 )

```

10	)
5	}
10	}

Table 7 Source Code Soal 1 - Compose

Model / Movie / Compose

1	package	com.example.scrollablelist_modul3.model
2		
3	data	class Movie (
4	val	title: String,
5	val	year: String,
6	val	plot: String,
7	val	imdb: String,
8	val	posterResId: Int
9	)	

Table 8 Source Code Soal 1 - Compose

navigation / AppNavigation / Compose

1	package	com.example.scrollablelist_modul3.navigation
2		
3		
4	import	androidx.compose.runtime.Composable
5	import	androidx.navigation.NavType
6	import	androidx.navigation.navArgument
7	import	androidx.navigation.compose.NavHost
8	import	androidx.navigation.compose.composable
9	import	androidx.navigation.compose.rememberNavController
	import	com.example.scrollablelist_modul3.ui.DetailScreen
	import	com.example.scrollablelist_modul3.ui.MovieListScreen
	import	com.example.scrollablelist_modul3.R
	@Composable	
	fun	AppNavigation() {
	val	navController = rememberNavController()
	NavHost(navController = navController, startDestination =	
	Screen.MovieList.route)	{
	composable(Screen.MovieList.route)	{
	MovieListScreen(navController)	
	}	
	composable(	
	route = Screen.Detail.route,	
	arguments = listOf(	
	navArgument("title") { type = NavType.StringType	
	},	
	navArgument("year") { type = NavType.StringType	
	},	
	navArgument("plot") { type = NavType.StringType	
	},	
	navArgument("posterResId") { type =	
	NavType.IntType	}
	)	
	) {	backStackEntry ->

	<pre> val title = backStackEntry.arguments?.getString("title") ?: "" val year = backStackEntry.arguments?.getString("year") ?: "" val plot = backStackEntry.arguments?.getString("plot") ?: "" val posterResId = backStackEntry.arguments?.getInt("posterResId") ?: R.drawable.ngaran DetailScreen(title = title, year = year, plot = plot, posterResId = posterResId) } } } </pre>
--	---

Table 9 Source Code Soal 1 - Compose

navigation / Screen / Compose

1	package com.example.scrollablelist_modul3.navigation
2	import android.net.Uri
3	
4	sealed class Screen(val route: String) {
5	object MovieList : Screen("movie_list")
6	object Detail :
7	Screen("movie_detail/{title}/{year}/{plot}/{posterResId}") {
8	fun createRoute(title: String, year: String, plot: String,
9	posterResId: Int): String {
	return
	"movie_detail/\${Uri.encode(title)}/\${Uri.encode(year)}/\${Uri.encode(p
	lot)}/\${posterResId}"
	}
	}
	}

Table 10 Source Code Soal 1 - Compose

ui.theme / DetailScreen / Compose

1	package com.example.scrollablelist_modul3.ui
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Modifier
9	import androidx.compose.ui.draw.clip
	import androidx.compose.ui.res.painterResource
	import androidx.compose.ui.res.stringResource
	import androidx.compose.ui.unit.dp
	import com.example.scrollablelist_modul3.R
	@OptIn(ExperimentalMaterial3Api::class)
	@Composable

	<pre> fun DetailScreen(title: String, year: String, plot: String, posterResId: Scaffold(topBar TopAppBar(title Text(text "Movie Detail")))) Column(modifier .fillMaxSize() .padding(paddingValues) .padding(16.dp)) Image(painter = painterResource(id = posterResId), contentDescription = null, modifier = Modifier .fillMaxWidth() .height(550.dp) .clip(RoundedCornerShape(16.dp))) Spacer(modifier = Modifier.height(16.dp)) Text(text = String.format(stringResource(R.string.movie_detail_title), title, year), style = MaterialTheme.typography.headlineSmall) Spacer(modifier = Modifier.height(8.dp)) Text(text = "\${stringResource(R.string.plot_label)}:\n\$plot") } } </pre>
--	---

Table 11 Source Code Soal 1 - Compose

Ui.theme / MovieListScreen / Compose

1	package	com.example.scrollablelist_modul3.ui
2		
3	import	androidx.compose.foundation.layout.*
4	import	androidx.compose.foundation.lazy.LazyColumn
5	import	androidx.compose.foundation.lazy.items
6	import	androidx.compose.material3.*
7	import	androidx.compose.runtime.Composable
8	import	androidx.compose.ui.Modifier
9	import	androidx.compose.ui.unit.dp
	import	androidx.navigation.NavController
	import	com.example.scrollablelist_modul3.data.MovieRepository
	import	com.example.scrollablelist_modul3.navigation.Screen
	import	androidx.compose.foundation.Image
	import	androidx.compose.foundation.shape.RoundedCornerShape
	import	androidx.compose.ui.draw.clip

```

import androidx.compose.ui.res.painterResource
import android.content.Intent
import android.net.Uri
import androidx.compose.ui.platform.LocalContext

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MovieListScreen(navController: NavController) {
    val movieList = MovieRepository.getMovies()
    val context = LocalContext.current

    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text(text = "KDrama Cinema")
                }
            )
        }
    ) {
        LazyColumn(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
                .padding(8.dp)
        ) {
            items(movieList) { movie ->
                Card(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(vertical = 8.dp),
                    shape = RoundedCornerShape(16.dp),
                    elevation =
                        CardDefaults.cardElevation(4.dp)
                ) {
                    Row(modifier = Modifier.padding(16.dp)) {
                        // Poster Film
                        Image(
                            painter = painterResource(id =
                                movie.posterResId),
                            contentDescription = null,
                            modifier = Modifier
                                .size(width = 100.dp, height =
                                    150.dp)
                                .clip(RoundedCornerShape(8.dp))
                        )

                        Spacer(modifier =
                            Modifier.width(16.dp))

                        // Detail Film
                        Column(modifier = Modifier.weight(1f))
                    }

                    Row(
                        modifier =

```

Modifier.fillMaxWidth(),	horizontalArrangement	=
Arrangement.SpaceBetween	)	{
	Text(	
	text	= movie.title,
	style	=
MaterialTheme.typography.titleMedium	)	
	Text(	
	text	= movie.year,
	style	=
MaterialTheme.typography.bodySmall	)	
	}	
	Spacer(modifier	=
Modifier.height(8.dp))		
	Text(	
	text	= movie.plot,
	style	=
MaterialTheme.typography.bodySmall,	maxLines	= 3
	)	
	Spacer(modifier	=
Modifier.height(8.dp))		
	Row(	
	modifier	=
Modifier.fillMaxWidth(),	horizontalArrangement	=
Arrangement.End	)	{
	Button(	
	onClick	= {
	val intent	=
Intent(Intent.ACTION_VIEW,	Uri.parse(movie.imdb))	
context.startActivity(intent)		
	},	
	modifier	=
Modifier.padding(end	=	8.dp)
	)	{
	Text(text	= "IMDB")
	}	
	Button(	
	onClick	= {
	navController.navigate(	
Screen.Detail.createRoute(		
	movie.title,	
	movie.year,	
	movie.plot,	

	<pre>movie.posterResId)) }) Text(text = "Detail") } } } } } } }</pre>
--	--

Table 12 Source Code Soal 1 - Compose

MainActivity / Compose

1	package	com.example.scrollablelist_modul3
2		
3	import	android.os.Bundle
4	import	androidx.activity.ComponentActivity
5	import	androidx.activity.compose.setContent
6	import	com.example.scrollablelist_modul3.navigation.AppNavigation
7	import	
8		com.example.scrollablelist_modul3.ui.theme.ScrollableListModul3Theme
9		
	class	MainActivity : ComponentActivity() {
	override fun	onCreate(savedInstanceState: Bundle?) {
	super.onCreate(savedInstanceState)	
	setContent	{
	ScrollableListModul3Theme	{
	AppNavigation()	
	}	
	}	
	}	
	}	

Table 13 Source Code Soal 1 - Compose

```

1 package com.example.m3xml.data
2
3 import com.example.m3xml.R
4
5 object DataSource {
6     fun getFilms(): List<Film> {
7         return listOf(
8             Film(
9                 1,
10                "Unchanny Counter Season 1",
11                "2020",
12                "Para Counter yang karyawan toko mi di siang hari dan
13 pemburu iblis di malam hari, memakai berbagai kemampuan khusus untuk
14 memburu roh-roh jahat yang mengincar manusia.",
15                R.drawable.dukun,
16                "https://www.imdb.com/title/tt13273826/"
17            ),
18            Film(
19                2,
20                "Sweet Home",
21                "2020",
22                "Saat banyak manusia berubah menjadi monster ganas dan
23 dunia terpuruk ke dalam teror, sekelompok penyintas berjuang untuk
24 hidup-dan mempertahankan kemanusiaan mereka.",
25                R.drawable.rm,
26                "https://www.imdb.com/title/tt11612120/?ref=nv_sr_srsg_0_tt_8_nm_0_
27 in_0_q_Sweet%2520Home"
28            ),
29            Film(
30                3,
31                "Money Heist Korea",
32                "2022",
33                "Pencuri menguasai gedung percetakan uang milik Korea
34 bersatu dan menawan banyak sandera. Polisi harus menghentikan aksi
35 mereka, serta dalang yang bersembunyi di baliknya.",
36                R.drawable.duitheist, // ganti dengan nama file gambar
37                Anda
38            ),
39            Film(
40                4,
41                "My Name",
42                "2021",
43                "Setelah ayahnya dibunuh, seorang wanita yang ingin
44 membalas dendam memutuskan untuk memercayai bos kriminal yang
45 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
46                R.drawable.ngaran, // ganti dengan nama file gambar
47                Anda
48            ),
49            Film(
50                5,
51                "My Name",
52                "2021",
53                "Setelah ayahnya dibunuh, seorang wanita yang ingin
54 membalas dendam memutuskan untuk memercayai bos kriminal yang
55 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
56                R.drawable.ngaran, // ganti dengan nama file gambar
57                Anda
58            ),
59            Film(
60                6,
61                "My Name",
62                "2021",
63                "Setelah ayahnya dibunuh, seorang wanita yang ingin
64 membalas dendam memutuskan untuk memercayai bos kriminal yang
65 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
66                R.drawable.ngaran, // ganti dengan nama file gambar
67                Anda
68            ),
69            Film(
70                7,
71                "My Name",
72                "2021",
73                "Setelah ayahnya dibunuh, seorang wanita yang ingin
74 membalas dendam memutuskan untuk memercayai bos kriminal yang
75 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
76                R.drawable.ngaran, // ganti dengan nama file gambar
77                Anda
78            ),
79            Film(
80                8,
81                "My Name",
82                "2021",
83                "Setelah ayahnya dibunuh, seorang wanita yang ingin
84 membalas dendam memutuskan untuk memercayai bos kriminal yang
85 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
86                R.drawable.ngaran, // ganti dengan nama file gambar
87                Anda
88            ),
89            Film(
90                9,
91                "My Name",
92                "2021",
93                "Setelah ayahnya dibunuh, seorang wanita yang ingin
94 membalas dendam memutuskan untuk memercayai bos kriminal yang
95 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
96                R.drawable.ngaran, // ganti dengan nama file gambar
97                Anda
98            ),
99            Film(
100               10,
101               "My Name",
102               "2021",
103               "Setelah ayahnya dibunuh, seorang wanita yang ingin
104 membalas dendam memutuskan untuk memercayai bos kriminal yang
105 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
106               R.drawable.ngaran, // ganti dengan nama file gambar
107               Anda
108           ),
109           Film(
110               11,
111               "My Name",
112               "2021",
113               "Setelah ayahnya dibunuh, seorang wanita yang ingin
114 membalas dendam memutuskan untuk memercayai bos kriminal yang
115 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
116               R.drawable.ngaran, // ganti dengan nama file gambar
117               Anda
118           ),
119           Film(
120               12,
121               "My Name",
122               "2021",
123               "Setelah ayahnya dibunuh, seorang wanita yang ingin
124 membalas dendam memutuskan untuk memercayai bos kriminal yang
125 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
126               R.drawable.ngaran, // ganti dengan nama file gambar
127               Anda
128           ),
129           Film(
130               13,
131               "My Name",
132               "2021",
133               "Setelah ayahnya dibunuh, seorang wanita yang ingin
134 membalas dendam memutuskan untuk memercayai bos kriminal yang
135 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
136               R.drawable.ngaran, // ganti dengan nama file gambar
137               Anda
138           ),
139           Film(
140               14,
141               "My Name",
142               "2021",
143               "Setelah ayahnya dibunuh, seorang wanita yang ingin
144 membalas dendam memutuskan untuk memercayai bos kriminal yang
145 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
146               R.drawable.ngaran, // ganti dengan nama file gambar
147               Anda
148           ),
149           Film(
150               15,
151               "My Name",
152               "2021",
153               "Setelah ayahnya dibunuh, seorang wanita yang ingin
154 membalas dendam memutuskan untuk memercayai bos kriminal yang
155 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
156               R.drawable.ngaran, // ganti dengan nama file gambar
157               Anda
158           ),
159           Film(
160               16,
161               "My Name",
162               "2021",
163               "Setelah ayahnya dibunuh, seorang wanita yang ingin
164 membalas dendam memutuskan untuk memercayai bos kriminal yang
165 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
166               R.drawable.ngaran, // ganti dengan nama file gambar
167               Anda
168           ),
169           Film(
170               17,
171               "My Name",
172               "2021",
173               "Setelah ayahnya dibunuh, seorang wanita yang ingin
174 membalas dendam memutuskan untuk memercayai bos kriminal yang
175 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
176               R.drawable.ngaran, // ganti dengan nama file gambar
177               Anda
178           ),
179           Film(
180               18,
181               "My Name",
182               "2021",
183               "Setelah ayahnya dibunuh, seorang wanita yang ingin
184 membalas dendam memutuskan untuk memercayai bos kriminal yang
185 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
186               R.drawable.ngaran, // ganti dengan nama file gambar
187               Anda
188           ),
189           Film(
190               19,
191               "My Name",
192               "2021",
193               "Setelah ayahnya dibunuh, seorang wanita yang ingin
194 membalas dendam memutuskan untuk memercayai bos kriminal yang
195 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
196               R.drawable.ngaran, // ganti dengan nama file gambar
197               Anda
198           ),
199           Film(
200               20,
201               "My Name",
202               "2021",
203               "Setelah ayahnya dibunuh, seorang wanita yang ingin
204 membalas dendam memutuskan untuk memercayai bos kriminal yang
205 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
206               R.drawable.ngaran, // ganti dengan nama file gambar
207               Anda
208           ),
209           Film(
210               21,
211               "My Name",
212               "2021",
213               "Setelah ayahnya dibunuh, seorang wanita yang ingin
214 membalas dendam memutuskan untuk memercayai bos kriminal yang
215 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
216               R.drawable.ngaran, // ganti dengan nama file gambar
217               Anda
218           ),
219           Film(
220               22,
221               "My Name",
222               "2021",
223               "Setelah ayahnya dibunuh, seorang wanita yang ingin
224 membalas dendam memutuskan untuk memercayai bos kriminal yang
225 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
226               R.drawable.ngaran, // ganti dengan nama file gambar
227               Anda
228           ),
229           Film(
230               23,
231               "My Name",
232               "2021",
233               "Setelah ayahnya dibunuh, seorang wanita yang ingin
234 membalas dendam memutuskan untuk memercayai bos kriminal yang
235 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
236               R.drawable.ngaran, // ganti dengan nama file gambar
237               Anda
238           ),
239           Film(
240               24,
241               "My Name",
242               "2021",
243               "Setelah ayahnya dibunuh, seorang wanita yang ingin
244 membalas dendam memutuskan untuk memercayai bos kriminal yang
245 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
246               R.drawable.ngaran, // ganti dengan nama file gambar
247               Anda
248           ),
249           Film(
250               25,
251               "My Name",
252               "2021",
253               "Setelah ayahnya dibunuh, seorang wanita yang ingin
254 membalas dendam memutuskan untuk memercayai bos kriminal yang
255 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
256               R.drawable.ngaran, // ganti dengan nama file gambar
257               Anda
258           ),
259           Film(
260               26,
261               "My Name",
262               "2021",
263               "Setelah ayahnya dibunuh, seorang wanita yang ingin
264 membalas dendam memutuskan untuk memercayai bos kriminal yang
265 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
266               R.drawable.ngaran, // ganti dengan nama file gambar
267               Anda
268           ),
269           Film(
270               27,
271               "My Name",
272               "2021",
273               "Setelah ayahnya dibunuh, seorang wanita yang ingin
274 membalas dendam memutuskan untuk memercayai bos kriminal yang
275 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
276               R.drawable.ngaran, // ganti dengan nama file gambar
277               Anda
278           ),
279           Film(
280               28,
281               "My Name",
282               "2021",
283               "Setelah ayahnya dibunuh, seorang wanita yang ingin
284 membalas dendam memutuskan untuk memercayai bos kriminal yang
285 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
286               R.drawable.ngaran, // ganti dengan nama file gambar
287               Anda
288           ),
289           Film(
290               29,
291               "My Name",
292               "2021",
293               "Setelah ayahnya dibunuh, seorang wanita yang ingin
294 membalas dendam memutuskan untuk memercayai bos kriminal yang
295 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
296               R.drawable.ngaran, // ganti dengan nama file gambar
297               Anda
298           ),
299           Film(
300               30,
301               "My Name",
302               "2021",
303               "Setelah ayahnya dibunuh, seorang wanita yang ingin
304 membalas dendam memutuskan untuk memercayai bos kriminal yang
305 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
306               R.drawable.ngaran, // ganti dengan nama file gambar
307               Anda
308           ),
309           Film(
310               31,
311               "My Name",
312               "2021",
313               "Setelah ayahnya dibunuh, seorang wanita yang ingin
314 membalas dendam memutuskan untuk memercayai bos kriminal yang
315 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
316               R.drawable.ngaran, // ganti dengan nama file gambar
317               Anda
318           ),
319           Film(
320               32,
321               "My Name",
322               "2021",
323               "Setelah ayahnya dibunuh, seorang wanita yang ingin
324 membalas dendam memutuskan untuk memercayai bos kriminal yang
325 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
326               R.drawable.ngaran, // ganti dengan nama file gambar
327               Anda
328           ),
329           Film(
330               33,
331               "My Name",
332               "2021",
333               "Setelah ayahnya dibunuh, seorang wanita yang ingin
334 membalas dendam memutuskan untuk memercayai bos kriminal yang
335 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
336               R.drawable.ngaran, // ganti dengan nama file gambar
337               Anda
338           ),
339           Film(
340               34,
341               "My Name",
342               "2021",
343               "Setelah ayahnya dibunuh, seorang wanita yang ingin
344 membalas dendam memutuskan untuk memercayai bos kriminal yang
345 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
346               R.drawable.ngaran, // ganti dengan nama file gambar
347               Anda
348           ),
349           Film(
350               35,
351               "My Name",
352               "2021",
353               "Setelah ayahnya dibunuh, seorang wanita yang ingin
354 membalas dendam memutuskan untuk memercayai bos kriminal yang
355 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
356               R.drawable.ngaran, // ganti dengan nama file gambar
357               Anda
358           ),
359           Film(
360               36,
361               "My Name",
362               "2021",
363               "Setelah ayahnya dibunuh, seorang wanita yang ingin
364 membalas dendam memutuskan untuk memercayai bos kriminal yang
365 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
366               R.drawable.ngaran, // ganti dengan nama file gambar
367               Anda
368           ),
369           Film(
370               37,
37
```

3	),
2	Film(
3	5,
3	"The 8 Show",
3	"2024",
4	"Indonesia's preeminent superhero and his alter ego
3	Sancaka enter the cinematic universe to battle the wicked Pengkor and
5	his orphan army.",
3	R.drawable.delapan, // ganti dengan nama file gambar
6	Anda
3	"https://www.imdb.com/title/tt9201084/"
7	)
3	)
8	}
3	}

Table 14 Source Code Soal 1 - XML

Data / Film / XML

1	package	com.example.m3xml.data
2		
3	import	android.os.Parcelable
4	import	kotlinx.parcelize.Parcelize
5		
6	@Parcelize	
7	data	class Film(
8	val	id: Int,
9	val	title: String,
10	val	year: String,
11	val	plot: String,
1	val	poster: Int,
	val	imdbUrl: String
	) : Parcelable	

Table 15 Source Code Soal 1 - XML

Ui.theme / DetailFragment / XML

1	package	com.example.m3xml.ui.theme
2		
3	import	android.annotation.SuppressLint
4	import	android.os.Bundle
5	import	android.view.LayoutInflater
6	import	android.view.View
7	import	android.view.ViewGroup
8	import	androidx.fragment.app.Fragment
9	import	androidx.fragment.app.activityViewModels
10	import	androidx.navigation.fragment.navArgs
11	import	com.example.m3xml.databinding.FragmentDetailBinding
12	import	com.example.m3xml.viewmodel.FilmViewModel
13		
14	class	DetailFragment : Fragment() {
15		
16	private var	_binding: FragmentDetailBinding? = null
17	private val	binding get() = _binding!!
18		
19	private val	viewModel: FilmViewModel by activityViewModels()

```

20     private val args: DetailFragmentArgs by navArgs()
21
22     override fun onCreateView(
23         inflater: LayoutInflater, container: ViewGroup?,
24         savedInstanceState: Bundle?
25     ): View {
26         _binding = FragmentDetailBinding.inflate(inflater,
27 container,
28         return binding.root
29     }
30
31     @SuppressWarnings("SetTextI18n")
32     override fun onViewCreated(view: View, savedInstanceState:
33 Bundle?) {
34         super.onViewCreated(view, savedInstanceState)
35
36         val filmId = args.filmId
37         val film = viewModel.getFilmById(filmId)
38
39         film?.let {
40             binding.ivDetailPoster.setImageResource(it.poster)
41             binding.tvDetailTitle.text = "${it.title}
42 (${it.year})"
43             binding.tvDetailPlot.text = it.plot
44         }
45     }
46
47     override fun onDestroyView() {
48         super.onDestroyView()
49         _binding = null
50     }
51 }

```

Table 16 Source Code Soal 1 - XML

Ui.theme / FilmAdapter / XML

```

1 package com.example.m3xml.ui.theme
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.view.LayoutInflater
6 import android.view.ViewGroup
7 import androidx.recyclerview.widget.RecyclerView
8 import com.example.m3xml.databinding.ListItemBinding
9 import com.example.m3xml.data.Film
10
11 class FilmAdapter(private val filmList: List<Film>) :
12     RecyclerView.Adapter<FilmAdapter.FilmViewHolder>() {
13
14     var onDetailClick: ((Int) -> Unit)? = null
15
16     inner class FilmViewHolder(val binding: ListItemBinding) :
17         RecyclerView.ViewHolder(binding.root)
18
19     override fun onCreateViewHolder(parent: ViewGroup, viewType:
20 Int): FilmViewHolder {
21         val binding =

```

```

22 ListItemBinding.inflate(LayoutInflater.from(parent.context),
23 parent, false)
24     return FilmViewHolder(binding)
25 }
26
27 override fun onBindViewHolder(holder: FilmViewHolder,
28 position: Int) {
29     val film = filmList[position]
30     with(holder.binding) {
31         ivPoster.setImageResource(film.poster)
32         tvTitle.text = film.title
33         tvYear.text = film.year
34         tvPlot.text = film.plot
35
36         btnImdb.setOnClickListener {
37             val intent = Intent(Intent.ACTION_VIEW)
38             intent.data = Uri.parse(film.imdbUrl)
39             holder.itemView.context.startActivity(intent)
40         }
41
42         btnDetail.setOnClickListener {
43             onDetailClick?.invoke(film.id)
44         }
45     }
46 }
47
48 override fun getItemCount() = filmList.size
49 }

```

Table 17 Source Code Soal 1 - XML

Ui.theme / ListFragment / XML

```

1 package com.example.m3xml.ui.theme
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.fragment.app.activityViewModels
9 import androidx.navigation.fragment.findNavController
10 import androidx.recyclerview.widget.GridLayoutManager
11 import androidx.recyclerview.widget.LinearLayoutManager
12 import com.example.m3xml.databinding.FragmentListBinding
13 import com.example.m3xml.viewmodel.FilmViewModel
14
15 class ListFragment : Fragment() {
16
17     private var _binding: FragmentListBinding? = null
18     private val binding get() = _binding!!
19     private val filmViewModel: FilmViewModel by
20     activityViewModels()
21
22     override fun onCreateView(
23 inflater: LayoutInflater, container: ViewGroup?,

```


24	savedInstanceState:	Bundle?
25	):	View {
26	_binding	= FragmentListBinding.inflate(inflater,
27	container,	false)
28	return	binding.root
29	}	
30		
31	override fun onCreateView(view: View, savedInstanceState:	
32	Bundle?)	{
33	super.onCreateView(view,	savedInstanceState)
34		
35	val filmAdapter	= FilmAdapter(emptyList())
36		
37	val orientation	= resources.configuration.orientation
38	if (orientation ==	
39	android.content.res.Configuration.ORIENTATION_LANDSCAPE)	{
40	binding.recyclerView.layoutManager	=
41	GridLayoutManager(context,	2)
42	}	else {
43	binding.recyclerView.layoutManager	=
44	LinearLayoutManager(context)	
45	}	
46		
47	binding.recyclerView.adapter	= filmAdapter
48		
49	filmViewModel.films.observe(viewLifecycleOwner) { films -	
>		
	val adapter	= FilmAdapter(films)
	binding.recyclerView.adapter	= adapter
	adapter.onDetailClick	= { filmId ->
	val action	=
	ListFragmentDirections.actionListFragmentToDetailFragment(filmId)	
	findNavController().navigate(action)	
	}	
	}	
	}	
	override fun onDestroyView()	{
	super.onDestroyView()	
	_binding	= null
	}	
	}	

Table 18 Source Code Soal 1 - XML

Viewmodel / FilmViewModel / XML

1	package	com.example.m3xml.viewmodel
2		
3	import	androidx.lifecycle.LiveData
4	import	androidx.lifecycle.MutableLiveData
5	import	androidx.lifecycle.ViewModel
6	import	com.example.m3xml.data.DataSource
7	import	com.example.m3xml.data.Film
8		
9	class	FilmViewModel : ViewModel() {
10	private val	_films = MutableLiveData<List<Film>>()
11	val	films: LiveData<List<Film>> = _films
12		
13	init	{
14	_films.value	= DataSource.getFilms()
15	}	
16		
17	fun	getFilmById(id: Int): Film? {
18	return	_films.value?.find { it.id == id }
19	}	
20	}	

Table 19 Source Code Soal 1 - XML

MainActivity.kt / XML

1	package	com.example.m3xml
2		
3	import	android.os.Bundle
4	import	androidx.appcompat.app.AppCompatActivity
5	import	com.example.m3xml.databinding.ActivityMainBinding //
6	Ganti	dengan package Anda
7		
8	class	MainActivity : AppCompatActivity() {
9		
10	private lateinit	var binding: ActivityMainBinding
11		
12	override fun	onCreate(savedInstanceState: Bundle?) {
13	super.onCreate	(savedInstanceState)
14	binding =	ActivityMainBinding.inflate(layoutInflater)
15	setContentView	(binding.root)
16	}	
17	}	

Table 20 Source Code Soal 1 - XML

Layout / activity_main.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?
2	<androidx.constraintlayout.widget.ConstraintLayout		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:layout_width="match_parent"		
7	android:layout_height="match_parent"		
8	tools:context=".MainActivity">		
9			
10	<androidx.fragment.app.FragmentContainerView		

11	android:id="@+id/nav_host_fragment"
12	
13	android:name="androidx.navigation.fragment.NavHostFragment"
14	android:layout_width="0dp"
15	android:layout_height="0dp"
16	app:defaultNavHost="true"
17	app:layout_constraintBottom_toBottomOf="parent"
18	app:layout_constraintEnd_toEndOf="parent"
19	app:layout_constraintStart_toStartOf="parent"
20	app:layout_constraintTop_toTopOf="parent"
21	app:navGraph="@navigation/nav_graph" />
22	
23	</androidx.constraintlayout.widget.ConstraintLayout>

Table 21 Source Code Soal 1 - XML

Layout / fragment_detail.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<ScrollView		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:layout_width="match_parent"		
7	android:layout_height="match_parent"		
8	tools:context=".ui.theme.DetailFragment">		
9			
1	<androidx.constraintlayout.widget.ConstraintLayout		
0	android:layout_width="match_parent"		
1	android:layout_height="wrap_content"		
1	android:paddingBottom="16dp">		
1			
2	<TextView		
1	android:id="@+id/tv_header_detail"		
3	android:layout_width="0dp"		
1	android:layout_height="wrap_content"		
4	android:layout_marginStart="16dp"		
1	android:layout_marginTop="16dp"		
5	android:layout_marginEnd="16dp"		
1	android:text="Movie		Detail"
6			
1	android:textAppearance="@style/TextAppearance.Material3.HeadlineLarge"		
7			
1	android:textStyle="bold"		
8	app:layout_constraintEnd_toEndOf="parent"		
1	app:layout_constraintStart_toStartOf="parent"		
9	app:layout_constraintTop_toTopOf="parent" />		
2			
0	<com.google.android.material.imageview.ShapeableImageView		
2	android:id="@+id/iv_detail_poster"		
1	android:layout_width="0dp"		
2	android:layout_height="0dp"		
2	android:layout_marginStart="16dp"		
2	android:layout_marginTop="16dp"		
3	android:layout_marginEnd="16dp"		
2	android:scaleType="centerCrop"		
4	app:layout_constraintDimensionRatio="2:3"		
	app:layout_constraintEnd_toEndOf="parent"		

2	app:layout_constraintStart_toStartOf="parent"
5	
2	app:layout_constraintTop_toBottomOf="@id/tv_header_detail"
6	app:shapeAppearanceOverlay="@style/roundedImageView"
2	tools:srcCompat="@drawable/dukun" />
7	
2	<TextView
8	android:id="@+id/tv_detail_title"
2	android:layout_width="0dp"
9	android:layout_height="wrap_content"
3	android:layout_marginTop="16dp"
0	
3	android:textAppearance="?attr/textAppearanceHeadlineSmall"
1	android:textStyle="bold"
3	app:layout_constraintEnd_toEndOf="@id/iv_detail_poster"
2	
3	app:layout_constraintStart_toStartOf="@id/iv_detail_poster"
3	
3	app:layout_constraintTop_toBottomOf="@id/iv_detail_poster"
4	tools:text="Uncanny Counter Season 1 (2020)" />
3	
5	<TextView
3	android:id="@+id/tv_detail_plot_label"
6	android:layout_width="wrap_content"
3	android:layout_height="wrap_content"
7	android:layout_marginTop="16dp"
3	android:text="Plot:"
8	android:textAppearance="?attr/textAppearanceTitleMedium"
3	android:textStyle="bold"
9	
4	app:layout_constraintStart_toStartOf="@id/tv_detail_title"
0	
4	app:layout_constraintTop_toBottomOf="@id/tv_detail_title" />
1	
4	<TextView
2	android:id="@+id/tv_detail_plot"
4	android:layout_width="0dp"
3	android:layout_height="wrap_content"
4	android:layout_marginTop="4dp"
4	android:textAppearance="?attr/textAppearanceBodyMedium"
4	app:layout_constraintEnd_toEndOf="@id/tv_detail_title"
5	
4	app:layout_constraintStart_toStartOf="@id/tv_detail_plot_label"
6	
4	app:layout_constraintTop_toBottomOf="@id/tv_detail_plot_label"
7	tools:text="Para Counter yang karyawan toko mi di siang
4	hari dan pemburu iblis di malam hari, memakai berbagai kemampuan
8	khusus untuk memburu roh-roh jahat yang mengincar manusia." />
4	
	</androidx.constraintlayout.widget.ConstraintLayout>
	</ScrollView>

Table 22 Source Code Soal 1 - XML

Layout / fragment_list.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:layout_width="match_parent"		
7	android:layout_height="match_parent"		
8	tools:context=".ui.theme.ListFragment">		
9			
1	<TextView		
0	android:id="@+id/tv_header_title"		
1	android:layout_width="0dp"		
1	android:layout_height="wrap_content"		
1	android:layout_marginStart="16dp"		
2	android:layout_marginTop="16dp"		
1	android:layout_marginEnd="16dp"		
3	android:text="KDrama		Cinema"
1			
4	android:textAppearance="@style/TextAppearance.Material3.HeadlineLarge"		
1			
5	android:textStyle="bold"		
1	app:layout_constraintEnd_toEndOf="parent"		
6	app:layout_constraintStart_toStartOf="parent"		
1	app:layout_constraintTop_toTopOf="parent"		/>
7			
1	<androidx.recyclerview.widget.RecyclerView		
8	android:id="@+id/recycler_view"		
1	android:layout_width="0dp"		
9	android:layout_height="0dp"		
2	android:layout_marginTop="16dp"		
0	android:clipToPadding="false"		
2	android:paddingBottom="8dp"		
1			
2	app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"		
2	app:layout_constraintBottom_toBottomOf="parent"		
2	app:layout_constraintEnd_toEndOf="parent"		
3	app:layout_constraintStart_toStartOf="parent"		
2	app:layout_constraintTop_toBottomOf="@id/tv_header_title"		
4	tools:listitem="@layout/list_item"		/>
2			
5	</androidx.constraintlayout.widget.ConstraintLayout>		

Table 23 Source Code Soal 1 - XML

Layout / list_item.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:layout_width="match_parent"		
7	android:layout_height="wrap_content"		
8	android:layout_margin="8dp"		
9	app:cardCornerRadius="16dp"		
10	app:cardElevation="4dp">		

```

11
12     <androidx.constraintlayout.widget.ConstraintLayout
13         android:layout_width="match_parent"
14         android:layout_height="wrap_content"
15         android:padding="16dp">
16
17
18     <com.google.android.material.imageview.ShapeableImageView
19         android:id="@+id/iv_poster"
20         android:layout_width="100dp"
21         android:layout_height="150dp"
22         android:scaleType="centerCrop"
23         app:layout_constraintStart_toStartOf="parent"
24         app:layout_constraintTop_toTopOf="parent"
25
26     app:shapeAppearanceOverlay="@style/roundedImageView"
27         tools:src="@tools:sample/avatars"      />
28
29     <TextView
30         android:id="@+id/tv_title"
31         android:layout_width="0dp"
32         android:layout_height="wrap_content"
33         android:layout_marginStart="16dp"
34         android:layout_marginEnd="8dp"
35
36     android:textAppearance="?attr/textAppearanceHeadline6"
37         app:layout_constraintEnd_toStartOf="@+id/tv_year"
38         app:layout_constraintStart_toEndOf="@id/iv_poster"
39         app:layout_constraintTop_toTopOf="@id/iv_poster"
40         tools:text="Judul      Film      Panjang      Sekali"  />
41
42     <TextView
43         android:id="@+id/tv_year"
44         android:layout_width="wrap_content"
45         android:layout_height="wrap_content"
46         android:textAppearance="?attr/textAppearanceBody2"
47         app:layout_constraintEnd_toEndOf="parent"
48         app:layout_constraintTop_toTopOf="@id/tv_title"
49         tools:text="2025"      />
50
51     <TextView
52         android:id="@+id/tv_plot_label"
53         android:layout_width="wrap_content"
54         android:layout_height="wrap_content"
55         android:layout_marginTop="8dp"
56         android:text="Plot:"
57         android:textStyle="bold"
58         app:layout_constraintStart_toStartOf="@id/tv_title"
59         app:layout_constraintTop_toBottomOf="@id/tv_title"
60     />
61
62     <TextView
63         android:id="@+id/tv_plot"
64         android:layout_width="0dp"
65         android:layout_height="0dp"
66         android:layout_marginTop="4dp"

```

	<pre> android:ellipsize="end" android:maxLines="3" app:layout_constraintBottom_toTopOf="@id/btn_imdb" app:layout_constraintEnd_toEndOf="parent" app:layout_constraintStart_toStartOf="@id/tv_plot_label" app:layout_constraintTop_toBottomOf="@id/tv_plot_label" tools:text="Deskripsi plot film yang cukup panjang untuk mengisi beberapa baris dan menunjukkan fungsi elipsis." /> <Button android:id="@+id/btn_imdb" style="@style/Widget.MaterialComponents.Button.OutlinedButton" android:layout_width="0dp" android:layout_height="wrap_content" android:layout_marginTop="16dp" android:layout_marginEnd="8dp" android:text="IMDB" app:layout_constraintBottom_toBottomOf="parent" app:layout_constraintEnd_toStartOf="@id/btn_detail" app:layout_constraintStart_toStartOf="@id/tv_plot_label" /> <Button android:id="@+id/btn_detail" android:layout_width="0dp" android:layout_height="wrap_content" android:layout_marginStart="8dp" android:text="Detail" app:layout_constraintBottom_toBottomOf="parent" app:layout_constraintEnd_toEndOf="parent" app:layout_constraintStart_toEndOf="@id/btn_imdb" /> </androidx.constraintlayout.widget.ConstraintLayout> </com.google.android.material.card.MaterialCardView> </pre>
--	--

Table 24 Source Code Soal 1 - XML

Navigation / nav_graph.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<navigation		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:id="@+id/nav_graph"		
7	app:startDestination="@id/listFragment">		
8			
9	<fragment		

10	android:id="@+id/listFragment"	
11	android:name="com.example.m3xml.ui.theme.ListFragment"	
12	android:label="fragment_list"	
13	tools:layout="@layout/fragment_list"	>
14	<action	
15		
16	android:id="@+id/action_listFragment_to_detailFragment"	
17	app:destination="@id/detailFragment"	/>
18	</fragment>	
19	<fragment	
20	android:id="@+id/detailFragment"	
21		
22	android:name="com.example.m3xml.ui.theme.DetailFragment"	
23	android:label="fragment_detail"	
24	tools:layout="@layout/fragment_detail"	>
25	<argument	
26	android:name="filmId"	
27	app:argType="integer"	/>
28	</fragment>	
29	</navigation>	

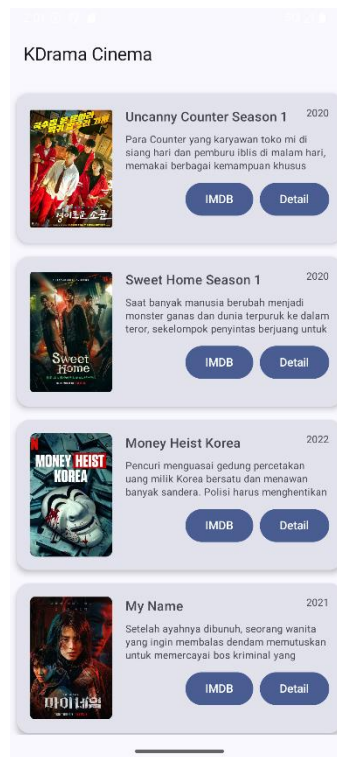
Table 25 Source Code Soal 1 - XML

Values/ themes.xml / XML

1	<resources xmlns:tools="http://schemas.android.com/tools">
2	<style name="Base.Theme.M3XML"
3	parent="Theme.Material3.DayNight.NoActionBar">
4	</style>
5	
6	<style name="Theme.M3XML" parent="Base.Theme.M3XML" />
7	
8	<style name="roundedImageView">
9	<item name="cornerFamily">rounded</item>
10	<item name="cornerSize">16dp</item>
11	</style>
12	
13	</resources>

Table 26 Source Code Soal 1 – XML

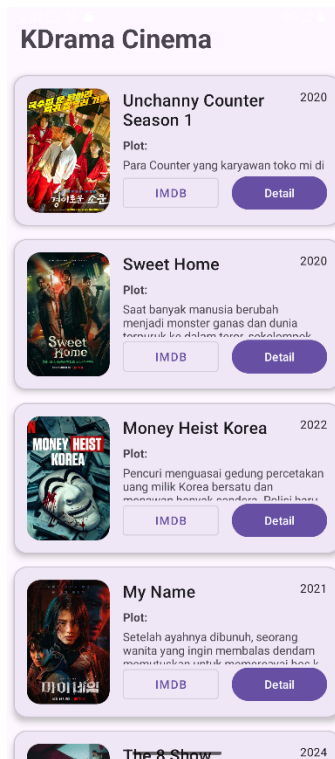
B. Output Program



Gambar 21 Screenshot Hasil Jawaban Soal 1 - Compose



Gambar 22 Screenshot Hasil Jawaban Soal 1 - Compose



Gambar 23 Screenshot Hasil Jawaban Soal 1 - XML



Gambar 24 Screenshot Hasil Jawaban Soal 1 – XML

C. Pembahasan

Jetpack Compose

1. model/Movie.kt (Struktur Data)

File ini adalah fondasi dari data yang akan ditampilkan.

- **Pada baris 3**, data class `Movie` dideklarasikan. data class adalah jenis kelas di Kotlin yang tujuan utamanya hanya untuk menyimpan data. Kelas ini mendefinisikan "cetakan" atau struktur untuk sebuah objek film, yang terdiri dari `id` (unik), `title` (judul), `description` (deskripsi), dan `imageRes` (ID gambar dari drawable).

2. data/MovieRepository.kt (Sumber Data)

File ini bertindak sebagai "database" tiruan atau sumber data untuk aplikasi.

- **Pada baris 5**, object `MovieRepository` dideklarasikan. Penggunaan object menjadikan `MovieRepository` sebuah *singleton*, artinya hanya ada satu instance dari repositori ini di seluruh aplikasi. Ini memastikan data yang diakses dari layar mana pun selalu konsisten.
- **Pada baris 6**, `private val movies` dideklarasikan. Ini adalah sebuah List privat yang berisi semua objek `Movie` yang akan ditampilkan. Data film di-hardcode langsung di sini.
- **Pada baris 41**, fungsi `getMovies()` didefinisikan untuk mengembalikan seluruh daftar film yang ada.
- **Pada baris 44**, fungsi `getMovieById(id: Int)` didefinisikan untuk mencari dan mengembalikan satu objek `Movie` berdasarkan `id` yang diberikan.

3. navigation/Screen.kt (Definisi Rute Navigasi)

File ini mendefinisikan semua kemungkinan tujuan (layar) navigasi dalam aplikasi secara terstruktur.

- **Pada baris 3**, sealed class `Screen` dideklarasikan. sealed class digunakan di sini untuk membuat grup rute yang terbatas dan aman-tipe (*type-safe*). Ini mencegah kesalahan pengetikan nama rute.
- **Pada baris 4**, object `MovieList` merepresentasikan rute untuk layar daftar film.
- **Pada baris 5**, object `Detail` merepresentasikan rute untuk layar detail film. Rute ini didefinisikan sebagai `"detail/{movieId}"`, di mana `{movieId}` adalah placeholder untuk argumen (ID film) yang akan dikirimkan.
- **Pada baris 7**, fungsi `createRoute(movieId: Int)` dibuat untuk memudahkan pembuatan rute lengkap ke layar detail dengan menyisipkan ID film yang sebenarnya ke dalam placeholder.

4. navigation/AppNavigation.kt (Pengatur Navigasi)

File ini adalah pusat kendali navigasi. Ia mengatur bagaimana semua layar terhubung.

- **Pada baris 11**, fungsi Composable `AppNavigation` dideklarasikan, yang menjadi inti dari sistem navigasi.
- **Pada baris 12**, `val navController = rememberNavController()` digunakan untuk membuat dan mengingat `NavController`, yaitu objek yang bertanggung jawab untuk mengelola navigasi (seperti `Maps()`, `popBackStack()`, dll).
- **Pada baris 13**, `NavHost` digunakan sebagai kontainer yang akan menampilkan tujuan (layar) Composable berdasarkan rute saat ini. `startDestination` diatur ke `Screen.MovieList.route`, yang berarti layar daftar film adalah layar pertama yang ditampilkan saat aplikasi dibuka.
- **Pada baris 17**, `composable(route = Screen.MovieList.route)` mendefinisikan apa yang harus ditampilkan untuk rute daftar film. Di sini, ia memanggil `MovieListScreen`, sambil meneruskan `navController` agar layar tersebut bisa melakukan navigasi.
- **Pada baris 21**, `composable(route = Screen.Detail.route, ...)` mendefinisikan layar detail.
- **Pada baris 22**, `arguments = listOf(navArgument("movieId") { type = NavType.IntType })` mendefinisikan bahwa rute ini menerima sebuah argumen bernama `"movieId"` yang bertipe `Int`.

- **Pada baris 25**, `val movieId = backStackEntry.arguments?.getInt("movieId") ?: 0` digunakan untuk mengekstrak nilai `movieId` yang dikirim dari layar sebelumnya.
- **Pada baris 26**, `DetailScreen` dipanggil dengan `movieId` yang telah diekstrak, memungkinkan layar detail untuk menampilkan data yang benar.

5. `ui/theme/MovieListScreen.kt` (Layar Daftar Film)

Ini adalah Composable untuk layar utama yang menampilkan daftar semua film.

- **Pada baris 18**, fungsi Composable `MovieListScreen` dideklarasikan, menerima `NavController` sebagai parameter untuk keperluan navigasi.
- **Pada baris 19**, `val movies = MovieRepository.getMovies()` digunakan untuk mengambil data seluruh film dari repositori.
- **Pada baris 21**, `LazyColumn` digunakan untuk menampilkan daftar yang bisa di-scroll. `LazyColumn` sangat efisien karena hanya merender item yang terlihat di layar.
- **Pada baris 24**, `items(movies)` melakukan iterasi pada setiap movie di dalam daftar `movies` dan membuat sebuah `MovieItem` untuk masing-masing film.
- **Pada baris 29**, di dalam `MovieItem`, `Card` digunakan untuk membungkus setiap item daftar dengan tampilan kartu yang memiliki bayangan dan sudut membulat.
- **Pada baris 32**, `modifier = Modifier.clickable { ... }` membuat seluruh kartu dapat diklik.
- **Pada baris 33**, di dalam blok `clickable`, `NavController.navigate(Screen.Detail.createRoute(movie.id))` dieksekusi. Ini adalah aksi navigasi yang membawa pengguna ke layar detail, sambil mengirimkan id dari film yang diklik.
- **Pada baris 37**, `Row` digunakan untuk menyusun gambar dan teks (judul, deskripsi) secara horizontal.
- **Pada baris 39**, `Image` digunakan untuk menampilkan poster film. `painterResource` memuat gambar dari `drawable`.
- **Pada baris 45**, `Column` digunakan untuk menyusun judul dan deskripsi film secara vertikal.

6. `ui/theme/DetailScreen.kt` (Layar Detail Film)

Ini adalah Composable untuk layar yang menampilkan detail dari satu film yang dipilih.

- **Pada baris 19**, fungsi Composable `DetailScreen` dideklarasikan, menerima `movieId` dan `NavController` sebagai parameter.
- **Pada baris 20**, `val movie = MovieRepository.getMovieById(movieId)` digunakan untuk mengambil data spesifik dari satu film berdasarkan `movieId` yang diterima dari navigasi.
- **Pada baris 22**, `BackHandler(onBack = { NavController.popBackStack() })` digunakan untuk menangani penekanan tombol kembali (baik fisik maupun gestur). `NavController.popBackStack()` akan mengembalikan pengguna ke layar sebelumnya (yaitu `MovieListScreen`).
- **Pada baris 25**, `Column` digunakan untuk menyusun semua elemen UI detail secara vertikal dan diposisikan di tengah.
- **Pada baris 32**, `Image` digunakan untuk menampilkan poster film yang dipilih dengan ukuran yang lebih besar.
- **Pada baris 38 & 39**, `Text` digunakan untuk menampilkan judul dan deskripsi film dengan gaya teks yang berbeda untuk memberikan penekanan.

7. `MainActivity.kt` (Aktivitas Utama)

File ini adalah titik masuk aplikasi, tempat semuanya dimulai.

- **Pada baris 16**, di dalam `onCreate`, `setContent` dipanggil untuk memulai UI Jetpack Compose.
- **Pada baris 18**, `Surface` digunakan sebagai container latar belakang.

- **Pada baris 22**, `AppNavigation()` dipanggil. Ini adalah satu-satunya panggilan penting di sini, yang secara efektif menyerahkan seluruh kontrol UI dan navigasi ke `Composable AppNavigation`

XML

1. Data / DataSource (DataSource.kt)

File ini berfungsi sebagai sumber data statis untuk aplikasi.

- **Pada baris 5, object DataSource** digunakan untuk mendeklarasikan kelas sebagai sebuah *singleton*. Pola desain ini memastikan bahwa hanya ada satu instance dari `DataSource` yang digunakan di seluruh siklus hidup aplikasi, sehingga data yang diakses bersifat konsisten dan terpusat.
- **Pada baris 6, fun getFilms(): List<Film>** adalah sebuah fungsi publik yang ketika dipanggil akan mengembalikan data berupa `List` (daftar) yang berisi objek-objek dari kelas `Film`.
- **Pada baris 7, return listOf(...)** mengembalikan sebuah `List` yang nilainya telah ditentukan secara langsung di dalam kode (statis).
- **Pada baris 8-14, Film(1, "Unchanny Counter Season 1", ...)** merupakan proses inisialisasi atau pembuatan instance dari objek `Film`. Setiap argumen yang dilewatkan (seperti `id`, judul, tahun, plot, ID resource drawable, dan URL) mengisi properti yang telah didefinisikan di dalam data class `Film`.

2. Data / Film (Film.kt)

File ini mendefinisikan model atau struktur data untuk entitas film.

- **Pada baris 7, data class Film(...)** adalah sebuah fitur Kotlin untuk membuat kelas yang tujuan utamanya adalah untuk menyimpan data. Kelas ini mendefinisikan properti yang dimiliki oleh sebuah entitas film, yaitu `id`, `title`, `year`, `plot`, `poster`, dan `imdbUrl`.
- **Pada baris 6, @Parcelize** merupakan sebuah anotasi dari library Kotlin Android Extensions yang secara otomatis meng-generate implementasi dari `Parcelable`.
- **Pada baris 14, : Parcelable** adalah implementasi dari interface `Parcelable` milik Android. Mekanisme ini memungkinkan objek dari kelas `Film` untuk diserialisasi sehingga dapat dikirim antar komponen Android, misalnya sebagai argumen dalam navigasi antar `Fragment`.

3. Viewmodel / FilmViewModel (FilmViewModel.kt)

Kelas ini menerapkan pola arsitektur `ViewModel` yang berfungsi untuk menyimpan dan mengelola data terkait UI. `ViewModel` dirancang untuk bertahan dari perubahan konfigurasi, seperti rotasi layar.

- **Pada baris 11, private val _films = MutableLiveData<List<Film>>()** membuat sebuah properti privat berupa `MutableLiveData`. Objek ini dapat menyimpan daftar film dan nilainya dapat diubah dari dalam `ViewModel`.
- **Pada baris 12, val films: LiveData<List<Film>> = _films** mengekspos data `_films` sebagai `LiveData` yang bersifat *read-only*. Hal ini merupakan praktik yang direkomendasikan untuk memastikan bahwa data hanya dapat dimodifikasi oleh `ViewModel`, sementara komponen UI (seperti `Fragment`) hanya dapat mengamatinya.
- **Pada baris 14, init { ... }** adalah blok inisialisasi yang dieksekusi secara otomatis ketika instance `ViewModel` pertama kali dibuat. Blok ini bertugas untuk memuat data film dari `DataSource` ke dalam `_films`.
- **Pada baris 18, fun getFilmById(id: Int): Film?** adalah sebuah fungsi publik yang memungkinkan komponen lain untuk mengambil data satu film secara spesifik dari daftar berdasarkan `id`-nya. Tipe kembalian `Film?` mengindikasikan bahwa fungsi ini dapat mengembalikan `null` jika film dengan `id` tersebut tidak ditemukan.

4. Ui.theme / FilmAdapter (FilmAdapter.kt)

Kelas Adapter ini berfungsi sebagai jembatan antara sumber data (`List<Film>`) dengan komponen `RecyclerView` yang menampilkannya.

- **Pada baris 12, class FilmAdapter(...)** merupakan deklarasi kelas adapter yang menerima `List<Film>` pada konstruktornya sebagai sumber data untuk ditampilkan.

- Pada baris 15, **var onDetailClick: ((Int) -> Unit)?** mendeklarasikan sebuah properti fungsional (lambda) yang dapat di-override dari luar. Properti ini berfungsi sebagai *callback* untuk menangani event klik pada tombol detail, dengan mengirimkan id film sebagai parameter.
- Pada baris 25, **override fun onBindViewHolder(...)** adalah fungsi yang dipanggil oleh RecyclerView untuk menampilkan data pada posisi tertentu. Di dalam fungsi ini, data dari objek film diikat ke setiap elemen View di dalam ViewHolder.
- Pada baris 32, **btnImdb.setOnClickListener** mendaftarkan sebuah *click listener* pada tombol IMDB. Ketika tombol diklik, sebuah Intent dengan aksi ACTION_VIEW dibuat untuk membuka imdbUrl yang tersimpan pada data film.
- Pada baris 38, **btnDetail.setOnClickListener** mendaftarkan *click listener* pada tombol Detail. Ketika diklik, ia akan mengeksekusi lambda onDetailClick dan meneruskan film.id dari item yang bersangkutan.

5. Ui.theme / ListFragment (ListFragment.kt)

Fragment ini merepresentasikan layar yang menampilkan daftar film kepada pengguna.

- Pada baris 20, **private val filmViewModel: FilmViewModel by activityViewModels()** adalah cara untuk mendapatkan instance FilmViewModel yang terikat pada siklus hidup Activity. Ini memungkinkan data di ViewModel dapat dibagikan antara beberapa Fragment (ListFragment dan DetailFragment).
- Pada baris 35, **if (orientation == android.content.res.Configuration.ORIENTATION_LANDSCAPE)** adalah implementasi logika untuk layout responsif. Kode ini memeriksa orientasi perangkat saat ini dan memilih LayoutManager yang sesuai untuk RecyclerView: GridLayoutManager untuk landscape dan LinearLayoutManager untuk portrait.
- Pada baris 43, **filmViewModel.films.observe(...)** mendaftarkan Observer ke LiveData films di dalam ViewModel. Blok kode di dalam observer ini akan dieksekusi secara otomatis setiap kali data films diperbarui.
- Pada baris 46, **adapter.onDetailClick = { filmId -> ... }** mengimplementasikan *callback* onDetailClick dari FilmAdapter. Blok kode ini mendefinisikan aksi yang akan dilakukan ketika tombol detail pada sebuah item diklik.
- Pada baris 47, **val action = ListFragmentDirections.actionListFragmentToDetailFragment(filmId)** menggunakan plugin *Navigation Safe Args* untuk membuat objek aksi navigasi yang *type-safe*. Aksi ini membawa filmId sebagai argumen untuk dikirim ke DetailFragment.
- Pada baris 48, **findNavController().navigate(action)** mengeksekusi perintah navigasi untuk berpindah dari ListFragment ke DetailFragment.

6. Ui.theme / DetailFragment (DetailFragment.kt)

Fragment ini merepresentasikan layar yang menampilkan informasi rinci dari film yang telah dipilih.

- Pada baris 20, **private val args: DetailFragmentArgs by navArgs()** menggunakan delegasi properti dari *Navigation Safe Args* untuk secara aman mengekstrak argumen yang dikirimkan melalui aksi navigasi.
- Pada baris 34, **val filmId = args.filmId** mengambil nilai argumen filmId dari objek args.
- Pada baris 35, **val film = viewModel.getFilmById(filmId)** memanggil fungsi pada FilmViewModel untuk mengambil objek Film secara keseluruhan berdasarkan filmId yang telah diterima.
- Pada baris 37, **film?.let { ... }** adalah sebuah *scope function* yang melakukan pengecekan *null*. Blok kode di dalamnya hanya akan dieksekusi jika objek film tidak null, mencegah NullPointerException.
- Pada baris 38-40, baris-baris kode ini mengisi komponen-komponen UI (seperti ImageView dan TextView) pada layout fragment_detail.xml dengan data yang sesuai dari properti objek film (poster, judul, tahun, dan plot).

7. MainActivity.kt

File ini adalah komponen Activity utama yang berfungsi sebagai entry point aplikasi.

- Pada baris 8, **class MainActivity : AppCompatActivity()** mendeklarasikan kelas aktivitas utama yang mewarisi fungsionalitas dari AppCompatActivity.
- Pada baris 10, **private lateinit var binding: ActivityMainBinding** mendeklarasikan variabel untuk menampung instance dari kelas binding yang dihasilkan secara otomatis untuk activity_main.xml, memungkinkan akses view yang aman.
- Pada baris 12, **override fun onCreate(...)** adalah fungsi *lifecycle callback* yang dipanggil saat Activity pertama kali dibuat.
- Pada baris 14, **binding = ActivityMainBinding.inflate(layoutInflater)** melakukan *inflate* terhadap layout XML dan menginisialisasi objek binding.
- Pada baris 15, **setContentView(binding.root)** menetapkan root view dari layout yang telah di-inflate sebagai konten utama dari Activity.

8. Layout / activity_main.xml

File layout ini berfungsi sebagai kerangka dasar antarmuka pengguna aplikasi.

- Pada baris 9, **<androidx.fragment.app.FragmentContainerView ...>** adalah sebuah View khusus yang dirancang untuk menampung Fragment. Komponen ini bertindak sebagai NavHost yang akan menampilkan tujuan navigasi.
- Pada baris 19, **app:navGraph="@navigation/nav_graph"** adalah atribut yang menghubungkan FragmentContainerView ini dengan grafik navigasi yang didefinisikan dalam nav_graph.xml, yang mengatur alur perpindahan antar Fragment.

9. Layout / fragment_list.xml

File layout ini mendefinisikan struktur visual untuk ListFragment.

- Pada baris 9, **<TextView ...>** berfungsi sebagai elemen teks statis untuk menampilkan judul pada bagian atas layar.
- Pada baris 21, **<androidx.recyclerview.widget.RecyclerView ...>** adalah komponen utama pada layout ini. RecyclerView digunakan untuk menampilkan daftar data yang besar secara efisien dengan mendaur ulang View untuk setiap item.

10. Layout / fragment_detail.xml

File layout ini mendefinisikan struktur visual untuk DetailFragment.

- Pada baris 2, **<ScrollView ...>** digunakan sebagai View induk untuk memastikan bahwa konten di dalamnya dapat di-scroll jika tingginya melebihi ukuran layar.
- Pada baris 29, **<com.google.android.material.imageview.ShapeableImageView ...>** adalah komponen ImageView dari library Material Design yang mendukung pembentukan sudut, digunakan di sini untuk menampilkan poster film dengan sudut membulat.
- Pada baris 48, **<TextView android:id="@+id/tv_detail_title" ...>** dan baris 85, **<TextView android:id="@+id/tv_detail_plot" ...>** adalah komponen TextView yang masing-masing berfungsi sebagai penampung untuk menampilkan judul dan plot film.

11. Layout / list_item.xml

File layout ini berfungsi sebagai templat untuk setiap item individual di dalam RecyclerView.

- Pada baris 2, **<com.google.android.material.card.MaterialCardView ...>** membungkus setiap item dalam sebuah View berbentuk kartu, yang memberikan elevasi (efek bayangan) dan batas sudut yang jelas, sesuai dengan pedoman Material Design.
- Pada baris 79, **<Button android:id="@+id/btn_imdb" ...>** dan baris 90, **<Button android:id="@+id/btn_detail" ...>** adalah komponen Button yang menyediakan interaksi bagi pengguna pada setiap item, yaitu untuk melihat link IMDB dan untuk menavigasi ke halaman detail.

12. Navigation / nav_graph.xml

File ini mendefinisikan semua alur navigasi antar Fragment di dalam aplikasi menggunakan Navigation Component.

- **Pada baris 6, `app:startDestination="@id/listFragment"`** menetapkan listFragment sebagai Fragment awal yang akan ditampilkan ketika grafik navigasi ini dimuat.
- **Pada baris 14, `<action ...>`** mendefinisikan sebuah jalur navigasi yang valid dari listFragment (asal) ke detailFragment (tujuan).
- **Pada baris 24, `<argument android:name="filmId" app:argType="integer" />`** mendeklarasikan bahwa detailFragment menerima sebuah argumen bernama filmId dengan tipe data integer. Deklarasi ini memungkinkan pengiriman data antar Fragment dengan cara yang aman (type-safe).

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/AdiYaus/Praktikum-PemrogramanMobile.git>

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Jawaban

Alasan utama mengapa RecyclerView masih relevan dan terus digunakan secara luas meskipun LazyColumn menawarkan sintaksis yang lebih ringkas adalah karena keduanya berasal dari paradigma dan ekosistem UI yang berbeda secara fundamental.

- **RecyclerView** adalah komponen inti dari **Android View System**, yang merupakan pendekatan tradisional berbasis XML untuk membangun UI.
- **LazyColumn** adalah komponen dari **Jetpack Compose**, yaitu *toolkit* UI deklaratif modern dari Google.

Meskipun LazyColumn lebih modern, RecyclerView tetap menjadi komponen yang vital karena beberapa faktor berikut:

1. **Basis Kode yang Sudah Ada (*Legacy Codebase*)** Alasan paling signifikan adalah keberadaan jutaan aplikasi di Google Play Store yang dibangun menggunakan Android View System. Proses rekayasa ulang (*refactoring*) atau penulisan ulang seluruh aplikasi hanya untuk mengganti RecyclerView dengan LazyColumn sering kali tidak efisien dari segi biaya, waktu, dan sumber daya. Oleh karena itu, dalam pemeliharaan dan pengembangan fitur pada proyek-proyek ini, pengembang tetap harus menggunakan dan menguasai RecyclerView.
2. **Interoperabilitas dan Proyek Hibrida** Jetpack Compose dirancang untuk dapat berinteroperasi dengan Android View System. Banyak tim pengembang yang mengadopsi Compose secara bertahap, di mana layar-layar baru dibuat menggunakan Compose, sementara layar yang sudah ada dan kompleks tetap dipertahankan dalam format XML. Dalam skenario hibrida seperti ini, RecyclerView dan LazyColumn dapat hidup berdampingan dalam satu basis kode yang sama.
3. **Kematangan, Stabilitas, dan Kontrol Tingkat Lanjut** RecyclerView telah diperkenalkan sejak tahun 2014 dan telah melalui berbagai iterasi optimalisasi. Komponen ini sangat matang, stabil, dan teruji dalam berbagai skenario produksi. RecyclerView memberikan developer kontrol yang sangat mendetail (*fine-grained control*) atas berbagai aspek, antara lain:
 - **LayoutManager**: Memungkinkan pembuatan struktur layout yang sangat kustom, melebihi sekadar daftar linear atau grid.
 - **ItemAnimator**: Memberikan kontrol penuh atas implementasi animasi untuk operasi penambahan, penghapusan, atau pembaruan item.
 - **RecycledViewPool**: Memfasilitasi pembagian View yang telah didaur ulang antar beberapa RecyclerView untuk mencapai tingkat optimalisasi performa yang lebih tinggi. Untuk kasus penggunaan yang sangat kompleks dan menuntut kustomisasi tingkat lanjut, RecyclerView masih menjadi pilihan yang solid.
4. **Ekosistem dan Basis Pengetahuan** Sebagai komponen yang telah lama ada, RecyclerView memiliki ekosistem yang sangat luas, mencakup pustaka pihak ketiga, artikel teknis, dan tutorial yang tak terhitung jumlahnya. Komunitas pengembang telah memecahkan berbagai

tantangan kompleks terkait `RecyclerView`, sehingga basis pengetahuan yang tersedia sangat melimpah.

5. **Keahlian dan Familiaritas Pengembang** Tidak semua tim pengembang telah sepenuhnya beralih ke Jetpack Compose. Banyak pengembang yang memiliki keahlian dan pengalaman mendalam dengan Android View System. Bagi mereka, implementasi fitur menggunakan `RecyclerView` dapat menjadi lebih cepat dan efisien karena tingkat familiaritas yang tinggi.

Kesimpulan

Meskipun `RecyclerView` memerlukan lebih banyak kode *boiler-plate* (misalnya, pembuatan `Adapter` dan `ViewHolder`) dibandingkan `LazyColumn`, ia bukanlah teknologi yang usang. Pemilihan antara kedua komponen ini tidak didasarkan pada superioritas absolut, melainkan pada **arsitektur dan teknologi dasar yang digunakan dalam sebuah proyek**.

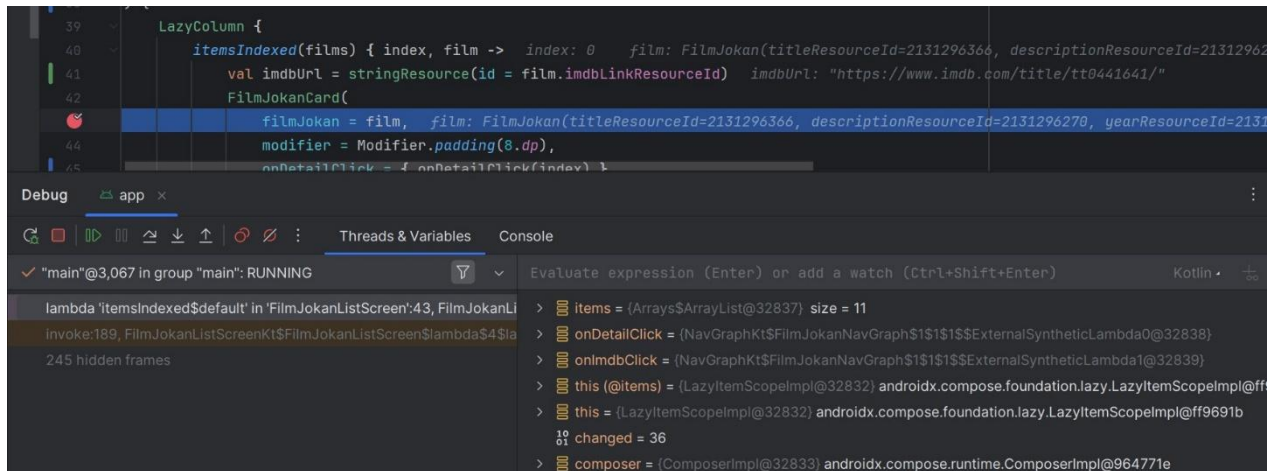
Untuk proyek yang dibangun di atas Android View System dengan layout XML, `RecyclerView` adalah komponen standar yang tepat untuk digunakan. Sebaliknya, untuk proyek yang sepenuhnya dibangun menggunakan Jetpack Compose, `LazyColumn` adalah pilihan utama yang modern dan efisien.

MODUL 4 : ViewModel and Debugging

SOAL 1

Soal Praktikum:

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
 - b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
 - c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
 - d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out
2. Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya
Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 25 Contoh Penggunaan Debugger

A. Source Code

Data/MovieRepository/ Compose

```

1 package com.example.modul4.data
2
3 import com.example.modul4.R
4 import com.example.modul4.model.Movie
5 import timber.log.Timber
6
7 object MovieRepository {
8     fun getMovies(): List<Movie> {
9         val movies = listOf(
10             Movie(
11                 title = "The Uncanny Counter",
12                 year = "2020",
13                 plot = "Para Counter yang karyawan toko mi di siang
14 hari dan pemburu iblis di malam hari, memakai berbagai kemampuan
15 khusus untuk memburu roh-roh jahat yang mengincar manusia.",
16                 imdb = "https://www.imdb.com/title/tt13273826/",
17                 posterResId = R.drawable.dukun
18             ),
19             Movie(
20                 title = "Sweet Home",
21                 year = "2020",
22                 plot = "Saat banyak manusia berubah menjadi monster
23 ganas dan dunia terpuruk ke dalam teror, sekelompok penyintas berjuang
24 untuk hidup-dan mempertahankan kemanusiaan mereka..",
25                 imdb =
26 "https://www.imdb.com/title/tt11612120/?ref_=nv_sr_srsq_0_tt_8_nm_0_
27 in_0_q_Sweet%2520Home",
28                 posterResId = R.drawable.rm
29             ),
30             Movie(
31                 title = "Money Heist Korea",
32                 year = "2022",
33                 plot = "Pencuri menguasai gedung percetakan uang
34 milik Korea bersatu dan menawan banyak sandera. Polisi harus
35 menghentikan aksi mereka, serta dalang yang bersembunyi di baliknya.",

```

```

36         imdb =
37         "https://www.imdb.com/title/tt13696452/?ref_=nv_sr_srsrg_0_tt_8_nm_0_
38         in_0_q_Money%2520Heist%2520Ko",
39         posterResId = R.drawable.duitheist
40     ),
41     Movie(
42         title = "My Name",
43         year = "2021",
44         plot = "Setelah ayahnya dibunuh, seorang wanita yang
45         ingin membalas dendam memutuskan untuk memercayai bos kriminal yang
46         berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
47         imdb =
48         "https://www.imdb.com/title/tt12940504/?ref_=nv_sr_srsrg_3_tt_8_nm_0_
49         in_0_q_My%2520Name",
50         posterResId = R.drawable.ngaran
51     ),
52     Movie(
53         title = "The 8 Show",
54         year = "2024",
55         plot = "Delapan orang yang terjebak di gedung delapan
56         lantai misterius mengikuti acara yang menarik tetapi berbahaya. Dengan
57         berjalannya waktu, mereka akan mendapat hadiah uang.",
58         imdb =
59         "https://www.imdb.com/title/tt30423279/?ref_=nv_sr_srsrg_0_tt_8_nm_0_
60         in_0_q_The%25208%2520Show",
61         posterResId = R.drawable.delapan
62     ),
63     Movie(
64         title = "Kingdom",
65         year = "2020",
66         plot = "Saat rumor aneh tentang raja yang sakit
67         menguasai kerajaan, putra mahkota menjadi satu-satunya harapan mereka
68         untuk melawan wabah misterius yang menjangkiti negeri.",
69         imdb =
70         "https://www.imdb.com/title/tt6611916/?ref_=nv_sr_srsrg_6_tt_8_nm_0_i
71         n_0_q_Kingdom",
72         posterResId = R.drawable.raja
73     ),
74     Movie(
75         title = "The Frog",
76         year = "2024",
77         plot = "Di musim panas nan damai, seorang wanita
78         misterius masuk ke sebuah penginapan-memicu beragam peristiwa yang
79         mengganggu kehidupan si pemilik dan orang-orang di sekitarnya.",
80         imdb = "https://www.imdb.com/title/tt26767508/",
81         posterResId = R.drawable.frog
82     ),
83     Movie(
84         title = "All of Us Are Dead",
85         year = "2022",
86         plot = "Sebuah SMA menjadi titik nol merebaknya wabah
87         virus zombi. Para murid yang terperangkap pun harus berjuang untuk
88         kabur jika tak mau terinfeksi dan berubah menjadi buas.",
89         imdb = "https://www.imdb.com/title/tt14169960/",
90         posterResId = R.drawable.allofus
91     ),

```

92	Movie(
93	title = "Taxi Driver",
94	year = "2021",
95	plot = "Seorang sopir taksi di Seoul mengantarkan seorang
96	wartawan Jerman untuk menyelidiki isu kerusuhan di Gwangju tanpa tahu
97	apa yang menanti mereka. Berdasarkan kisah nyata.",
98	imdb = "https://www.imdb.com/title/tt13759970/",
99	posterResId = R.drawable.taxi
10	),
0	Movie(
10	title = "A Killer Paradox",
1	year = "2024",
10	plot = "Saat suatu pembunuhan tak disengaja
2	memunculkan pembunuhan serupa lainnya, pemuda biasa ini terjebak dalam
10	aksi kucing-kucingan tanpa henti dengan detektif yang lihai.",
3	imdb = "https://www.imdb.com/title/tt28642796/",
10	posterResId = R.drawable.paradox
4	)
10	)
5	Timber.d("Data film berhasil dimuat. Total: \${movies.size}
10	film.")
	return movies
	}
	}

Table 27 Source Code Soal 1 - Compose

Model / Movie / Compose

1	package	com.example.scrollablelist_modul3.model
2		
3	data	class Movie(
4	val	title: String,
5	val	year: String,
6	val	plot: String,
7	val	imdb: String,
8	val	posterResId: Int
9	)	

Table 28 Source Code Soal 1 - Compose

navigation / AppNavigation / Compose

1	package	com.example.scrollablelist_modul3.navigation
2		
3		
4	import	androidx.compose.runtime.Composable
5	import	androidx.navigation.NavType
6	import	androidx.navigation.navArgument
7	import	androidx.navigation.compose.NavHost
8	import	androidx.navigation.compose.composable
9	import	androidx.navigation.compose.rememberNavController
	import	com.example.scrollablelist_modul3.ui.DetailScreen
	import	com.example.scrollablelist_modul3.ui.MovieListScreen
	import	com.example.scrollablelist_modul3.R
	@Composable	

	<pre> fun AppNavigation() { val navController = rememberNavController() NavHost(navController = navController, startDestination = Screen.MovieList.route) { composable(Screen.MovieList.route) { MovieListScreen(navController) } composable(route = Screen.Detail.route, arguments = listOf(navArgument("title") { type = NavType.StringType }, navArgument("year") { type = NavType.StringType }, navArgument("plot") { type = NavType.StringType }, navArgument("posterResId") { type = NavType.IntType }) { val backStackEntry -> { val title = backStackEntry.arguments?.getString("title") ?: "" val year = backStackEntry.arguments?.getString("year") ?: "" val plot = backStackEntry.arguments?.getString("plot") ?: "" val posterResId = backStackEntry.arguments?.getInt("posterResId") ?: R.drawable.ngaran DetailScreen(title = title, year = year, plot = plot, posterResId = posterResId) } } } } </pre>
--	---

Table 29 Source Code Soal 1 - Compose

navigation / Screen / Compose

1	package	com.example.scrollablelist_modul3.navigation
2	import	android.net.Uri
3		
4	sealed class	Screen(val route: String) {
5	object	MovieList : Screen("movie_list")
6	object	Detail :
7		Screen("movie_detail/{title}/{year}/{plot}/{posterResId}") {
8	fun	createRoute(title: String, year: String, plot: String,
9	posterResId:	Int): String {
	return	
		"movie_detail/\${Uri.encode(title)}/\${Uri.encode(year)}/\${Uri.encode(p

	<pre> lot) }/\$posterResId" } } } </pre>
--	--

Table 30 Source Code Soal 1 - Compose

ui.theme / DetailScreen / Compose

1	package	com.example.scrollablelist_modul3.ui
2		
3	import	androidx.compose.foundation.Image
4	import	androidx.compose.foundation.layout.*
5	import	androidx.compose.foundation.shape.RoundedCornerShape
6	import	androidx.compose.material3.*
7	import	androidx.compose.runtime.Composable
8	import	androidx.compose.ui.Modifier
9	import	androidx.compose.ui.draw.clip
	import	androidx.compose.ui.res.painterResource
	import	androidx.compose.ui.res.stringResource
	import	androidx.compose.ui.unit.dp
	import	com.example.scrollablelist_modul3.R
		@OptIn(ExperimentalMaterial3Api::class)
		@Composable
	fun	DetailScreen(title: String, year: String, plot: String,
	posterResId:	Int) {
	Scaffold(	
	topBar	= {
	TopAppBar(	
	title	= {
	Text(text	= "Movie Detail")
	}	
	)	
	}	
	) {	paddingValues ->
	Column(	
	modifier	= Modifier
	.fillMaxSize()	
	.padding(paddingValues)	
	.padding(16.dp)	
	)	{
	Image(	
	painter = painterResource(id = posterResId),	
	contentDescription = null,	
	modifier = Modifier	
	.fillMaxWidth()	
	.height(550.dp)	
	.clip(RoundedCornerShape(16.dp))	
	)	
	Spacer(modifier = Modifier.height(16.dp))	
	Text(	
	text	=
	String.format(stringResource(R.string.movie_detail_title),	
	title,	year),
	style = MaterialTheme.typography.headlineSmall	
	)	
	Spacer(modifier = Modifier.height(8.dp))	

	<pre> Text(text "\${stringResource(R.string.plot_label)}:\n\$plot") } } } </pre>	=
--	--	---

Table 31 Source Code Soal 1 - Compose

Ui.theme / MovieListScreen / Compose

1	package	com.example.modul4.ui.screens
2		
3	import	android.content.Intent
4	import	android.net.Uri
5	import	androidx.compose.foundation.Image
6	import	androidx.compose.foundation.layout.*
7	import	androidx.compose.foundation.lazy.LazyColumn
8	import	androidx.compose.foundation.lazy.items
9	import	androidx.compose.foundation.shape.RoundedCornerShape
	import	androidx.compose.material3.*
	import	androidx.compose.runtime.*
	import	androidx.compose.ui.Modifier
	import	androidx.compose.ui.draw.clip
	import	androidx.compose.ui.platform.LocalContext
	import	androidx.compose.ui.res.painterResource
	import	androidx.compose.ui.unit.dp
	import	androidx.navigation.NavController
	import	com.example.modul4.navigation.Screen
	import	com.example.modul4.viewmodel.MovieViewModel
	import	com.example.modul4.viewmodel.SharedMovieViewModel
	import	timber.log.Timber
	@OptIn(ExperimentalMaterial3Api::class)	
	@Composable	
	fun	MovieListScreen(
	navController:	NavController,
	movieViewModel:	MovieViewModel,
	sharedViewModel:	SharedMovieViewModel
	)	{
	val context	= LocalContext.current
	val movieList by movieViewModel.movies.collectAsState()	
	Scaffold(	
	topBar	= {
	TopAppBar(title = { Text("KDrama Cinema") })	}
	)	{
	paddingValues	->
	LazyColumn(	
	modifier	= Modifier
	.fillMaxSize()	
	.padding(paddingValues)	
	.padding(8.dp)	
	)	{
	items(movieList)	{ movie ->
	Card(	
	modifier	= Modifier
	.fillMaxWidth()	
	.padding(vertical = 8.dp),	

	<pre> shape = RoundedCornerShape(16.dp), elevation CardDefaults.cardElevation(4.dp)) Row(modifier = Modifier.padding(16.dp)) { Image(painter = painterResource(id = movie.posterResId), contentDescription = null, modifier = Modifier .size(width = 100.dp, height = 150.dp) .clip(RoundedCornerShape(8.dp))) Spacer(modifier = Modifier.width(16.dp)) Column(modifier = Modifier.weight(1f)) { Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) { Text(text = movie.title, style = MaterialTheme.typography.titleMedium) Text(text = movie.year, style = MaterialTheme.typography.bodySmall) } Spacer(modifier = Modifier.height(8.dp)) Text(text = movie.plot, style = MaterialTheme.typography.bodySmall, maxLines = 3) Spacer(modifier = Modifier.height(8.dp)) Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.End </pre>
--	---

	<pre> { private val _movies = MutableStateFlow<List<Movie>>(emptyList()) val movies: StateFlow<List<Movie>> get() = _movies private val _selectedMovie = MutableStateFlow<Movie?>(null) val selectedMovie: StateFlow<Movie?> get() = _selectedMovie init { val movieList = MovieRepository.getMovies() _movies.value = movieList Timber.d("User \$userId - Loaded movie list with \${movieList.size} items") } fun onMovieClick(movie: Movie) { _selectedMovie.value = movie Timber.d("User \$userId - Movie clicked: \${movie.title}") } } </pre>
--	---

Table 33 Source Code Soal 1 - Compose

Viewmodel / MovieViewModelFactory / Compose

1	package com.example.modul4.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class MovieViewModelFactory(private val userId: String) :
7	ViewModelProvider.Factory {
8	@Suppress("UNCHECKED_CAST")
9	override fun <T : ViewModel> create(modelClass: Class<T>):
	T {
	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java)) {
	return MovieViewModel(userId) as T
	}
	throw IllegalArgumentException("Unknown ViewModel
	class")
	}
	}

Table 34 Source Code Soal 1 - Compose

Viewmodel / SharedViewModel / Compose

1	package com.example.modul4.viewmodel
2	
3	import androidx.compose.runtime.getValue
4	import androidx.compose.runtime.mutableStateOf
5	import androidx.compose.runtime.setValue
6	import androidx.lifecycle.ViewModel
7	import com.example.modul4.model.Movie
8	
9	class SharedMovieViewModel : ViewModel() {
	var selectedMovie by mutableStateOf<Movie?>(null)
	private set

	<pre> fun selectMovie (movie: Movie) { selectedMovie = movie } </pre>
--	---

Table 35 Source Code Soal 1 - Compose

MainActivity / Compose

1	package	com.example.scrollablelist_modul3
2		
3	import	android.os.Bundle
4	import	androidx.activity.ComponentActivity
5	import	androidx.activity.compose.setContent
6	import	com.example.scrollablelist_modul3.navigation.AppNavigation
7	import	
8		com.example.scrollablelist_modul3.ui.theme.ScrollableListModul3Theme
9		
	class	MainActivity : ComponentActivity() {
	override fun	onCreate (savedInstanceState: Bundle?) {
	super.onCreate (savedInstanceState)	
	setContent	{
	ScrollableListModul3Theme	{
	AppNavigation()	
	}	
	}	
	}	
	}	

Table 36 Source Code Soal 1 - Compose

MyApplication.kt / Compose

1	package	com.example.modul4
2		
3	import	android.app.Application
4	import	com.example.modul4.BuildConfig
5	import	timber.log.Timber
6		
7	class	MyApplication : Application() {
8	override fun	onCreate() {
9	super.onCreate()	
	if (BuildConfig.DEBUG)	{
	Timber.plant (Timber.DebugTree())	
	}	
	}	
	}	

Table 37 Source Code Soal 1 - Compose

Data / DataSource / XML

```

1 package com.example.m3xml.data
2
3 import com.example.m3xml.R
4
5 object DataSource {
6     fun getFilms(): List<Film> {
7         return listOf(
8             Film(
9                 1,
10                "Unchanny Counter Season 1",
11                "2020",
12                "Para Counter yang karyawan toko mi di siang hari dan
13 pemburu iblis di malam hari, memakai berbagai kemampuan khusus untuk
14 memburu roh-roh jahat yang mengincar manusia.",
15                R.drawable.dukun,
16                "https://www.imdb.com/title/tt13273826/"
17            ),
18            Film(
19                2,
20                "Sweet Home",
21                "2020",
22                "Saat banyak manusia berubah menjadi monster ganas dan
23 dunia terpuruk ke dalam teror, sekelompok penyintas berjuang untuk
24 hidup-dan mempertahankan kemanusiaan mereka.",
25                R.drawable.rm,
26                "https://www.imdb.com/title/tt11612120/?ref=nv_sr_srsrg_0_tt_8_nm_0_
27 in_0_q_Sweet%2520Home"
28            ),
29            Film(
30                3,
31                "Money Heist Korea",
32                "2022",
33                "Pencuri menguasai gedung percetakan uang milik Korea
34 bersatu dan menawan banyak sandera. Polisi harus menghentikan aksi
35 mereka, serta dalang yang bersembunyi di baliknya.",
36                R.drawable.duitheist, // ganti dengan nama file gambar
37                Anda
38            ),
39            Film(
40                4,
41                "My Name",
42                "2021",
43                "Setelah ayahnya dibunuh, seorang wanita yang ingin
44 membalas dendam memutuskan untuk memercayai bos kriminal yang
45 berkuasa-dan memasuki kepolisian atas petunjuk pria itu.",
46                R.drawable.ngaran, // ganti dengan nama file gambar
47                Anda
48            ),
49            Film(
50                5,
51                "The 8 Show",
52                "2021",
53                "Seorang pria yang terjebak dalam lift selama 8 jam dan
54 harus bertahan hidup dengan menggunakan akal budi dan keberanian.",
55                R.drawable.lift, // ganti dengan nama file gambar
56                Anda
57            ),
58            Film(
59                6,
60                "The 8 Show",
61                "2021",
62                "Seorang pria yang terjebak dalam lift selama 8 jam dan
63 harus bertahan hidup dengan menggunakan akal budi dan keberanian.",
64                R.drawable.lift, // ganti dengan nama file gambar
65                Anda
66            )
67        )
68    }
69 }

```

3	in_0_q_My%2520Name"
2	),
3	Film(
3	5,
3	"The 8 Show",
4	"2024",
3	"Indonesia's preeminent superhero and his alter ego
5	Sancaka enter the cinematic universe to battle the wicked Pengkor and
3	his orphan army.",
6	R.drawable.delapan, // ganti dengan nama file gambar
3	Anda
7	"https://www.imdb.com/title/tt9201084/"
3	)
8	)
3	}
	}

Table 38 Source Code Soal 1 - XML

Data / Film / XML

1	package	com.example.m3xml.data
2		
3	import	android.os.Parcelable
4	import	kotlinx.parcelize.Parcelize
5		
6	@Parcelize	
7	data	class Film(
8	val	id: Int,
9	val	title: String,
10	val	year: String,
11	val	plot: String,
1	val	poster: Int,
	val	imdbUrl: String
	) : Parcelable	

Table 39 Source Code Soal 1 - XML

Ui.theme / DetailFragment / XML

1	package	com.example.m3xml.ui.theme
2		
3	import	android.content.Intent
4	import	android.net.Uri
5	import	android.os.Bundle
6	import	android.view.View
7	import	androidx.fragment.app.Fragment
8	import	androidx.fragment.app.activityViewModels
9	import	androidx.navigation.fragment.navArgs
10	import	com.example.m3xml.R
11	import	com.example.m3xml.data.Film
12	import	com.example.m3xml.databinding.FragmentDetailBinding
13	import	com.example.m3xml.viewmodel.FilmViewModel
14	import	com.example.m3xml.viewmodel.ViewModelFactory
15	import	timber.log.Timber
16		
17	class DetailFragment	: Fragment(R.layout.fragment_detail) {
18		

19	private var _binding: FragmentDetailBinding? = null
20	private val binding get() = _binding!!
21	private val viewModel: FilmViewModel by activityViewModels
22	{
23	ViewModelFactory("Ini dari DetailFragment")
24	}
25	private val args: DetailFragmentArgs by navArgs()
26	
27	override fun onCreateView(view: View, savedInstanceState:
28	Bundle?) {
29	super.onCreateView(view, savedInstanceState)
30	_binding = FragmentDetailBinding.bind(view)
31	
32	val filmId = args.filmId
33	
34	val film: Film? = viewModel.getFilmById(filmId)
35	
36	if (film != null) {
37	Timber.d("Membuka halaman detail untuk film:
38	ID=\${film.id}, Nama='\${film.title}')
39	
40	binding.ivDetailPoster.setImageResource(film.poster)
41	binding.tvDetailTitle.text = "\${film.title}
42	(\${film.year})"
43	binding.tvDetailPlot.text = film.plot
44	binding.ivDetailPoster.setOnClickListener {
45	Timber.d("Poster '\${film.title}' diklik,
46	membuka URL IMDb.")
47	val openUrlIntent = Intent(Intent.ACTION_VIEW,
48	Uri.parse(film.imdbUrl))
49	startActivity(openUrlIntent)
	}
	} else {
	Timber.e("Film dengan ID \$filmId tidak ditemukan.")
	}
	}
	override fun onDestroyView() {
	super.onDestroyView()
	_binding = null
	}
	}

Table 40 Source Code Soal 1 - XML

Ui.theme / FilmAdapter / XML

1	package com.example.m3xml.ui.theme
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.DiffUtil
6	import androidx.recyclerview.widget.ListAdapter
7	import androidx.recyclerview.widget.RecyclerView
8	import com.example.m3xml.data.Film
9	import com.example.m3xml.databinding.ListItemBinding


```

10
11 class                                     FilmAdapter(
12     private val onDetailClick: (Film) -> Unit,
13     private val onImdbClick: (Film) -> Unit
14 ) : ListAdapter<Film,
15 FilmAdapter.FilmViewHolder>(FilmDiffCallback()) {
16
17     override fun onCreateViewHolder(parent: ViewGroup, viewType:
18 Int): FilmViewHolder {
19         val binding =
20 ListItemBinding.inflate(LayoutInflater.from(parent.context),
21 parent, false)
22         return FilmViewHolder(binding)
23     }
24
25     override fun onBindViewHolder(holder: FilmViewHolder,
26 position: Int) {
27         val film = getItem(position)
28         holder.bind(film)
29     }
30
31     inner class FilmViewHolder(private val binding:
32 ListItemBinding) :
33 RecyclerView.ViewHolder(binding.root) {
34
35         fun bind(film: Film) {
36             binding.ivItemPoster.setImageResource(film.poster)
37             binding.tvItemTitle.text = film.title
38
39             binding.buttonItemDetail.setOnClickListener {
40                 onDetailClick(film)
41             }
42
43             binding.buttonItemImdb.setOnClickListener {
44                 onImdbClick(film)
45             }
46         }
47     }
48 }
49
50 class FilmDiffCallback : DiffUtil.ItemCallback<Film>() {
51     override fun areItemsTheSame(oldItem: Film, newItem: Film):
52 Boolean {
53         return oldItem.id == newItem.id
54     }
55
56     override fun areContentsTheSame(oldItem: Film, newItem:
57 Film): Boolean {
58         return oldItem == newItem
59     }
60 }

```

Table 41 Source Code Soal 1 - XML

Ui.theme / ListFragment / XML

```
1 package com.example.m3xml.ui.theme
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import android.view.View
7 import androidx.fragment.app.Fragment
8 import androidx.fragment.app.activityViewModels
9 import androidx.lifecycle.Lifecycle
10 import androidx.lifecycle.lifecycleScope
11 import androidx.lifecycle.repeatOnLifecycle
12 import androidx.navigation.fragment.findNavController
13 import androidx.recyclerview.widget.LinearLayoutManager
14 import com.example.m3xml.R
15 import com.example.m3xml.databinding.FragmentListBinding
16 import com.example.m3xml.viewmodel.FilmViewModel
17 import com.example.m3xml.viewmodel.ViewModelFactory
18 import kotlinx.coroutines.launch
19 import timber.log.Timber
20
21 class ListFragment : Fragment(R.layout.fragment_list) {
22
23     private var _binding: FragmentListBinding? = null
24     private val binding get() = _binding!!
25
26     private val viewModel: FilmViewModel by activityViewModels {
27         ViewModelFactory("Ini dari ListFragment")
28     }
29
30     override fun onCreateView(view: View, savedInstanceState:
31 Bundle?) {
32         super.onCreateView(view, savedInstanceState)
33         _binding = FragmentListBinding.inflate(layoutInflater, view, true)
34
35         val filmAdapter = FilmAdapter(
36             onDetailClick = { film ->
37                 Timber.d("Tombol detail untuk film
38 '${film.title}' ditekan.")
39                 viewModel.onFilmClicked(film)
40             },
41             onImdbClick = { film ->
42                 Timber.d("Tombol IMDb untuk film '${film.title}'
43 ditekan.")
44                 val openUrlIntent = Intent(Intent.ACTION_VIEW,
45 Uri.parse(film.imdbUrl))
46                 startActivity(openUrlIntent)
47             }
48         )
49
50         binding.recyclerView.apply {
51             layoutManager = LinearLayoutManager(context)
52             adapter = filmAdapter
53         }
54     }
55 }
```

	<pre> viewLifecycleOwner.lifecycleScope.launch { repeatOnLifecycle(Lifecycle.State.STARTED) { launch { viewModel.filmList.collect { filmList -> filmAdapter.submitList(filmList) } } launch { viewModel.navigateToDetail.collect { film -> film?.let { val action = ListFragmentDirections.actionListFragmentToDetailFragment(it.id) findNavController().navigate(action) } } } viewModel.onNavigateToDetailComplete() } } override fun onDestroyView() { super.onDestroyView() _binding = null } </pre>
--	---

Table 42 Source Code Soal 1 - XML

Viewmodel / FilmViewModel / XML

1	package	com.example.m3xml.viewmodel
2		
3	import	androidx.lifecycle.ViewModel
4	import	com.example.m3xml.data.DataSource
5	import	com.example.m3xml.data.Film
6	import	kotlinx.coroutines.flow.MutableStateFlow
7	import	kotlinx.coroutines.flow.StateFlow
8	import	kotlinx.coroutines.flow.asStateFlow
9	import	kotlinx.coroutines.flow.update
10	import	timber.log.Timber
11		
12	class	FilmViewModel(private val someString: String) :
13	ViewModel()	{
14	private	val _filmList =
15	MutableStateFlow<List<Film>>	(emptyList())
16	val	filmList: StateFlow<List<Film>> = _filmList
17		
18	private	val _navigateToDetail =
19	MutableStateFlow<Film?>	(null)
20	val	navigateToDetail: StateFlow<Film?> =
	_navigateToDetail.asStateFlow()	
	init	{
	Timber.i("FilmViewModel	created with string:
	\$someString")	
	loadFilms()	

	<pre> } private fun loadFilms() { _filmList.value = DataSource.getFilms() Timber.i("Data film berhasil dimuat. Jumlah data: \${_filmList.value.size}") } fun getFilmById(id: Int): Film? { return _filmList.value.find { it.id == id } } fun onFilmClicked(film: Film) { _navigateToDetail.update { film } } fun onNavigateToDetailComplete() { _navigateToDetail.update { null } } } </pre>
--	---

Table 43 Source Code Soal 1 - XML

MainActivity.kt / XML

1	package	com.example.m3xml
2		
3	import	android.os.Bundle
4	import	androidx.appcompat.app.AppCompatActivity
5	import	com.example.m3xml.databinding.ActivityMainBinding //
6	Ganti	dengan package Anda
7		
8	class	MainActivity : AppCompatActivity() {
9		
10	private lateinit	var binding: ActivityMainBinding
11		
12	override fun	onCreate(savedInstanceState: Bundle?) {
13		super.onCreate(savedInstanceState)
14		binding = ActivityMainBinding.inflate(layoutInflater)
15		setContentView(binding.root)
16		}
17	}	}

Table 44 Source Code Soal 1 - XML

MyAppllication.kt / XML

1	package	com.example.m3xml
2		
3	import	android.app.Application
4	import	timber.log.Timber
5		
6	class	MyApplication : Application() {
7		
8	override fun	onCreate() {
9		super.onCreate()
10	if	(BuildConfig.DEBUG) {
11		Timber.plant(Timber.DebugTree())

12	}
13	}
14	}
15	
16	
17	

Table 45 Source Code Soal 1 - XML

Layout / activity_main.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:layout_width="match_parent"		
7	android:layout_height="match_parent"		
8	tools:context=".MainActivity">		
9			
10	<androidx.fragment.app.FragmentContainerView		
11	android:id="@+id/nav_host_fragment"		
12			
13	android:name="androidx.navigation.fragment.NavHostFragment"		
14	android:layout_width="0dp"		
15	android:layout_height="0dp"		
16	app:defaultNavHost="true"		
17	app:layout_constraintBottom_toBottomOf="parent"		
18	app:layout_constraintEnd_toEndOf="parent"		
19	app:layout_constraintStart_toStartOf="parent"		
20	app:layout_constraintTop_toTopOf="parent"		
21	app:navGraph="@navigation/nav_graph"		/>
22			
23	</androidx.constraintlayout.widget.ConstraintLayout>		

Table 46 Source Code Soal 1 - XML

Layout / fragment_detail.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<ScrollView		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:layout_width="match_parent"		
7	android:layout_height="match_parent"		
8	tools:context=".ui.theme.DetailFragment">		
9			
1	<androidx.constraintlayout.widget.ConstraintLayout		
0	android:layout_width="match_parent"		
1	android:layout_height="wrap_content"		
1	android:paddingBottom="16dp">		
1			
2	<TextView		
1	android:id="@+id/tv_header_detail"		
3	android:layout_width="0dp"		

```

1         android:layout_height="wrap_content"
4         android:layout_marginStart="16dp"
1         android:layout_marginTop="16dp"
5         android:layout_marginEnd="16dp"
1         android:text="Movie" Detail"
6
1     android:textAppearance="@style/TextAppearance.Material3.HeadlineLarge"
7
1         android:textStyle="bold"
8         app:layout_constraintEnd_toEndOf="parent"
1         app:layout_constraintStart_toStartOf="parent"
9         app:layout_constraintTop_toTopOf="parent" />
2
0     <com.google.android.material.imageview.ShapeableImageView
2         android:id="@+id/iv_detail_poster"
1         android:layout_width="0dp"
2         android:layout_height="0dp"
2         android:layout_marginStart="16dp"
2         android:layout_marginTop="16dp"
3         android:layout_marginEnd="16dp"
2         android:scaleType="centerCrop"
4         app:layout_constraintDimensionRatio="2:3"
2         app:layout_constraintEnd_toEndOf="parent"
5         app:layout_constraintStart_toStartOf="parent"
2
6     app:layout_constraintTop_toBottomOf="@id/tv_header_detail"
2         app:shapeAppearanceOverlay="@style/roundedImageView"
7         tools:srcCompat="@drawable/dukun" />
2
8     <TextView
2         android:id="@+id/tv_detail_title"
9         android:layout_width="0dp"
3         android:layout_height="wrap_content"
0         android:layout_marginTop="16dp"
3
1         android:textAppearance="?attr/textAppearanceHeadlineSmall"
3         android:textStyle="bold"
2         app:layout_constraintEnd_toEndOf="@id/iv_detail_poster"
3
3         app:layout_constraintStart_toStartOf="@id/iv_detail_poster"
3
4         app:layout_constraintTop_toBottomOf="@id/iv_detail_poster"
3         tools:text="Uncanny Counter Season 1 (2020)" />
5
3     <TextView
6         android:id="@+id/tv_detail_plot_label"
3         android:layout_width="wrap_content"
7         android:layout_height="wrap_content"
3         android:layout_marginTop="16dp"
8         android:text="Plot:"
3         android:textAppearance="?attr/textAppearanceTitleMedium"
9         android:textStyle="bold"
4
0         app:layout_constraintStart_toStartOf="@id/tv_detail_title"
4
1         app:layout_constraintTop_toBottomOf="@id/tv_detail_title" />

```

4	
2	<TextView
4	android:id="@+id/tv_detail_plot"
3	android:layout_width="0dp"
4	android:layout_height="wrap_content"
4	android:layout_marginTop="4dp"
4	android:textAppearance="?attr/textAppearanceBodyMedium"
5	app:layout_constraintEnd_toEndOf="@id/tv_detail_title"
4	
6	app:layout_constraintStart_toStartOf="@id/tv_detail_plot_label"
4	
7	app:layout_constraintTop_toBottomOf="@id/tv_detail_plot_label"
4	tools:text="Para Counter yang karyawan toko mi di siang
8	hari dan pemburu iblis di malam hari, memakai berbagai kemampuan
4	khusus untuk memburu roh-roh jahat yang mengincar manusia." />
	</androidx.constraintlayout.widget.ConstraintLayout>
	</ScrollView>

Table 47 Source Code Soal 1 - XML

Layout / fragment_list.xml / XML

1	<?xml	version="1.0"	encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout		
3	xmlns:android="http://schemas.android.com/apk/res/android"		
4	xmlns:app="http://schemas.android.com/apk/res-auto"		
5	xmlns:tools="http://schemas.android.com/tools"		
6	android:layout_width="match_parent"		
7	android:layout_height="match_parent"		
8	tools:context=".ui.theme.ListFragment">		
9			
1	<TextView		
0	android:id="@+id/tv_header_title"		
1	android:layout_width="0dp"		
1	android:layout_height="wrap_content"		
1	android:layout_marginStart="16dp"		
2	android:layout_marginTop="16dp"		
1	android:layout_marginEnd="16dp"		
3	android:text="KDrama		Cinema"
1			
4	android:textAppearance="@style/TextAppearance.Material3.HeadlineLarge"		
1			
5	android:textStyle="bold"		
1	app:layout_constraintEnd_toEndOf="parent"		
6	app:layout_constraintStart_toStartOf="parent"		
1	app:layout_constraintTop_toTopOf="parent"		/>
7			
1	<androidx.recyclerview.widget.RecyclerView		
8	android:id="@+id/recycler_view"		
1	android:layout_width="0dp"		
9	android:layout_height="0dp"		
2	android:layout_marginTop="16dp"		
0	android:clipToPadding="false"		
2	android:paddingBottom="8dp"		
1			
2	app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"		
2	app:layout_constraintBottom_toBottomOf="parent"		

2	app:layout_constraintEnd_toEndOf="parent"	
3	app:layout_constraintStart_toStartOf="parent"	
2	app:layout_constraintTop_toBottomOf="@id/tv_header_title"	
4	tools:listitem="@layout/list_item"	/>
2		
5	</androidx.constraintlayout.widget.ConstraintLayout>	

Table 48 Source Code Soal 1 - XML

Layout / list_item.xml / XML

1	<?xml version="1.0" encoding="utf-8"?>	
2	<androidx.cardview.widget.CardView	
3	xmlns:android="http://schemas.android.com/apk/res/android"	
4	xmlns:app="http://schemas.android.com/apk/res-auto"	
5	xmlns:tools="http://schemas.android.com/tools"	
6	android:layout_width="match_parent"	
7	android:layout_height="wrap_content"	
8	android:layout_marginStart="8dp"	
9	android:layout_marginTop="4dp"	
10	android:layout_marginEnd="8dp"	
11	android:layout_marginBottom="4dp"	
12	app:cardCornerRadius="8dp"	
13	app:cardElevation="2dp">	
14		
15	<androidx.constraintlayout.widget.ConstraintLayout	
16	android:layout_width="match_parent"	
17	android:layout_height="wrap_content"	
18	android:paddingBottom="8dp">	
19		
20	<ImageView	
21	android:id="@+id/iv_item_poster"	
22	android:layout_width="100dp"	
23	android:layout_height="150dp"	
24	android:scaleType="centerCrop"	
25	app:layout_constraintBottom_toBottomOf="parent"	
26	app:layout_constraintStart_toStartOf="parent"	
27	app:layout_constraintTop_toTopOf="parent"	
28	tools:srcCompat="@tools/sample/avatars" />	
29		
30	<TextView	
31	android:id="@+id/tv_item_title"	
32	android:layout_width="0dp"	
33	android:layout_height="wrap_content"	
34	android:layout_marginStart="16dp"	
35	android:layout_marginTop="8dp"	
36	android:layout_marginEnd="16dp"	
37		
38	android:textAppearance="?attr/textAppearanceTitleMedium"	
39	android:textStyle="bold"	
40	app:layout_constraintEnd_toEndOf="parent"	
41		
42	app:layout_constraintStart_toEndOf="@id/iv_item_poster"	
43	app:layout_constraintTop_toTopOf="parent"	
44	tools:text="Judul Film yang Sangat Panjang Sekali"	
45	/>	


```

46         <LinearLayout
47             android:layout_width="0dp"
48             android:layout_height="wrap_content"
49             android:layout_marginTop="8dp"
             android:orientation="horizontal"

app:layout_constraintEnd_toEndOf="@id/tv_item_title"

app:layout_constraintStart_toStartOf="@id/tv_item_title"

app:layout_constraintTop_toBottomOf="@id/tv_item_title">

        <Button
            android:id="@+id/button_item_detail"
            style="?attr/materialButtonOutlinedStyle"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginEnd="4dp"
            android:layout_weight="1"
            android:text="Detail"                                />

        <Button
            android:id="@+id/button_item_imdb"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="4dp"
            android:layout_weight="1"
            android:text="IMDb"                                />

    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>

</androidx.cardview.widget.CardView>

```

15	
16	android:id="@+id/action_listFragment_to_detailFragment"
17	app:destination="@id/detailFragment" />
18	</fragment>
19	<fragment
20	android:id="@+id/detailFragment"
21	
22	android:name="com.example.m3xml.ui.theme.DetailFragment"
23	android:label="fragment_detail"
24	tools:layout="@layout/fragment_detail" >
25	<argument
26	android:name="filmId"
27	app:argType="integer" />
28	</fragment>
29	</navigation>

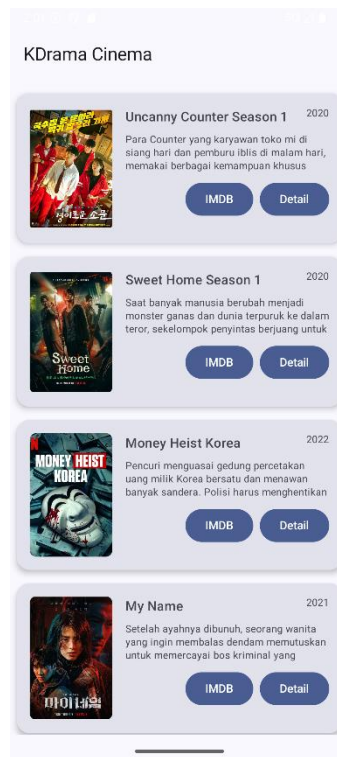
Table 50 Source Code Soal 1 - XML

Values/ themes.xml / XML

1	<resources xmlns:tools="http://schemas.android.com/tools">
2	<style name="Base.Theme.M3XML"
3	parent="Theme.Material3.DayNight.NoActionBar">
4	</style>
5	
6	<style name="Theme.M3XML" parent="Base.Theme.M3XML" />
7	
8	<style name="roundedImageView">
9	<item name="cornerFamily">rounded</item>
10	<item name="cornerSize">16dp</item>
11	</style>
12	
13	</resources>

Table 51 Source Code Soal 1 - XML

B. Output Program



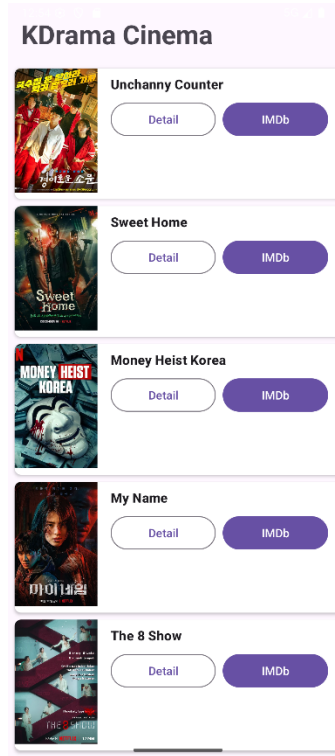
Gambar 26 Screenshot Hasil Jawaban Soal 1 - Compose



Gambar 27 Screenshot Hasil Jawaban Soal 1 - Compose



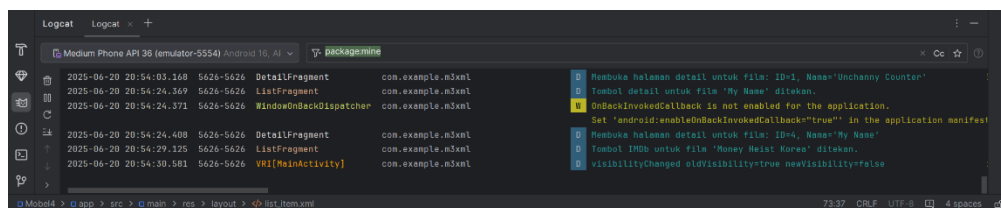
Gambar 28 Screenshot Penggunaan Timber Soal 1 - Compose



Gambar 29 Screenshot Hasil Jawaban Soal 1 - XML



Gambar 30 Screenshot Hasil Jawaban Soal 1 - XML



Gambar 31 Screenshot Penggunaan Timber Soal 1 – XML

C. Pembahasan Jetpack Compose

1. model/Movie.kt (Struktur Data)

File ini adalah fondasi dari data yang akan ditampilkan.

- **Pada baris 3**, data class Movie dideklarasikan. data class adalah jenis kelas di Kotlin yang tujuan utamanya hanya untuk menyimpan data. Kelas ini mendefinisikan "cetakan" atau struktur untuk sebuah objek film, yang terdiri dari id (unik), title (judul), description (deskripsi), dan imageRes (ID gambar dari drawable).

2. data/MovieRepository.kt (Sumber Data)

File ini bertindak sebagai "database" tiruan atau sumber data untuk aplikasi.

- **Pada baris 5**, object MovieRepository dideklarasikan. Penggunaan object menjadikan MovieRepository sebuah *singleton*, artinya hanya ada satu instance dari repositori ini di seluruh aplikasi. Ini memastikan data yang diakses dari layar mana pun selalu konsisten.
- **Pada baris 6**, private val movies dideklarasikan. Ini adalah sebuah List privat yang berisi semua objek Movie yang akan ditampilkan. Data film di-hardcode langsung di sini.
- **Pada baris 41**, fungsi getMovies() didefinisikan untuk mengembalikan seluruh daftar film yang ada.
- **Pada baris 44**, fungsi getMovieById(id: Int) didefinisikan untuk mencari dan mengembalikan satu objek Movie berdasarkan id yang diberikan.

3. navigation/Screen.kt (Definisi Rute Navigasi)

File ini mendefinisikan semua kemungkinan tujuan (layar) navigasi dalam aplikasi secara terstruktur.

- **Pada baris 3**, sealed class Screen dideklarasikan. sealed class digunakan di sini untuk membuat grup rute yang terbatas dan aman-tipe (*type-safe*). Ini mencegah kesalahan pengetikan nama rute.
- **Pada baris 4**, object MovieList merepresentasikan rute untuk layar daftar film.
- **Pada baris 5**, object Detail merepresentasikan rute untuk layar detail film. Rute ini didefinisikan sebagai "detail/{movieId}", di mana {movieId} adalah placeholder untuk argumen (ID film) yang akan dikirimkan.
- **Pada baris 7**, fungsi createRoute(movieId: Int) dibuat untuk memudahkan pembuatan rute lengkap ke layar detail dengan menyisipkan ID film yang sebenarnya ke dalam placeholder.

4. navigation/AppNavigation.kt (Pengatur Navigasi)

File ini adalah pusat kendali navigasi. Ia mengatur bagaimana semua layar terhubung.

- **Pada baris 11**, fungsi Composable AppNavigation dideklarasikan, yang menjadi inti dari sistem navigasi.
- **Pada baris 12**, val navController = rememberNavController() digunakan untuk membuat dan mengingat NavController, yaitu objek yang bertanggung jawab untuk mengelola navigasi (seperti Maps(), popBackStack(), dll).
- **Pada baris 13**, NavHost digunakan sebagai kontainer yang akan menampilkan tujuan (layar) Composable berdasarkan rute saat ini. startDestination diatur ke Screen.MovieList.route, yang berarti layar daftar film adalah layar pertama yang ditampilkan saat aplikasi dibuka.
- **Pada baris 17**, composable(route = Screen.MovieList.route) mendefinisikan apa yang harus ditampilkan untuk rute daftar film. Di sini, ia memanggil MovieListScreen, sambil meneruskan navController agar layar tersebut bisa melakukan navigasi.
- **Pada baris 21**, composable(route = Screen.Detail.route, ...) mendefinisikan layar detail.
- **Pada baris 22**, arguments = listOf(navArgument("movieId") { type = NavType.IntType }) mendefinisikan bahwa rute ini menerima sebuah argumen bernama "movieId" yang bertipe Int.
- **Pada baris 25**, val movieId = backStackEntry.arguments?.getInt("movieId") ?: 0 digunakan untuk mengekstrak nilai movieId yang dikirim dari layar sebelumnya.
- **Pada baris 26**, DetailScreen dipanggil dengan movieId yang telah diekstrak, memungkinkan layar detail untuk menampilkan data yang benar.

5. ui/theme/MovieListScreen.kt (Layar Daftar Film)

Ini adalah Composable untuk layar utama yang menampilkan daftar semua film.

- **Pada baris 18**, fungsi Composable MovieListScreen dideklarasikan, menerima navController sebagai parameter untuk keperluan navigasi.
- **Pada baris 19**, val movies = MovieRepository.getMovies() digunakan untuk mengambil data seluruh film dari repositori.
- **Pada baris 21**, LazyColumn digunakan untuk menampilkan daftar yang bisa di-scroll. LazyColumn sangat efisien karena hanya merender item yang terlihat di layar.
- **Pada baris 24**, items(movies) melakukan iterasi pada setiap movie di dalam daftar movies dan membuat sebuah MovieItem untuk masing-masing film.
- **Pada baris 29**, di dalam MovieItem, Card digunakan untuk membungkus setiap item daftar dengan tampilan kartu yang memiliki bayangan dan sudut membulat.
- **Pada baris 32**, modifier = Modifier.clickable { ... } membuat seluruh kartu dapat diklik.
- **Pada baris 33**, di dalam blok clickable, navController.navigate(Screen.Detail.createRoute(movie.id)) dieksekusi. Ini adalah aksi

navigasi yang membawa pengguna ke layar detail, sambil mengirimkan id dari film yang diklik.

- **Pada baris 37**, Row digunakan untuk menyusun gambar dan teks (judul, deskripsi) secara horizontal.
- **Pada baris 39**, Image digunakan untuk menampilkan poster film. painterResource memuat gambar dari drawable.
- **Pada baris 45**, Column digunakan untuk menyusun judul dan deskripsi film secara vertikal.

6. ui/theme/DetailScreen.kt (Layar Detail Film)

Ini adalah Composable untuk layar yang menampilkan detail dari satu film yang dipilih.

- **Pada baris 19**, fungsi Composable DetailScreen dideklarasikan, menerima movieId dan navController sebagai parameter.
- **Pada baris 20**, val movie = MovieRepository.getMovieById(movieId) digunakan untuk mengambil data spesifik dari satu film berdasarkan movieId yang diterima dari navigasi.
- **Pada baris 22**, BackHandler(onBack = { navController.popBackStack() }) digunakan untuk menangani penekanan tombol kembali (baik fisik maupun gestur). navController.popBackStack() akan mengembalikan pengguna ke layar sebelumnya (yaitu MovieListScreen).
- **Pada baris 25**, Column digunakan untuk menyusun semua elemen UI detail secara vertikal dan diposisikan di tengah.
- **Pada baris 32**, Image digunakan untuk menampilkan poster film yang dipilih dengan ukuran yang lebih besar.
- **Pada baris 38 & 39**, Text digunakan untuk menampilkan judul dan deskripsi film dengan gaya teks yang berbeda untuk memberikan penekanan.

7. MainActivity.kt (Aktivitas Utama)

File ini adalah titik masuk aplikasi, tempat semuanya dimulai.

- **Pada baris 16**, di dalam onCreate, setContent dipanggil untuk memulai UI Jetpack Compose.
- **Pada baris 18**, Surface digunakan sebagai container latar belakang.
- **Pada baris 22**, AppNavigation() dipanggil. Ini adalah satu-satunya panggilan penting di sini, yang secara efektif menyerahkan seluruh kontrol UI dan navigasi ke Composable AppNavigation

XML

1. Data / DataSource (DataSource.kt)

File ini berfungsi sebagai sumber data statis untuk aplikasi.

- **Pada baris 5**, object DataSource digunakan untuk mendeklarasikan kelas sebagai sebuah *singleton*. Pola desain ini memastikan bahwa hanya ada satu instance dari DataSource yang digunakan di seluruh siklus hidup aplikasi, sehingga data yang diakses bersifat konsisten dan terpusat.
- **Pada baris 6**, fun getFilms(): List<Film> adalah sebuah fungsi publik yang ketika dipanggil akan mengembalikan data berupa List (daftar) yang berisi objek-objek dari kelas Film.
- **Pada baris 7**, return listOf(...) mengembalikan sebuah List yang nilainya telah ditentukan secara langsung di dalam kode (statis).
- **Pada baris 8-14**, Film(1, "Unchanny Counter Season 1", ...) merupakan proses inisialisasi atau pembuatan instance dari objek Film. Setiap argumen yang dilewatkan (seperti id, judul, tahun, plot, ID resource drawable, dan URL) mengisi properti yang telah didefinisikan di dalam data class Film.

2. Data / Film (Film.kt)

File ini mendefinisikan model atau struktur data untuk entitas film.

- Pada baris 7, **data class Film(...)** adalah sebuah fitur Kotlin untuk membuat kelas yang tujuan utamanya adalah untuk menyimpan data. Kelas ini mendefinisikan properti yang dimiliki oleh sebuah entitas film, yaitu id, title, year, plot, poster, dan imdbUrl.
- Pada baris 6, **@Parcelize** merupakan sebuah anotasi dari library Kotlin Android Extensions yang secara otomatis meng-generate implementasi dari Parcelable.
- Pada baris 14, **: Parcelable** adalah implementasi dari interface Parcelable milik Android. Mekanisme ini memungkinkan objek dari kelas Film untuk diserialisasi sehingga dapat dikirim antar komponen Android, misalnya sebagai argumen dalam navigasi antar Fragment.

3. Viewmodel / FilmViewModel (FilmViewModel.kt)

Kelas ini bertanggung jawab untuk menyimpan, mengelola, dan menyediakan data yang dibutuhkan oleh UI, serta bertahan dari perubahan konfigurasi seperti rotasi layar.

- Pada blok init, sebuah log Timber.i(...) ditempatkan untuk mencatat peristiwa pembuatan ViewModel, memenuhi syarat logging (poin d.a).
- Pada metode loadFilms, setelah data berhasil diambil dari DataSource, log Timber.i(...) kedua dicatat untuk menandakan bahwa data telah berhasil dimuat ke dalam list (poin d.a).
- Sebuah fungsi publik baru, getFilmById(id: Int), ditambahkan. Fungsi ini berperan penting untuk menyediakan data objek Film yang lengkap kepada DetailFragment hanya dengan menggunakan ID sebagai parameter pencarian.

4. Ui.theme / FilmAdapter (FilmAdapter.kt)

Kelas ini berfungsi sebagai jembatan antara sumber data (list film) dan RecyclerView yang menampilkannya.

- Konstruktor kelas FilmAdapter dimodifikasi untuk menerima dua parameter fungsi (lambda), yaitu onDetailClick dan onImdbClick. Ini memungkinkan penanganan aksi klik yang berbeda untuk setiap tombol.
- Di dalam ViewHolder, pada metode bind, data dari objek Film diikat ke View yang sesuai menggunakan ID yang benar (iv_item_poster, tv_item_title).
- Sebuah setOnClickListener dipasang pada button_item_detail. Ketika diklik, listener ini akan memanggil fungsi onDetailClick yang telah diteruskan dari ListFragment.
- Secara serupa, setOnClickListener juga dipasang pada button_item_imdb yang akan memanggil fungsi onImdbClick.

5. Ui.theme / ListFragment (ListFragment.kt)

Fragment ini bertindak sebagai controller untuk UI yang menampilkan daftar film.

- Saat membuat FilmAdapter, dua fungsi lambda didefinisikan dan diteruskan sebagai parameter.
- Lambda untuk onDetailClick berisi dua aksi: pertama, mencatat log peristiwa dengan Timber.d("Tombol detail ... ditekan.") (memenuhi syarat d.b); kedua, memanggil metode pada ViewModel untuk memicu event navigasi.
- Lambda untuk onImdbClick juga mencatat log dengan Timber.d("Tombol IMDb ... ditekan.") dan kemudian membuat sebuah Intent dengan ACTION_VIEW untuk membuka URL IMDb di browser eksternal.
- RecyclerView dikonfigurasi menggunakan LinearLayoutManager untuk memastikan item ditampilkan dalam format daftar vertikal sesuai desain awal.

6. Ui.theme / DetailFragment (DetailFragment.kt)

Fragment ini bertindak sebagai controller untuk UI yang menampilkan rincian dari satu film yang dipilih.

- Pertama, Fragment mengambil filmid (sebuah Integer) dari argumen navigasi yang diterima.
- filmid tersebut kemudian digunakan untuk memanggil metode viewModel.getFilmById() guna mendapatkan objek Film yang utuh.

- Setelah objek Film berhasil didapatkan, sebuah log dicatat menggunakan Timber.d("Membuka halaman detail untuk film: ..."), yang memenuhi syarat logging (poin d.c).
- Selanjutnya, data dari objek Film (poster, judul, tahun, plot) ditampilkan ke komponen-komponen View yang sesuai.
- Terakhir, setOnClickListener dipasang pada button_share. Ketika diklik, listener ini akan mencatat log peristiwa dengan Timber.d("Tombol Explicit Intent (Share) ditekan ...") (memenuhi syarat d.b) dan kemudian membuat serta meluncurkan Intent dengan ACTION_SEND untuk berbagi informasi film.

7. MainActivity.kt

File ini adalah komponen Activity utama yang berfungsi sebagai entry point aplikasi.

- Pada baris 8, **class MainActivity : AppCompatActivity()** mendeklarasikan kelas aktivitas utama yang mewarisi fungsionalitas dari AppCompatActivity.
- Pada baris 10, **private lateinit var binding: ActivityMainBinding** mendeklarasikan variabel untuk menampung instance dari kelas binding yang dihasilkan secara otomatis untuk activity_main.xml, memungkinkan akses view yang aman.
- Pada baris 12, **override fun onCreate(...)** adalah fungsi *lifecycle callback* yang dipanggil saat Activity pertama kali dibuat.
- Pada baris 14, **binding = ActivityMainBinding.inflate(layoutInflater)** melakukan *inflate* terhadap layout XML dan menginisialisasi objek binding.
- Pada baris 15, **setContentView(binding.root)** menetapkan root view dari layout yang telah di-inflate sebagai konten utama dari Activity.

8. Layout / activity_main.xml

File layout ini berfungsi sebagai kerangka dasar antarmuka pengguna aplikasi.

- Pada baris 9, **<androidx.fragment.app.FragmentContainerView ...>** adalah sebuah View khusus yang dirancang untuk menampung Fragment. Komponen ini bertindak sebagai NavHost yang akan menampilkan tujuan navigasi.
- Pada baris 19, **app:navGraph="@navigation/nav_graph"** adalah atribut yang menghubungkan FragmentContainerView ini dengan grafik navigasi yang didefinisikan dalam nav_graph.xml, yang mengatur alur perpindahan antar Fragment.

9. Layout / fragment_list.xml

File layout ini mendefinisikan struktur visual untuk ListFragment.

- Pada baris 9, **<TextView ...>** berfungsi sebagai elemen teks statis untuk menampilkan judul pada bagian atas layar.
- Pada baris 21, **<androidx.recyclerview.widget.RecyclerView ...>** adalah komponen utama pada layout ini. RecyclerView digunakan untuk menampilkan daftar data yang besar secara efisien dengan mendaur ulang View untuk setiap item.

10. Layout / fragment_detail.xml

File layout ini mendefinisikan struktur visual untuk DetailFragment.

- Pada baris 2, **<ScrollView ...>** digunakan sebagai View induk untuk memastikan bahwa konten di dalamnya dapat di-scroll jika tingginya melebihi ukuran layar.
- Pada baris 29, **<com.google.android.material.imageview.ShapeableImageView ...>** adalah komponen ImageView dari library Material Design yang mendukung pembentukan sudut, digunakan di sini untuk menampilkan poster film dengan sudut membulat.
- Pada baris 48, **<TextView android:id="@+id/tv_detail_title" ...>** dan baris 85, **<TextView android:id="@+id/tv_detail_plot" ...>** adalah komponen TextView yang masing-masing berfungsi sebagai penampung untuk menampilkan judul dan plot film.

11. Layout / list_item.xml

File layout ini berfungsi sebagai templat untuk setiap item individual di dalam RecyclerView.

- Pada baris 2, **<com.google.android.material.card.MaterialCardView ...>** membungkus setiap item dalam sebuah View berbentuk kartu, yang memberikan elevasi (efek bayangan) dan batas sudut yang jelas, sesuai dengan pedoman Material Design.

- Pada baris 79, `<Button android:id="@+id/btn_imdb" ...>` dan baris 90, `<Button android:id="@+id/btn_detail" ...>` adalah komponen Button yang menyediakan interaksi bagi pengguna pada setiap item, yaitu untuk melihat link IMDB dan untuk menavigasi ke halaman detail.

12. Navigation / nav_graph.xml

File ini mendefinisikan semua alur navigasi antar Fragment di dalam aplikasi menggunakan Navigation Component.

- Pada baris 6, `app:startDestination="@id/listFragment"` menetapkan listFragment sebagai Fragment awal yang akan ditampilkan ketika grafik navigasi ini dimuat.
- Pada baris 14, `<action ...>` mendefinisikan sebuah jalur navigasi yang valid dari listFragment (asal) ke detailFragment (tujuan).
- Pada baris 24, `<argument android:name="filmId" app:argType="integer" />` mendeklarasikan bahwa detailFragment menerima sebuah argumen bernama filmId dengan tipe data integer. Deklarasi ini memungkinkan pengiriman data antar Fragment dengan cara yang aman (type-safe).

13. MyApplication.kt

Ini adalah implementasi dari kelas Application kustom yang berfungsi sebagai titik masuk tunggal untuk inisialisasi level aplikasi.

- Kelas MyApplication merupakan turunan dari kelas `android.app.Application`.
- Di dalam metode `onCreate()`, yang merupakan lifecycle callback pertama saat aplikasi dimulai, dilakukan pengecekan kondisi menggunakan `if (BuildConfig.DEBUG)`. Variabel `BuildConfig.DEBUG` adalah flag yang secara otomatis bernilai `true` selama pengembangan (build variant "debug") dan `false` saat aplikasi dibuat untuk rilis.
- Di dalam blok kondisional tersebut, metode `Timber.plant(Timber.DebugTree())` dipanggil. Perintah ini menginisialisasi Timber agar aktif dan mencetak log ke Logcat, namun hanya pada build "debug". Hal ini memastikan bahwa pada versi rilis, semua panggilan logging tidak akan dieksekusi, sehingga aplikasi menjadi lebih bersih dan aman.

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Adiiaus/Praktikum-PemrogramanMobile.git>

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.

A. Jawaban

Dalam arsitektur aplikasi Android, Application class adalah sebuah kelas dasar (base class) yang esensial dan berfungsi sebagai titik masuk utama serta *global singleton instance* untuk seluruh aplikasi. Sebuah instance dari kelas ini akan dibuat secara otomatis oleh sistem Android ketika proses aplikasi pertama kali dimulai. Instance ini akan tetap ada selama siklus hidup aplikasi (*application lifecycle*) berlangsung, bahkan ketika komponen lain seperti Activity dihancurkan dan dibuat ulang akibat perubahan konfigurasi atau navigasi pengguna.

Fungsi dan Peran Utama Application Class

Application class memiliki beberapa fungsi strategis dalam pengembangan aplikasi Android, yang utamanya berpusat pada manajemen data dan inisialisasi global.

1. **Manajemen State Global (Global State Management)** Fungsi primer dari Application class adalah untuk mengelola state atau data yang bersifat global dan perlu diakses oleh berbagai komponen aplikasi. Hal ini memungkinkan Activity, Service, atau BroadcastReceiver untuk mengakses data bersama tanpa perlu melewatkannya secara eksplisit melalui mekanisme seperti Intent. Dengan demikian, konsistensi data di seluruh aplikasi dapat terjaga. Contoh data global yang lazim disimpan di sini mencakup status autentikasi pengguna, konfigurasi aplikasi, atau instance tunggal dari objek utilitas seperti klien jaringan (misalnya, Retrofit) atau *database helper* (misalnya, Room).
2. **Inisialisasi Tunggal (One-Time Initialization)** Metode onCreate() dari Application class merupakan lokasi yang ideal untuk melakukan proses inisialisasi yang hanya perlu dijalankan satu kali selama siklus hidup aplikasi. Ini sangat efisien untuk menginisialisasi *Software Development Kits* (SDK) pihak ketiga, seperti library untuk analisis (contoh: Firebase Analytics), *dependency injection* (contoh: Hilt, Koin), atau konfigurasi *crash reporting*. Menempatkan logika ini di Application class memastikan bahwa komponen-komponen tersebut siap digunakan sejak awal dan tidak diinisialisasi berulang kali setiap kali Activity dibuat.
3. **Reaksi terhadap Peristiwa Tingkat Aplikasi** Kelas ini dapat digunakan untuk merespons peristiwa tingkat sistem yang memengaruhi seluruh aplikasi. Melalui *override* metode seperti onLowMemory(), aplikasi dapat secara terpusat mengelola sumber daya dengan membersihkan *cache* atau objek yang tidak lagi esensial ketika sistem kehabisan memori. Demikian pula, metode onConfigurationChanged() dapat digunakan untuk menangani perubahan konfigurasi global, seperti perubahan lokal atau bahasa perangkat.

Praktik yang Harus Dihindari

Meskipun sangat berguna, terdapat praktik penggunaan Application class yang harus dihindari. Sangat tidak disarankan untuk menyimpan referensi ke Context dari komponen dengan siklus hidup yang lebih pendek, seperti Activity Context, di dalam Application class. Praktik ini dapat menyebabkan kebocoran memori (*memory leak*). Hal ini terjadi karena Application class yang memiliki siklus hidup panjang akan terus menahan referensi ke Activity, sehingga *Garbage Collector* tidak dapat membersihkan objek Activity tersebut dari memori meskipun sudah tidak lagi ditampilkan kepada pengguna.

Implementasi Konseptual

Untuk memanfaatkan fungsionalitas ini, pengembang perlu membuat sebuah kelas baru yang merupakan turunan (*subclass*) dari `android.app.Application`. Selanjutnya, kelas kustom tersebut harus dideklarasikan dalam file `AndroidManifest.xml` melalui atribut `android:name` pada tag `<application>`. Langkah ini menginstruksikan sistem Android untuk menggunakan kelas kustom tersebut sebagai objek aplikasi utama, bukan kelas Application bawaan.

Kesimpulan

Secara keseluruhan, Application class memegang peranan krusial dalam arsitektur aplikasi Android sebagai wadah terpusat untuk state global dan logika inisialisasi. Penggunaannya yang tepat dapat meningkatkan modularitas, menyederhanakan manajemen data antar komponen, dan memastikan konsistensi fitur di seluruh aplikasi, yang pada akhirnya menghasilkan arsitektur yang lebih solid dan mudah dikelola.

MODUL 5 : Connect to the Internet

SOAL 1

Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
 - b. Gunakan KotlinX Serialization sebagai library JSON.
 - c. Gunakan library seperti Coil atau Glide untuk image loading.
 - d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:
<https://developer.themoviedb.org/docs/getting-started>
 - e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
 - f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
 - g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

Data / Local / AppDatabase

```
1 package com.example.modul5.data.local
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7 import com.example.modul5.data.model.Movie
8
9 @Database(entities = [Movie::class], version = 2, exportSchema
10 = false)
11 abstract class AppDatabase : RoomDatabase() {
12     abstract fun movieDao(): MovieDao
13
14     companion object {
15         @Volatile
16         private var INSTANCE: AppDatabase? = null
17
18         fun getDatabase(context: Context): AppDatabase {
19             return INSTANCE ?: synchronized(this) {
20                 val instance = Room.databaseBuilder(
21                     context.applicationContext,
22                     AppDatabase::class.java,
```

23	"movie_database"
24	).fallbackToDestructiveMigration()
25	.build()
26	INSTANCE = instance
27	instance
28	
29	}
30	}
31	}
32	}

Table 52 Source Code Soal 1

Data / Local / MovieDao

1	package com.example.modul5.data.local
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import androidx.room.Update
8	import com.example.modul5.data.model.Movie
9	import kotlinx.coroutines.flow.Flow
	@Dao
	interface MovieDao {
	@Query("SELECT * FROM movies")
	fun getAllMovies(): Flow<List<Movie>>
	@Query("SELECT * FROM movies WHERE isBookmarked = 1")
	fun getBookmarkedMovies(): Flow<List<Movie>>
	@Insert(onConflict = OnConflictStrategy.REPLACE)
	suspend fun insertAll(movies: List<Movie>)
	@Update
	suspend fun updateMovie(movie: Movie)
	@Query("DELETE FROM movies")
	suspend fun clearAll()
	}

Table 53 Source Code Soal 1

Data / Model / Movie

1	package com.example.modul5.data.model
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	import kotlinx.serialization.SerialName
6	import kotlinx.serialization.Serializable
7	
8	@Serializable
9	@Entity(tableName = "movies")
	data class Movie(
	@PrimaryKey

	<pre> @SerializedName("id") val id: Int, @SerializedName("name") val title: String, @SerializedName("first_air_date") val year: String, @SerializedName("overview") val plot: String, @SerializedName("poster_path") val posterPath: String?, val isBookmarked: Boolean = false,) @Serializable data class MovieApiResponse(val results: List<Movie>) </pre>
--	---

Table 54 Source Code Soal 1

Data / remote / ApiService

1	package com.example.modul5.data.remote
2	
3	import com.example.modul5.data.model.MovieApiResponse
4	import retrofit2.http.GET
5	import retrofit2.http.Query
6	
7	interface ApiService {
8	@GET("discover/tv")
9	suspend fun getPopularMovies(
	@Query("api_key") apiKey: String,
	@Query("with_original_language") origin_language:
	String = "ko",
	@Query("sort_by") sortBy: String = "popularity.desc",
	@Query("with_watch_providers") providers: String = "8",
	@Query("watch_region") region: String = "ID",
	): MovieApiResponse
	}

Table 55 Source Code Soal 1

Data / remote / RetrofitClient

1	package	com.example.modul5.data.remote
2		
3	import	
4	com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverter	
5	Factory	
6	import	kotlinx.serialization.json.Json
7	import	okhttp3.MediaType.Companion.toMediaType
8	import	retrofit2.Retrofit
9	object	RetrofitClient {
	private const val	BASE_URL = "https://api.themoviedb.org/3/"
	private val	json = Json { ignoreUnknownKeys = true }
	val	instance: ApiService by lazy {
	val	retrofit = Retrofit.Builder()
		.baseUrl(BASE_URL)
		.addConverterFactory(json.asConverterFactory("application/json".toMed
		iaType()))
		.build()
		retrofit.create(ApiService::class.java)
	}	
	}	

Table 56 Source Code Soal 1

Data / repository / MovieRepository

1	package	com.example.modul5.data.repository
2		
3	import	com.example.modul5.BuildConfig
4	import	com.example.modul5.data.local.MovieDao
5	import	com.example.modul5.data.model.Movie
6	import	com.example.modul5.data.remote.ApiService
7	import	kotlinx.coroutines.Dispatchers
8	import	kotlinx.coroutines.flow.first
9	import	kotlinx.coroutines.withContext
	class	MovieRepository(
	private val	apiService: ApiService,
	private val	movieDao: MovieDao
	)	{
	val	movies = movieDao.getAllMovies()
	val	bookmarkedMovies = movieDao.getBookmarkedMovies()
	suspend fun	updateMovie(movie: Movie) {
		movieDao.updateMovie(movie)
	}	
	suspend fun	refreshMovies() {
	withContext(Dispatchers.IO)	{
	try	{
		val oldMovies = movieDao.getAllMovies().first()
	val	bookmarkedIds = oldMovies.filter {

	<pre> it.isBookmarked }.map { it.id }.toSet() val newMoviesFromApi = apiService.getPopularMovies(BuildConfig.TMDB_API_KEY).results val newMovies = newMoviesFromApi.map { apiMovie -> apiMovie.copy(isBookmarked = bookmarkedIds.contains(apiMovie.id)) } movieDao.insertAll(newMovies) } catch (e: Exception) { e.printStackTrace() } } } } </pre>
--	--

Table 57 Source Code Soal 1

Data / navigation / AppNavigation

1	package	com.example.modul5.navigation
2		
3	import	androidx.compose.runtime.Composable
4	import	androidx.lifecycle.viewmodel.compose.viewModel
5	import	androidx.navigation.compose.NavHost
6	import	androidx.navigation.compose.composable
7	import	androidx.navigation.compose.rememberNavController
8	import	com.example.modul5.ui.theme.DetailScreen
9	import	com.example.modul5.ui.theme.MovieListScreen
	import	com.example.modul5.viewmodel.MovieViewModel
	import	com.example.modul5.viewmodel.SharedMovieViewModel
	@Composable	
	fun	AppNavigation(
	movieViewModel:	MovieViewModel,
	sharedViewModel:	SharedMovieViewModel,
	)	{
	val	navController = rememberNavController()
	NavHost(navController = navController, startDestination =	
	Screen.MovieList.route)	{
	composable(Screen.MovieList.route)	{
	MovieListScreen(	
	navController	= navController,
	movieViewModel	= movieViewModel,
	sharedViewModel	= sharedViewModel
	)	
	}	
	composable(Screen.Detail.route)	{
	DetailScreen(sharedViewModel	= sharedViewModel)
	}	
	}	
	}	

Table 58 Source Code Soal 1

Data / navigation / Screen

1	package	com.example.modul5.navigation
2		
3	sealed class Screen (val route: String) {	
4	object MovieList : Screen("movie_list")	
5	object Detail : Screen("movie_detail")	
6	}	

Table 59 Source Code Soal 1

Ui.theme / BookmarkScreen

1	package	com.example.modul5.ui.theme
2		
3	import	androidx.compose.foundation.layout.Box
4	import	androidx.compose.foundation.layout.fillMaxSize
5	import	androidx.compose.foundation.layout.padding
6	import	androidx.compose.foundation.lazy.LazyColumn
7	import	androidx.compose.foundation.lazy.items
8	import	androidx.compose.material3.ExperimentalMaterial3Api
9	import	androidx.compose.material3.MaterialTheme
	import	androidx.compose.material3.Scaffold
	import	androidx.compose.material3.Text
	import	androidx.compose.material3.TopAppBar
	import	androidx.compose.runtime.Composable
	import	androidx.compose.runtime.collectAsState
	import	androidx.compose.runtime.getValue
	import	androidx.compose.ui.Alignment
	import	androidx.compose.ui.Modifier
	import	androidx.compose.ui.text.style.TextAlign
	import	androidx.navigation.NavController
	import	com.example.modul5.viewmodel.MovieViewModel
	import	com.example.modul5.viewmodel.SharedMovieViewModel
	@OptIn(ExperimentalMaterial3Api::class)	
	@Composable	
	fun	BookmarkScreen (
	navController:	NavController,
	movieViewModel:	MovieViewModel,
	sharedViewModel:	SharedMovieViewModel
	)	{
	val	bookmarkedList by
	movieViewModel.bookmarkedMovies.collectAsState()	
	Scaffold(	
	topBar	= {
	TopAppBar(title = { Text("Daftar Bookmark") })	}
	}	
	)	{
	if (bookmarkedList.isEmpty())	->
	Box(	{
	modifier =	Modifier
	.fillMaxSize()	
	.padding(innerPadding),	

	<pre> contentAlignment = Alignment.Center) Text(text = "Belum ada serial yang di-bookmark.", style = MaterialTheme.typography.bodyLarge, textAlign = TextAlign.Center) } } else { LazyColumn(modifier = Modifier .fillMaxSize() .padding(innerPadding)) { items(bookmarkedList) { movie -> MovieListItem(movie = movie, movieViewModel = movieViewModel, sharedViewModel = sharedViewModel, navController = navController) } } } } } } </pre>
--	---

Table 60 Source Code Soal 1

Ui.theme / DetailScreen

1	package com.example.modul5.ui.theme
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.lazy.LazyColumn
5	import androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Modifier
9	import androidx.compose.ui.draw.clip
	import androidx.compose.ui.layout.ContentScale
	import androidx.compose.ui.unit.dp
	import coil.compose.AsyncImage
	import com.example.modul5.viewmodel.SharedMovieViewModel
	 @OptIn(ExperimentalMaterial3Api::class)
	@Composable
	fun DetailScreen(sharedViewModel: SharedMovieViewModel) {
	val selectedMovie = sharedViewModel.selectedMovie ?: return
	 Scaffold(
	topBar = {
	TopAppBar(
	title = { Text("Movie Detail") }
	)
	)

	<pre>) { paddingValues -> LazyColumn(modifier = Modifier .fillMaxSize() .padding(paddingValues) .padding(16.dp)) { item { AsyncImage(model = "https://image.tmdb.org/t/p/w500\${selectedMovie.posterPath}", contentDescription = selectedMovie.title, modifier = Modifier .fillMaxWidth() .height(500.dp) .clip(RoundedCornerShape(16.dp)), contentScale = ContentScale.Crop) Spacer(modifier = Modifier.height(16.dp)) } item { Text(text = "\${selectedMovie.title} (\${selectedMovie.year.substring(0, 4)})", style = MaterialTheme.typography.headlineMedium) Spacer(modifier = Modifier.height(16.dp)) } item { Text(text = "Overview", style = MaterialTheme.typography.titleMedium) Spacer(modifier = Modifier.height(8.dp)) Text(text = selectedMovie.plot, style = MaterialTheme.typography.bodyLarge) } } } </pre>
--	--

Table 61 Source Code Soal 1

Ui.theme / MainScreen

```
1 package com.example.modul5.ui.theme
2
3 import androidx.compose.foundation.layout.padding
4 import androidx.compose.material.icons.Icons
5 import androidx.compose.material.icons.filled.Bookmark
6 import androidx.compose.material.icons.filled.Home
7 import androidx.compose.material3.*
8 import androidx.compose.runtime.Composable
9 import androidx.compose.runtime.getValue
10 import androidx.compose.ui.Modifier
11 import androidx.navigation.NavDestination.Companion.hierarchy
12 import androidx.navigation.NavGraph.Companion.findStartDestination
13 import androidx.navigation.compose.NavHost
14 import androidx.navigation.compose.composable
15 import androidx.navigation.compose.currentBackStackEntryAsState
16 import androidx.navigation.compose.rememberNavController
17 import com.example.modul5.navigation.Screen
18 import com.example.modul5.viewmodel.MovieViewModel
19 import com.example.modul5.viewmodel.SharedMovieViewModel
20
21 @OptIn(ExperimentalMaterial3Api::class)
22 @Composable
23 fun MainScreen(movieViewModel: MovieViewModel, sharedViewModel:
24 SharedMovieViewModel) {
25     val navController = rememberNavController()
26
27     Scaffold(
28         bottomBar = {
29             NavigationBar {
30                 val navBackStackEntry by
31 navController.currentBackStackEntryAsState()
32                 val currentDestination =
33 navBackStackEntry?.destination
34
35                 // Item untuk Home
36                 NavigationBarItem(
37                     icon = { Icon(Icons.Default.Home,
38 contentDescription = "Home") },
39                     label = { Text("Home") },
40                     selected =
41 currentDestination?.hierarchy?.any { it.route ==
42 Screen.MovieList.route } == true,
43                     onClick = {
44 navController.navigate(Screen.MovieList.route)
45
46 popUpTo(navController.graph.findStartDestination().id)
47 saveState = true
48                     launchSingleTop = true
49                     restoreState = true
50                 }
51             }
52         }
53     )
54 }
```

	<pre>) // Item untuk Bookmark NavigationBarItem(icon = { Icon(Icons.Default.Bookmark, contentDescription = "Bookmark") }, label = { Text("Bookmarks") }, selected = currentDestination?.hierarchy?.any { it.route == "bookmark_screen" } == true, onClick = { navController.navigate("bookmark_screen") popUpTo(navController.graph.findStartDestination().id) { saveState = true launchSingleTop = true restoreState = true } }) }) { innerPadding -> NavHost(navController = navController, startDestination = Screen.MovieList.route, modifier = Modifier.padding(innerPadding)) { composable(Screen.MovieList.route) { MovieListScreen(navController = navController, movieViewModel = movieViewModel, sharedViewModel = sharedViewModel) } composable("bookmark_screen") { BookmarkScreen(navController = navController, movieViewModel = movieViewModel, sharedViewModel = sharedViewModel) } composable(Screen.Detail.route) { DetailScreen(sharedViewModel = sharedViewModel) } } } } </pre>
--	---

Table 62 Source Code Soal 1

Ui.theme / MovieListItem

```
1 package com.example.modul5.ui.theme
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.foundation.layout.*
6 import androidx.compose.foundation.shape.RoundedCornerShape
7 import androidx.compose.material.icons.Icons
8 import androidx.compose.material.icons.filled.Bookmark
9 import androidx.compose.material.icons.outlined.BookmarkBorder
10 import androidx.compose.material3.*
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.draw.clip
15 import androidx.compose.ui.layout.ContentScale
16 import androidx.compose.ui.platform.LocalContext
17 import androidx.compose.ui.text.font.FontWeight
18 import androidx.compose.ui.text.style.TextOverflow
19 import androidx.compose.ui.unit.dp
20 import androidx.navigation.NavController
21 import coil.compose.AsyncImage
22 import com.example.modul5.data.model.Movie
23 import com.example.modul5.navigation.Screen
24 import com.example.modul5.viewmodel.MovieViewModel
25 import com.example.modul5.viewmodel.SharedMovieViewModel
26
27 @OptIn(ExperimentalMaterial3Api::class)
28 @Composable
29 fun MovieListItem(
30     movie: Movie,
31     movieViewModel: MovieViewModel,
32     sharedViewModel: SharedMovieViewModel,
33     navController: NavController
34 ) {
35     val context = LocalContext.current
36
37     Card(
38         modifier = Modifier
39             .fillMaxWidth()
40             .padding(vertical = 8.dp, horizontal = 16.dp),
41         elevation = CardDefaults.cardElevation(4.dp)
42     ) {
43         Row(modifier = Modifier.padding(16.dp)) {
44             AsyncImage(
45                 model =
46                 "https://image.tmdb.org/t/p/w500${movie.posterPath}",
47                 contentDescription = movie.title,
48                 modifier = Modifier
49                     .size(width = 100.dp, height = 150.dp)
50                     .clip(RoundedCornerShape(8.dp)),
51                 contentScale = ContentScale.Crop
52             )
53
54             Spacer(modifier = Modifier.width(16.dp))
55         }
56     }
57 }
```



```

        Column(modifier      =      Modifier.weight(1f))      {
            Row(
                modifier      =      Modifier.fillMaxWidth(),
                verticalAlignment      =
Alignment.CenterVertically,
                horizontalArrangement      =
Arrangement.SpaceBetween
            )
            Text(
                text      =      movie.title,
                style      =
MaterialTheme.typography.titleMedium,
                fontWeight      =      FontWeight.Bold,
                modifier      =      Modifier.weight(1f),
                maxLines      =      2,
                overflow      =      TextOverflow.Ellipsis
            )
            IconButton(onClick      =      {
movieViewModel.toggleBookmark(movie)      })      {
                Icon(
                    imageVector      =      if
(movie.isBookmarked)      Icons.Filled.Bookmark      else
Icons.Outlined.BookmarkBorder,
                    contentDescription      =      "Bookmark",
                    tint      =
MaterialTheme.colorScheme.primary
                )
            }
        }

        //      Tahun      Rilis
        Text(
            text = movie.year?.substring(0, 4) ?: "N/A",
            style = MaterialTheme.typography.bodySmall
        )

        Spacer(modifier      =      Modifier.height(8.dp))

        //      Plot
        Text(
            text      =      movie.plot      ?:      "No      overview
available.",
            style = MaterialTheme.typography.bodySmall,
            maxLines      =      3,
            overflow      =      TextOverflow.Ellipsis
        )

        Spacer(modifier      =      Modifier.weight(1f))

        Row(
            modifier      =      Modifier.fillMaxWidth(),
            horizontalArrangement      =      Arrangement.End
        )      {
            Button(onClick      =      {
                val query = Uri.encode(movie.title)

```


	<pre> modifier = Modifier.fillMaxSize().padding(paddingValues), contentAlignment = Alignment.Center) CircularProgressIndicator() } } else { LazyColumn(modifier = Modifier.padding(paddingValues)) { items(movieList) { movie -> MovieListItem(movie = movie, movieViewModel = movieViewModel, sharedViewModel = sharedViewModel, navController = navController) } } } } } </pre>
--	---

Table 64 Source Code Soal 1

Viewmodel / MovieViewModel

1	package com.example.modul5.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.modul5.data.model.Movie
6	import com.example.modul5.data.repository.MovieRepository
7	import kotlinx.coroutines.flow.SharingStarted
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.stateIn
	import kotlinx.coroutines.launch
	<pre> class MovieViewModel(private val repository: MovieRepository) : ViewModel() { val movies: StateFlow<List<Movie>> = repository.movies .stateIn(scope = viewModelScope, started = SharingStarted.WhileSubscribed(5000L), initialValue = emptyList()) val bookmarkedMovies: StateFlow<List<Movie>> = repository.bookmarkedMovies .stateIn(scope = viewModelScope, started = SharingStarted.WhileSubscribed(5000L), initialValue = emptyList()) init </pre>

	<pre> viewModelScope.launch { repository.refreshMovies() } fun toggleBookmark(movie: Movie) { viewModelScope.launch { val updatedMovie = movie.copy(isBookmarked = !movie.isBookmarked) repository.updateMovie(updatedMovie) } } </pre>
--	--

Table 65 Source Code Soal 1

Viewmodel / MovieViewModelFactory

1	package com.example.modul5.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.modul5.data.repository.MovieRepository
6	
7	class MovieViewModelFactory(private val repository:
8	MovieRepository) : ViewModelProvider.Factory {
9	@Suppress("UNCHECKED_CAST")
	override fun <T : ViewModel> create(modelClass: Class<T>):
	T {
	if
	(modelClass.isAssignableFrom(MovieViewModel::class.java)) {
	return MovieViewModel(repository) as T
	}
	throw IllegalArgumentException("Unknown ViewModel
	class")
	}
	}

Table 66 Source Code Soal 1

Viewmodel / SharedViewModel

1	package com.example.modul5.viewmodel
2	
3	import androidx.compose.runtime.getValue
4	import androidx.compose.runtime.mutableStateOf
5	import androidx.compose.runtime.setValue
6	import androidx.lifecycle.ViewModel
7	import com.example.modul5.data.model.Movie
8	
9	class SharedMovieViewModel : ViewModel() {
	var selectedMovie by mutableStateOf<Movie?>(null)
	private set
	fun selectMovie(movie: Movie) {
	selectedMovie = movie

	}
	}

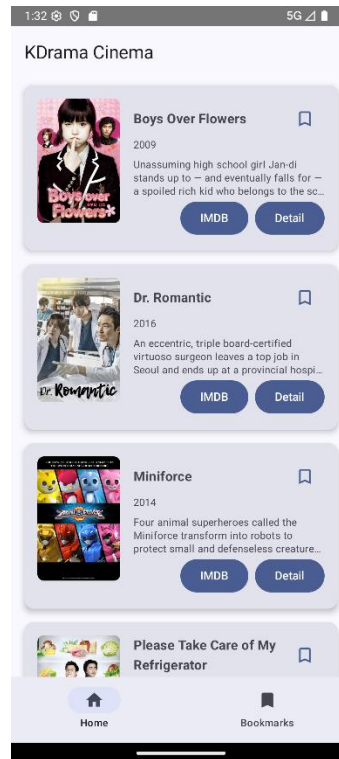
Table 67 Source Code Soal 1

MainActivity.kt

1	package	com.example.modul5
2		
3	import	android.os.Bundle
4	import	androidx.activity.ComponentActivity
5	import	androidx.activity.compose.setContent
6	import	androidx.lifecycle.ViewModelProvider
7	import	com.example.modul5.data.local.AppDatabase
8	import	com.example.modul5.data.remote.RetrofitClient
9	import	com.example.modul5.data.repository.MovieRepository
	import	com.example.modul5.ui.theme.MainScreen
	import	com.example.modul5.ui.theme.Modul5Theme
	import	com.example.modul5.viewmodel.MovieViewModel
	import	com.example.modul5.viewmodel.MovieViewModelFactory
	import	com.example.modul5.viewmodel.SharedMovieViewModel
	class MainActivity	: ComponentActivity() {
	override fun onCreate(savedInstanceState: Bundle?)	{
	super.onCreate(savedInstanceState)	
	val database	=
	AppDatabase.getDatabase(applicationContext)	
	val apiService	= RetrofitClient.instance
	val repository	= MovieRepository(apiService,
	database.movieDao())	
	val factory	= MovieViewModelFactory(repository)
	val movieViewModel: MovieViewModel	=
	ViewModelProvider(this, factory) [MovieViewModel::class.java]	
	val sharedViewModel: SharedMovieViewModel	=
	ViewModelProvider(this) [SharedMovieViewModel::class.java]	
	setContent	{
	Modul5Theme	{
	MainScreen(	
	movieViewModel	= movieViewModel,
	sharedViewModel	= sharedViewModel
	)	
	}	
	}	
	}	
	}	

Table 68 Source Code Soal 1

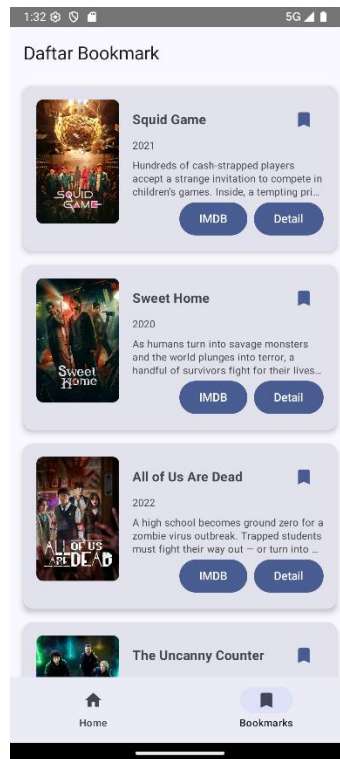
B. Output Program



Gambar 32 Screenshot Hasil Jawaban Soal 1



Gambar 33 Screenshot Hasil Jawaban Soal 1



Gambar 34 Screenshot Hasil Jawaban Soal 1

C. Pembahasan

1. Lapisan Data (Data Layer)

Lapisan ini bertanggung jawab atas penyediaan dan manajemen data aplikasi dari berbagai sumber.

1.1. Model Data (data/model)

- Berkas: Movie.kt (Definisi Entitas Film)

Berkas ini mendefinisikan struktur data untuk sebuah film, yang berfungsi sebagai model untuk data dari API dan juga sebagai skema tabel untuk database lokal.

- **Baris 5: `package com.example.modul5.data.model`** Mendeklarasikan bahwa berkas ini merupakan bagian dari paket `com.example.modul5.data.model`.
- **Baris 7-10: `import ...`** Mengimpor kelas-kelas yang diperlukan, seperti `Entity` dan `PrimaryKey` dari `Room`, serta `SerializedName` dan `Serializable` dari `KotlinX Serialization`.
- **Baris 12: `@Serializable`** Anotasi ini menandakan bahwa kelas `Movie` dapat diubah (diserialisasi) menjadi format lain seperti JSON dan sebaliknya (deserialisasi). Hal ini krusial untuk memproses respons dari API.
- **Baris 13: `@Entity(tableName = "movies")`** Anotasi dari pustaka `Room` yang menginstruksikan bahwa kelas `Movie` ini merepresentasikan sebuah tabel dalam database dengan nama `movies`.
- **Baris 14: `data class Movie(...)`** Mendeklarasikan `Movie` sebagai sebuah data class. Ini adalah cara ringkas di Kotlin untuk membuat kelas yang tujuan utamanya adalah menyimpan data.
- **Baris 15: `@PrimaryKey`** Menetapkan properti `id` yang didefinisikan pada baris berikutnya sebagai kunci utama (*primary key*) dari tabel `movies`.
- **Baris 16: `val id: Int`** Mendefinisikan properti `id` dengan tipe data `Int` untuk menyimpan identifikasi unik setiap film.
- **Baris 18: `@SerializedName("poster_path")`** Memetakan atribut `poster_path` dalam data JSON dari API ke properti `posterPath` di dalam kelas ini.
- **Baris 19: `val posterPath: String`** Mendefinisikan properti `posterPath` dengan tipe `String` untuk menyimpan path ke gambar poster film.
- **Baris 21: `val title: String`** Mendefinisikan properti untuk menyimpan judul film.
- **Baris 23: `val overview: String`** Mendefinisikan properti untuk menyimpan sinopsis atau deskripsi film.
- **Baris 25: `var isBookmarked: Boolean = false`** Mendefinisikan properti `isBookmarked` dengan nilai awal `false`. Properti ini digunakan untuk melacak apakah sebuah film telah ditandai oleh pengguna.

1.2. Sumber Data Jarak Jauh (data/remote)

- Berkas: ApiService.kt (Definisi Layanan API)

Berkas ini mendefinisikan metode-metode untuk berinteraksi dengan endpoint dari The Movie Database (TMDb) API.

- **Baris 5: `package com.example.modul5.data.remote`** Mendeklarasikan keanggotaan berkas dalam paket `data.remote`.
- **Baris 7-9: `import ...`** Mengimpor kelas `MovieApiResponse` sebagai tipe data kembalian dan anotasi `GET` serta `Query` dari pustaka `Retrofit`.
- **Baris 11: `interface ApiService`** Mendeklarasikan `ApiService` sebagai sebuah *interface*. `Retrofit` akan menggunakan *interface* ini untuk menghasilkan implementasi kode pemanggil jaringan.
- **Baris 14: `@GET("movie/now_playing")`** Anotasi `Retrofit` yang menandakan bahwa metode `getNowPlayingMovies` akan melakukan *request* HTTP `GET` ke *endpoint* `movie/now_playing` relatif terhadap URL dasar.

- **Baris 15: suspend fun getNowPlayingMovies(...)** Mendefinisikan fungsi `getNowPlayingMovies` sebagai `suspend`, yang berarti fungsi ini dapat dijalankan dalam sebuah *coroutine* tanpa memblokir *thread*.
- **Baris 16: @Query("api_key") apiKey: String = ...** Anotasi Retrofit yang akan menambahkan parameter kueri `api_key` ke dalam URL *request*. Nilai *default* untuk kunci API juga ditetapkan di sini.
- **Baris 17:): MovieApiResponse** Menetapkan bahwa fungsi ini akan mengembalikan sebuah objek `MovieApiResponse` yang telah diproses dari respons JSON.
- Berkas: `RetrofitClient.kt` (Konfigurasi Klien HTTP)

Berkas ini bertanggung jawab untuk membuat dan mengonfigurasi satu instance (singleton) dari Retrofit.

- **Baris 11: object RetrofitClient** Mendeklarasikan `RetrofitClient` sebagai sebuah object. Ini adalah cara Kotlin untuk mengimplementasikan pola desain *singleton*, memastikan hanya ada satu *instance* dari klien Retrofit di seluruh aplikasi.
- **Baris 13: private const val BASE_URL = ...** Mendefinisikan URL dasar dari API sebagai sebuah konstanta privat.
- **Baris 17: private val json = Json { ignoreUnknownKeys = true }** Menginisialisasi sebuah *instance* `Json` dari pustaka `KotlinX Serialization`. Konfigurasi `ignoreUnknownKeys = true` mencegah aplikasi mengalami *crash* jika API mengembalikan atribut JSON yang tidak didefinisikan dalam data class.
- **Baris 20: private val retrofit: Retrofit by lazy { ... }** Mendeklarasikan *instance* Retrofit menggunakan delegasi *by lazy*. Ini berarti objek Retrofit hanya akan dibuat pada saat pertama kali diakses, bukan saat aplikasi dimulai, sehingga meningkatkan efisiensi.
- **Baris 22: .baseUrl(BASE_URL)** Mengatur URL dasar untuk semua *request* yang dibuat oleh *instance* Retrofit ini.
- **Baris 23: .addConverterFactory(...)** Menambahkan sebuah `ConverterFactory`. Dalam kasus ini, `kotlinx.serialization` digunakan untuk secara otomatis mengonversi respons JSON dari API menjadi objek-objek Kotlin.
- **Baris 24: .build()** Membangun *instance* Retrofit dengan konfigurasi yang telah ditetapkan.
- **Baris 27: val instance: ApiService by lazy { ... }** Mengekspos *instance* dari `ApiService`. Sama seperti Retrofit, ini menggunakan *lazy delegation*.
- **Baris 28: retrofit.create(ApiService::class.java)** Membuat implementasi konkret dari *interface* `ApiService` menggunakan *instance* Retrofit yang telah dikonfigurasi.

1.3. Sumber Data Lokal (data/local)

- Berkas: `MovieDao.kt` (Objek Akses Data Film)

Interface ini mendefinisikan semua operasi database (CRUD - Create, Read, Update, Delete) untuk entitas `Movie`.

- **Baris 11: @Dao** Anotasi Room yang mengidentifikasi *interface* ini sebagai *Data Access Object*.
- **Baris 12: interface MovieDao** Deklarasi *interface*.
- **Baris 13: @Insert(onConflict = OnConflictStrategy.REPLACE)** Anotasi untuk operasi penyisipan. Strategi konflik `REPLACE` berarti jika film dengan id yang sama sudah ada, entri lama akan digantikan oleh yang baru.
- **Baris 14: suspend fun insertMovies(movies: List<Movie>)** Fungsi untuk menyisipkan daftar film ke dalam database.
- **Baris 16: @Query("SELECT * FROM movies")** Anotasi yang berisi kueri SQL untuk mengambil semua kolom dari tabel `movies`.

- **Baris 17: fun getAllMovies(): Flow<List<Movie>>** Fungsi yang mengembalikan data sebagai Flow. Ini memungkinkan UI untuk bereaksi secara otomatis terhadap perubahan data di database.
- **Baris 19: @Query("SELECT * FROM movies WHERE isBookmarked = 1")** Kueri SQL untuk mengambil hanya film-film yang kolom isBookmarked-nya bernilai true (atau 1 dalam terminologi SQL).
- **Baris 20: fun getBookmarkedMovies(): Flow<List<Movie>>** Fungsi yang mengembalikan daftar film yang di-bookmark sebagai Flow.
- **Baris 22: @Update** Anotasi untuk operasi pembaruan data pada entitas yang ada.
- **Baris 23: suspend fun updateMovie(movie: Movie)** Fungsi untuk memperbarui satu entitas Movie di dalam database.
- **Berkas: AppDatabase.kt (Definisi Database Aplikasi)**

Kelas abstrak ini merepresentasikan database Room aplikasi dan merupakan titik akses utama ke data yang disimpan secara lokal.

- **Baris 10: @Database(entities = [Movie::class], version = 1, exportSchema = false)** Anotasi utama untuk mengonfigurasi database. entities mendaftarkan semua kelas entitas (tabel). version digunakan untuk manajemen migrasi. exportSchema dinonaktifkan untuk proyek sederhana ini.
- **Baris 11: abstract class AppDatabase : RoomDatabase()** Kelas ini harus abstrak dan mewarisi RoomDatabase.
- **Baris 13: abstract fun movieDao(): MovieDao** Sebuah metode abstrak yang akan diimplementasikan oleh Room untuk menyediakan akses ke MovieDao.
- **Baris 15: companion object { ... }** Blok ini digunakan untuk mendefinisikan metode dan properti statis, khususnya untuk mengimplementasikan pola *singleton*.
- **Baris 17: @Volatile private var INSTANCE: AppDatabase? = null** Mendeklarasikan variabel INSTANCE yang akan menampung satu-satunya *instance* dari AppDatabase. Anotasi @Volatile memastikan bahwa perubahan pada variabel ini segera terlihat oleh semua *thread* lain.
- **Baris 19: fun getDatabase(context: Context): AppDatabase** Metode statis untuk mendapatkan *instance* database.
- **Baris 21: return INSTANCE ?: synchronized(this) { ... }** Menggunakan *Elvis operator* (?:) untuk memeriksa apakah INSTANCE sudah ada. Jika belum (null), blok synchronized akan dieksekusi untuk membuat *instance* baru secara aman dari *thread* (*thread-safe*).
- **Baris 22: val instance = Room.databaseBuilder(...)** Memulai proses pembangunan database menggunakan Room.databaseBuilder.
- **Baris 26: .build()** Menyelesaikan pembuatan *instance* database.

1.4. Repositori (data/repository)

- **Berkas: MovieRepository.kt (Manajer Sumber Data)**

Berkas ini berfungsi sebagai perantara yang mengelola operasi data antara ViewModel dengan sumber data, baik dari jaringan (API) maupun dari database lokal (Room).

- **Baris 7: class MovieRepository(...)** Mendeklarasikan kelas MovieRepository. Konstruktor kelas ini menerima apiService dan movieDao sebagai dependensi, menerapkan prinsip *Dependency Injection*.
- **Baris 8: private val apiService: ApiService** Deklarasi dependensi untuk ApiService, yang digunakan untuk melakukan panggilan ke API.
- **Baris 9: private val movieDao: MovieDao** Deklarasi dependensi untuk MovieDao, yang digunakan untuk berinteraksi dengan database lokal.
- **Baris 12: val allMovies = movieDao.getAllMovies()** Mengekspos sebuah Flow dari MovieDao yang berisi daftar semua film. ViewModel akan mengobservasi Flow ini untuk mendapatkan pembaruan data secara *real-time*.

- **Baris 14: `val bookmarkedMovies = movieDao.getBookmarkedMovies()`** Mengekspos Flow yang berisi daftar film yang telah ditandai (*bookmarked*).
- **Baris 17: `suspend fun updateMovie(movie: Movie)`** Mendefinisikan fungsi suspend untuk memperbarui entitas film di database. Ini biasanya dipanggil ketika status `isBookmarked` diubah.
- **Baris 18: `movieDao.updateMovie(movie)`** Memanggil fungsi `updateMovie` dari `MovieDao` untuk mengeksekusi operasi pembaruan di database.
- **Baris 21: `suspend fun refreshMovies()`** Mendefinisikan fungsi untuk mengambil data film terbaru dari API.
- **Baris 22: `try { ... } catch (e: Exception) { ... }`** Menggunakan blok try-catch untuk menangani kemungkinan Exception selama proses pemanggilan jaringan, misalnya ketika tidak ada koneksi internet.
- **Baris 24: `val response = apiService.getNowPlayingMovies()`** Memanggil fungsi di `ApiService` untuk mendapatkan daftar film yang sedang tayang dari server.
- **Baris 26: `movieDao.insertMovies(response.results)`** Setelah berhasil mendapatkan data dari API, data tersebut (`response.results`) dimasukkan ke dalam database lokal melalui `MovieDao`.

2. Lapisan ViewModel

Lapisan ini menghubungkan lapisan data dengan antarmuka pengguna, menangani logika bisnis dan manajemen status UI.

- Berkas: `MovieViewModel.kt` (Logika UI Utama)

Kelas ini bertanggung jawab untuk menyimpan dan mengelola data yang terkait dengan UI serta menangani logika bisnis.

- **Baris 11: `class MovieViewModel(private val repository: MovieRepository) : ViewModel()`** Mendeklarasikan kelas `MovieViewModel` yang mewarisi kelas `ViewModel` dari Android Jetpack. Kelas ini menerima `MovieRepository` sebagai dependensinya.
- **Baris 14: `val movies: Flow<List<Movie>> = repository.allMovies`** Mendeklarasikan properti publik `movies` yang mendapatkan datanya dari `repository.allMovies`. UI akan mengobservasi Flow ini.
- **Baris 16: `val bookmarkedMovies: Flow<List<Movie>> = repository.bookmarkedMovies`** Mendeklarasikan properti publik `bookmarkedMovies` yang mendapatkan datanya dari `repository.bookmarkedMovies`.
- **Baris 18: `init { ... }`** Blok `init` dieksekusi secara otomatis ketika sebuah instance `MovieViewModel` dibuat untuk pertama kalinya.
- **Baris 21: `viewModelScope.launch { ... }`** Meluncurkan sebuah *coroutine* baru dalam `viewModelScope`. *Coroutine* yang diluncurkan dalam *scope* ini secara otomatis akan dibatalkan jika `ViewModel` dihancurkan, sehingga mencegah kebocoran memori (*memory leak*).
- **Baris 22: `repository.refreshMovies()`** Memanggil fungsi `refreshMovies` dari `repository` untuk memastikan data termuat dari API saat aplikasi dimulai.
- **Baris 26: `fun toggleBookmark(movie: Movie)`** Mendefinisikan fungsi publik yang dipanggil oleh UI ketika pengguna menekan tombol *bookmark*.
- **Baris 27: `viewModelScope.launch { ... }`** Meluncurkan *coroutine* untuk menjalankan operasi database di *background thread*.
- **Baris 29: `val updatedMovie = movie.copy(isBookmarked = !movie.isBookmarked)`** Membuat salinan objek `movie` dengan nilai `isBookmarked` yang dibalik (dari `true` menjadi `false` atau sebaliknya). Ini adalah praktik yang baik untuk menjaga *immutability*.

- **Baris 31: repository.updateMovie(updatedMovie)** Memanggil repository untuk memperbarui film dengan data yang telah diubah di dalam database.
- **Berkas: SharedViewModel.kt (ViewModel untuk Data Bersama)**
 ViewModel ini dirancang untuk memfasilitasi komunikasi dan berbagi data antar Composable yang tidak memiliki hubungan induk-anak secara langsung.
 - **Baris 11: class SharedViewModel : ViewModel()** Deklarasi kelas SharedViewModel yang mewarisi ViewModel.
 - **Baris 14: var selectedMovie by mutableStateOf<Movie?>(null)** Mendeklarasikan properti selectedMovie menggunakan mutableStateOf. Ini berarti setiap perubahan pada nilai selectedMovie akan secara otomatis memicu rekomposisi pada *Composable* mana pun yang membaca nilai ini. Nilai awalnya adalah null.
 - **Baris 15: private set** Menjadikan *setter* dari properti selectedMovie privat. Ini berarti hanya SharedViewModel itu sendiri yang dapat mengubah nilainya, sementara kelas lain hanya dapat membacanya. Ini adalah praktik enkapsulasi yang baik.
 - **Baris 18: fun selectMovie(movie: Movie)** Sebuah fungsi publik yang dipanggil untuk memperbarui nilai selectedMovie.
- **Berkas: MovieViewModelFactory.kt (Pabrik ViewModel)**
 Kelas ini bertanggung jawab untuk membuat instance dari MovieViewModel, khususnya untuk menyediakan dependensi MovieRepository yang dibutuhkan oleh konstruktornya.
 - **Baris 8: class MovieViewModelFactory(...) : ViewModelProvider.Factory** Mendeklarasikan kelas MovieViewModelFactory yang mengimplementasikan *interface* ViewModelProvider.Factory.
 - **Baris 11: override fun <T : ViewModel> create(modelClass: Class<T>): T** Meng-override metode create, yang merupakan metode inti dari *factory*. Metode ini akan dipanggil oleh sistem ketika sebuah ViewModel perlu dibuat.
 - **Baris 13: if (modelClass.isAssignableFrom(MovieViewModel::class.java))** Memeriksa apakah kelas ViewModel yang diminta adalah MovieViewModel atau turunannya.
 - **Baris 15: return MovieViewModel(repository) as T** Jika pemeriksaan berhasil, sebuah *instance* baru MovieViewModel dibuat dengan meneruskan repository sebagai argumen, kemudian di-*cast* ke tipe generik T.
 - **Baris 18: throw IllegalArgumentException("Unknown ViewModel class")** Jika kelas yang diminta bukan MovieViewModel, sebuah pengecualian dilemparkan untuk menandakan kesalahan konfigurasi.

3. Lapisan Antarmuka Pengguna (UI Layer)

Lapisan ini bertanggung jawab untuk menampilkan data di layar dan menangani interaksi pengguna.

3.1. Navigasi (navigation)

- **Berkas: Screen.kt (Definisi Rute Navigasi)**
 Berkas ini mendefinisikan semua kemungkinan tujuan navigasi dalam aplikasi secara terpusat.
 - **Baris 7: sealed class Screen(val route: String, val title: String)** Menggunakan sealed class untuk membatasi hierarki kelas. Ini memastikan bahwa semua tujuan navigasi yang mungkin harus didefinisikan di dalam berkas ini, mencegah kesalahan pengetikan rute di tempat lain.
 - **Baris 9: object MovieList : Screen("movie_list", "Movies")** Mendefinisikan layar daftar film sebagai sebuah object tunggal. route adalah pengenalan unik untuk navigasi, dan title dapat digunakan untuk UI (misalnya, di *app bar*).
 - **Baris 11 & 13: object Detail, object Bookmark** Mendefinisikan layar Detail dan Bookmark dengan cara yang sama.
- **Berkas: AppNavigation.kt (Grafik Navigasi Utama)**
 Composable ini mengatur NavHost tingkat atas.

- **Baris 12:** `val navController = rememberNavController()` Membuat dan mengingat sebuah `NavController` yang akan bertahan selama *Composable* berada dalam komposisi.
- **Baris 15:** `NavHost(navController = navController, startDestination = Screen.MovieList.route)` Menginisialisasi `NavHost` yang merupakan kontainer untuk semua tujuan navigasi. `startDestination` menetapkan layar mana yang akan ditampilkan pertama kali.
- **Baris 17:** `composable(Screen.MovieList.route) { ... }` Mendefinisikan sebuah tujuan dalam grafik navigasi. Blok *lambda* ini akan dieksekusi ketika `navController` bernavigasi ke rute `Screen.MovieList.route`.
- **Baris 20:** `MainScreen(...)` Menempatkan `MainScreen` sebagai konten untuk rute awal, dengan meneruskan semua `ViewModel` dan `NavController` yang diperlukan.

3.2. Komponen dan Layar UI (ui/theme)

- Berkas: `MovieListItem.kt` (Item dalam Daftar)

`Composable` ini mendefinisikan tampilan untuk satu baris item film.

- **Baris 19:** `@Composable fun MovieListItem(...)` Mendeklarasikan fungsi *Composable* `MovieListItem` yang menerima tiga parameter: objek `movie` untuk ditampilkan, dan dua fungsi *lambda* (`onItemClick`, `onBookmarkClick`) sebagai *event handler*.
- **Baris 25:** `Card(...)` Menggunakan `Card Composable` untuk membungkus item, memberikan elevasi dan batas yang jelas.
- **Baris 29:** `.clickable { onItemClick(movie) }` Menambahkan *modifier* `clickable` ke `Card`, sehingga seluruh area kartu dapat merespons klik dan akan memicu fungsi `onItemClick`.
- **Baris 31:** `Row(...)` Menyusun elemen-elemen di dalamnya (gambar dan teks) secara horizontal.
- **Baris 33:** `AsyncImage(...)` Menggunakan *Composable* `AsyncImage` dari pustaka `Coil` untuk memuat dan menampilkan gambar poster dari URL secara asinkron.
- **Baris 40:** `Column(...)` Menyusun elemen-elemen di dalamnya (judul dan sinopsis) secara vertikal.
- **Baris 43:** `.weight(1f)` Memberikan bobot pada `Column` agar mengisi sisa ruang horizontal yang tersedia di dalam `Row`, mendorong ikon *bookmark* ke tepi kanan.
- **Baris 45-51:** `Text(...)` Menampilkan judul dan sinopsis film, dengan jumlah baris sinopsis dibatasi maksimal 3.
- **Baris 53:** `IconButton(onClick = { onBookmarkClick(movie) })` Membuat sebuah tombol ikon yang akan memicu fungsi `onBookmarkClick` ketika ditekan.
- **Baris 56:** `imageVector = if (movie.isBookmarked) ... else ...` Secara kondisional memilih ikon yang akan ditampilkan: `Icons.Filled.Bookmark` (terisi) jika `isBookmarked` bernilai `true`, dan `Icons.Outlined.BookmarkBorder` (garis tepi) jika `false`.

- Berkas: `MovieListScreen.kt` (Layar Daftar Film)

`Composable` ini bertanggung jawab untuk menampilkan daftar film utama.

- **Baris 16:** `@Composable fun MovieListScreen(...)` Deklarasi fungsi *Composable*.
- **Baris 22:** `val movies by movieViewModel.movies.collectAsState(...)` Mengonversi `Flow` dari `ViewModel` menjadi `State`. UI akan secara otomatis diperbarui setiap kali `Flow` memancarkan daftar baru.
- **Baris 25:** `LazyColumn { ... }` Menggunakan `LazyColumn` yang efisien untuk menampilkan daftar vertikal, karena hanya me-render item yang terlihat di layar.
- **Baris 27:** `items(movies, key = { it.id }) { movie -> ... }` Fungsi `items` dari `LazyColumn`. Parameter `key` yang unik untuk setiap item sangat penting untuk optimisasi performa.

- **Baris 32: `sharedViewModel.selectMovie(selectedMovie)`** Menyimpan film yang dipilih ke dalam `SharedViewModel` saat item diklik.
- **Baris 34: `navController.navigate(Screen.Detail.route)`** Menginstruksikan `NavController` untuk bernavigasi ke rute layar detail.
- **Baris 38: `movieViewModel.toggleBookmark(movieToBookmark)`** Memanggil fungsi di `ViewModel` untuk mengubah status *bookmark*.

- Berkas: `BookmarkScreen.kt` (Layar Daftar Bookmark)

`Composable` ini khusus menampilkan film yang telah di-bookmark.

- **Baris 15: `val bookmarkedMovies by movieViewModel.bookmarkedMovies.collectAsState(...)`** Struktur kode sangat mirip dengan `MovieListScreen`, namun sumber datanya adalah `bookmarkedMovies` dari `ViewModel`.
- **Baris 18-33: `LazyColumn { ... }`** Struktur `LazyColumn` dan items identik dengan `MovieListScreen`, hanya saja beroperasi pada daftar `bookmarkedMovies`.

- Berkas: `DetailScreen.kt` (Layar Detail Film)

`Composable` ini menampilkan informasi terperinci dari satu film yang dipilih.

- **Baris 14: `@Composable fun DetailScreen(sharedViewModel: SharedViewModel)`** Menerima `SharedViewModel` untuk mendapatkan data film yang akan ditampilkan.
- **Baris 16: `val selectedMovie = sharedViewModel.selectedMovie`** Membaca properti `selectedMovie` dari `SharedViewModel`.
- **Baris 19: `if (selectedMovie != null) { ... }`** Melakukan pemeriksaan *null-safety* untuk memastikan data film ada sebelum mencoba menampilkannya.
- **Baris 21-30: `AsyncImage, Spacer, Text`** Menyusun dan menampilkan elemen-elemen UI seperti gambar poster, judul, dan sinopsis dari film yang dipilih.

- Berkas: `MainScreen.kt` (Kerangka Utama Aplikasi)

`Composable` ini membangun struktur visual utama aplikasi menggunakan `Scaffold`.

- **Baris 29: `val internalNavController = rememberNavController()`** Membuat sebuah `NavController` lokal yang khusus digunakan untuk navigasi antar item di `NavigationBar` (Home dan Bookmark).
- **Baris 34: `Scaffold(...)`** Menggunakan `Scaffold` untuk mengimplementasikan tata letak dasar Material Design, dengan slot untuk `topBar` dan `bottomBar`.
- **Baris 35: `topBar = { TopAppBar(...) }`** Mendefinisikan `TopAppBar` (bilah atas) aplikasi.
- **Baris 43: `bottomBar = { NavigationBar { ... } }`** Mendefinisikan `NavigationBar` (bilah navigasi bawah).
- **Baris 50: `items.forEach { screen -> ... }`** Melakukan iterasi untuk setiap screen di dalam daftar items untuk membuat `NavigationBarItem`.
- **Baris 62: `onClick = { ... }`** Menentukan aksi ketika item navigasi diklik, yaitu memanggil `internalNavController.navigate(screen.route)` dengan konfigurasi *back stack* yang benar.
- **Baris 76: `NavHost(...)`** Menggunakan `NavHost` untuk menjadi kontainer bagi layar-layar yang dinavigasi oleh `internalNavController`.
- **Baris 79: `modifier = Modifier.padding(innerPadding)`** Menerapkan `innerPadding` yang disediakan oleh `Scaffold` agar konten tidak tertutup oleh bilah atas dan bawah.
- **Baris 81-89: `composable(...)`** Mendefinisikan grafik navigasi internal yang memetakan setiap rute ke `Composable` layarnya.

4. Titik Masuk Aplikasi (`MainActivity.kt`)

- Berkas: `MainActivity.kt` (Aktivitas Utama)

Ini adalah satu-satunya `Activity` dan merupakan pintu masuk aplikasi.

- **Baris 14: `class MainActivity : ComponentActivity()`** Deklarasi kelas `MainActivity`.

- **Baris 17-25:** `val database by lazy { ... }, val repository by lazy { ... }, val movieViewModel ... by viewModels { ... }, val sharedViewModel by viewModels()` Menginisialisasi semua dependensi utama aplikasi. `by lazy` digunakan untuk database dan repository. `by viewModels` adalah delegasi properti KTX yang menyediakan ViewModel yang terikat pada siklus hidup Activity.
- **Baris 21:** `MovieViewModelFactory(repository)` Menggunakan `MovieViewModelFactory` untuk menyediakan repository ke `MovieViewModel`.
- **Baris 27:** `override fun onCreate(savedInstanceState: Bundle?)` Metode siklus hidup Activity yang dipanggil saat Activity pertama kali dibuat.
- **Baris 29:** `setContent { ... }` Fungsi ekstensi yang mendefinisikan konten UI Activity menggunakan Jetpack Compose.
- **Baris 31:** `Modul5Theme { ... }` Menerapkan tema kustom aplikasi ke seluruh konten di dalamnya.
- **Baris 33:** `AppNavigation(...)` Memanggil *Composable* navigasi tingkat atas untuk membangun UI dan alur navigasi aplikasi.

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Adiiaus/Praktikum-PemrogramanMobile.git>